How to use Polynomially-Hard iO: Turing Machine Obfuscation and More

Jesko Dujmovic* Yao-Ching Hsieh† Abhishek Jain‡ Willy Quach§

August 8, 2026

Abstract

We revisit the notion of PViO [Jain-Jin, FOCS'22] – an indistinguishability obfuscation (iO) scheme for Turing machines with unbounded input length that guarantees security for pairs of machines whose equivalence can be proven in Cook's Theory PV.

Known constructions of PViO require subexponentially-hard iO for circuits. We give the first construction based on *polynomially*-hard iO and other standard assumptions. We further show how to replace iO with EFiO – an efficiently falsifiable variant, thus obtaining a construction based on efficiently falsifiable assumptions.

Central to our result is a new twist to the celebrated punctured programming technique [Sahai-Waters, STOC'14], where one can program an obfuscated probabilistic function on its entire input domain in *one shot* instead of an input-by-input manner. Our key ingredient is the notion of function secret sharing [Boyle-Gilboa-Ishai, EUROCRYPT'15]. We further show the versatility of our technique by removing the use of complexity-leveraging in two applications of iO: unleveled fully homomorphic encryption, and adaptively-sound succinct non-interactive arguments for "trapdoor" languages.

*Northeastern University, mail@ind-jesko.net
†University of Washington, ychsieh@cs.washington.edu
‡NTT Research and Johns Hopkins, abhishek.jain@ntt-research.com
§CISPA, willy.quach@cispa.de

Contents

1 Introduction

Indistinguishability obfuscation (iO) [BGI+01, GGH+13] guarantees that the obfuscations of any two functionally equivalent programs are computationally indistinguishable. Following a long line of work, iO was recently realized from well-studied assumptions [JLS21, JLS22, RVV24]. Furthermore, a parallel line of work over the last decade has established iO as a seeming panacea that can enable most cryptographic applications.

A common limitation of all existing constructions of iO for general-purpose computations is that they employ a security reduction that incurs a loss exponential in the input length of the program. This means that they can only handle programs with an a priori bounded input length and require assumptions that are $2^{|x|}$-hard for programs of maximum input length $|x|$. As a consequence, the size of the obfuscated program grows with $|x|$.

Prior works [GGSW13, LZ21, JJ22] have observed that this limitation might be inherent. Intuitively, since the iO security guarantee only holds for functionally equivalent programs, the security reduction must be able to decide whether the (adversarially chosen) programs are equivalent. A natural way to check equivalence is by iterating over all inputs to the programs one by one. This brute-force strategy (employed in prior works) results in a $2^{|x|}$ security loss.

iO via Mathematical Proofs of Equivalence. Jain and Jin [JJ22] presented an approach to overcome this barrier by leveraging mathematical proofs of equivalence. They consider the following relaxed notion of security that we refer to as PViO: obfuscations of two programs are computationally indistinguishable if the programs are functionally equivalent, and, moreover, their equivalence can be proven in a proof system called Theory PV [Coo75]. Intuitively, since a PV proof of equivalence is an efficient witness to attest to the equivalence of two programs, the notion of PViO is not subject to the aforementioned barriers. Although not all functionally equivalent programs admit such proofs of equivalence, recent work has demonstrated that many applications of iO can be obtained from PViO when used in combination with cryptographic primitives whose basic properties (such as correctness) can be proven in PV [JJ22, MDS25, ABG+25, JJMP25].[1]

Jain and Jin presented a construction of PViO for Turing machines with *unbounded* input length. Their key insight is that a PV proof of equivalence can be used to devise an efficient security reduction that incurs only a polynomial loss in the input length. Unfortunately, their construction still relies on *sub-exponential* hardness assumptions. Specifically, they construct PViO assuming the sub-exponential hardness of iO for circuits and of learning with errors (LWE), where the sub-exponential function depends (only) on the security parameter of the scheme.

This raises the following question:

Can we build PViO from polynomial hardness assumptions?

1.1 Our Results (I)

Our main result is a positive resolution to this question. We present a construction of PViO for Turing machines of unbounded input length relying only on the *polynomial hardness* of iO for circuits of fixed polynomial size, of LWE and of time-lock puzzles [RSW96]. We further show how to weaken the requirement on iO to achieve a stronger result based only on efficiently falsifiable

[1] This is analogous to how iO can often be combined with other cryptographic primitives to handle programs that are not functionally equivalent [SW14].

assumptions [Nao03].[2] We expand upon our results in the following.

Theorem 1.1 ((Informal) PViO from Polynomial Hardness)**.** *There exists a PViO scheme for all Turing machines under the polynomial hardness of the following assumptions:*

- *iO for circuits of size at most $p(\lambda)$, where $p(\cdot)$ is a fixed polynomial and λ is the security parameter.*
- *Learning with errors (with sub-exponential modulus-to-noise ratio), and time-lock puzzles.*

Time-lock puzzles are known from various assumptions such as the polynomial hardness of either repeated squaring [RSW96], or iO and non-parallelizing languages [BGJ+16], or non-parallelizing languages, ring LWE and decomposed LWE [AMR25].

In fact, it suffices to use relaxed time-lock puzzles [BGJ+16], a weaker variant of time-lock puzzles secure against *time-bounded* adversaries instead of *depth-bounded* adversaries. These in turn are implied by iO and the assumption that non-uniformity doesn't provide arbitrary polynomial speed-up for uniform polynomial computation [BS23].

One-Shot Programming. Central to our approach is a new twist on the celebrated punctured programming technique of [SW14] that is widely used in the iO literature. Recall that the standard iO security guarantee only holds for deterministic circuits. However, when building new applications from iO, we often need to obfuscate probabilistic circuits. The standard approach to achieve this goal is to obfuscate a deterministic circuit that generates randomness using a pseudorandom function (PRF). To argue security, we modify the program *input-by-input* using the punctured programming technique. This, however, incurs a security loss proportional to the input domain size, thus necessitating the use of sub-exponential hardness assumptions. This is in fact one of the key reasons behind the use of sub-exponential hardness assumptions in [JJ22].

To avoid this security loss, we devise a *one-shot programming* technique wherein we change the functionality of the obfuscated program on the entire domain at once. Our key ingredient is the notion of *function secret sharing* (FSS) [BGI15] satisfying a strong form of correctness[3]. In a nutshell, FSS allows two parties, given respective function shares, to locally derive additive shares of any desired output of the function. Moreover, FSS can be built from the *polynomial hardness* of LWE. We believe that this technique is of broader interest, since many applications of iO in the literature require an input-by-input puncturing argument. To support this view, we present two additional applications of our technique; see Section 1.2.

A Stronger Result from Efficiently Falsifiable Assumptions. Theorem 1.1 relies on iO for circuits of size polynomial in the security parameter. This assumption, however, is not efficiently falsifiable [Nao03], for the reasons discussed earlier. We show how to weaken this requirement to obtain a stronger result based on efficiently falsifiable assumptions.

To formally state our result, we first recall the notion of EFiO, a relaxation of iO for circuits introduced in [JJ22]. Informally, a circuit obfuscation scheme is said to achieve EFiO security if it guarantees indistinguishability for any pair of circuits whose equivalence can be proven in the extended Frege propositional proof system [CR79]. Intuitively, EFiO can be viewed as a circuit analogue of PViO which is defined for Turing machines. Cook's propositional translation theorem

[2] We say that a hardness assumption is *efficiently falsifiable* if the challenger in the security experiment runs in polynomial time, and can decide in polynomial time whether an adversary wins. For iO, the latter requirement amounts to checking whether two circuits are functionally equivalent.

[3] FSS evaluation for a large enough class of computations is known to be a PRF [BGI15].

[Coo75, CR79] shows that a proof of equivalence of a pair of circuit families in theory PV can be translated into proofs of circuit equivalence on each input length in the extended Frege system.

Unlike standard iO for circuits, EFiO is an efficiently falsifiable assumption thanks to the propositional proof of equivalence. We show that it suffices to use EFiO instead of iO in Theorem 1.1. Thus, we obtain PViO from efficiently falsifiable assumptions.

Theorem 1.2 ((Informal) PViO from Efficiently Falsifiable Assumptions)**.** *There exists a PViO scheme for all Turing machines under the same assumptions as Theorem 1.1, except that we replace iO with EFiO.*

To achieve this result, we rely on a careful use of cryptographic tools that admit proofs of their key properties (such as perfect correctness) in the extended Frege system, and which can be instantiated from LWE. As it turns out, no known construction of FSS – a key component in our scheme – admits such a proof. At a high level, this is because all known constructions only achieve correctness with overwhelming probability. To this end, we propose a new method to transform a function secret sharing with approximate correctness – here in the sense that additive reconstruction of the FSS is only up to a bounded additive error – into one that achieves *perfect* correctness. The key ingredient in this transformation is the notion of correlation-intractable (CI) hashing [CGH98]. Approximately correct FSS [DHRW16, PS18, ACK23] as well as CI hashing [CCH+19, PS19] are known from the polynomial hardness of LWE. We believe that this technique is of independent interest and is likely to be relevant for future work on PViO.

1.2 Our Results (II)

We propose our one-shot programming technique, based on function secret sharing, as a versatile proof technique of independent interest. We provide evidence by showing how to use this technique for the following applications of iO.

Fully Homomorphic Encryption. First, we show a construction of fully homomorphic encryption (FHE) relying on polynomial hardness assumptions.

Theorem 1.3 ((Informal) FHE from Polynomial Hardness)**.** *There exists an (unleveled) fully homomorphic encryption scheme under the polynomial hardness of the following assumptions:*

- iO *for circuits (of size at most $p(\lambda)$, where $p(\cdot)$ is a fixed polynomial and λ is the security parameter).*
- *Function secret sharing, time-lock puzzles, and Decisional Diffie-Hellman.*

Previously, iO-based constructions were only known under either the sub-exponential security of iO and of other additional assumptions [CLTV15], or under the polynomial security of iO and the *exponential* hardness of DDH [ACH20]. Both these works used an input-by-input argument; in contrast, we use our one-shot programming technique. Our construction additionally relies on a special-purpose encryption scheme, which can be instantiated from DDH.

Here again, by carefully instantiating our cryptographic tools, we show that it suffices to use EFiO instead of iO in Theorem 1.3.

Theorem 1.4 ((Informal) FHE from Efficiently Falsifiable Assumptions)**.** *There exists an unleveled FHE scheme under the following assumptions:*

- *EFiO.*

- *Function secret sharing, time-lock puzzles* [4]*, Decisional Diffie-Hellman.*
- *Correlation intractable hashing for the FSS-correctness-error relation (LWE with superpolynomial modulus-to-noise ratio).*

Adaptively-sound SNARGs for Trapdoor Languages. Our second application is to succinct non-interactive arguments (SNARGs) – a non-interactive, computationally-sound proof system in the common reference string model with proofs shorter than the statement and witness size. We present a construction of *adaptively-sound* SNARGs for trapdoor languages based on *polynomial hardness assumptions*. Trapdoor languages [CGKS23, DJJ+25] are a subclass of NP where there exists a (hard to compute) trapdoor to efficiently decide the language.

Our construction is a simple variant of the Sahai-Waters [SW14] SNARG for NP based on iO and pseudorandom functions (PRFs). Their original construction only achieves non-adaptive soundness. We prove that it, in fact, satisfies adaptive soundness for trapdoor languages when the PRF is instantiated with FSS.

Theorem 1.5 (Informal)**.** *The Sahai-Waters construction is an adaptively sound SNARG for trapdoor languages when the PRF is instantiated with an FSS evaluation.*

Previously, such a result was only known from *sub-exponential* hardness of iO and of one-way functions [WW24, WZ24, WW25].

Our One-Shot Programming Technique via Function Secret Sharing. Finally, we briefly discuss our core one-shot programming technique. In a nutshell, we use FSS to change the functionality of the obfuscated program on every input. Thus, we rely on FSS *with perfect correctness* (over the randomness of the function shares), and this functionality change needs to be supported by the FSS, which depends on the exact application. In whole generality, one could rely on such FSS for all (bounded-depth) circuits, which can be instantiated from LWE (with sub-exponential modulus-to-noise ratio).

However, given a specific application, one can hope to weaken the expressivity required from the FSS. For example, in our application to FHE, the one-shot programming roughly corresponds to decryption of a special-purpose encryption scheme. Appropriately instantiating the latter allows us to rely on FSS for NC^1 instead, thus enabling our technique to be instantiated from the decisional composite residuosity (DCR) assumption [OSY21, RS21].

Future Directions. We leave the following fascinating research directions for future work.

- (EFiO from Standard Assumptions.) Our work shows that PViO can be built from (weaker) *polynomial hardness assumptions* (Theorems 1.1 and 1.2). Our construction relies on EFiO for small circuits, which is an efficiently-falsifiable assumption. It remains open, however, to construct EFiO from the polynomial hardness of standard assumptions, for instance the ones known to imply general-purpose iO [JLS21, JLS22, RVV24].
- (Probabilistic iO from Polynomial Assumptions.) Our one-shot programming technique opens the possibility of recovering applications of so-called *probabilistic iO* (piO) [CLTV15], from polynomial hardness assumptions. In this work, we achieve this in the specific case of FHE (Theorems 1.3 and 1.4). Our current techniques require algebraic structure from

[4] Again, relaxed time-lock puzzles suffice.

the probabilistic program. Extending our techniques to handle more general probabilistic programs is an exciting avenue of future research.

2 Technical Overview

We give here a broad overview of our results and techniques. The first part of this overview is dedicated to improving the construction of PViO from [JJ22]:

- We start by providing a brief summary of the relevant parts of [JJ22] in Section 2.1.
- Next, we present our one-shot programming technique in Section 2.2. Readers interested in our techniques independently of PViO may jump directly to this section.
- Finally, we present our improved construction of PViO in Section 2.3.

In the second part of this overview, we present more applications of our techniques, notably to fully homomorphic encryption, in Section 2.4.

FSS vs Shiftable PRFs. As discussed earlier, a key ingredient underlying our techniques is the notion of function secret sharing (FSS). For convenience of presentation, throughout this overview, we describe our technical ideas through the lens of *shiftable PRFs* [PS18, LV22], which can be viewed as a strengthening of FSS. We emphasize, however, that FSS suffices for our final claims; see Section 2.5.

2.1 Background on iO via Propositional Proofs of Equivalence

As discussed in the introduction, Jain and Jin [JJ22] introduced the notion of PViO, a relaxation of iO for Turing machines that guarantees security for pairs of programs with short proofs of equivalence in theory PV. Their construction relies on *inefficiently falsifiable* assumptions, both in the form of relying on sub-exponential hardness assumptions, and relying on general-purpose iO for circuits. Addressing these drawbacks will be the focus of the present section, Section 2.2, and Section 2.3.

We start with an overview of the [JJ22] construction, in order to highlight the sources of the drawbacks. Looking ahead, we will mostly improve on their proof techniques, and only lightly modify their construction.

The [JJ22] Template. The construction of [JJ22] goes in two steps. First, it introduces a new template to build iO for *circuits*. Then, the latter is upgraded to an obfuscation for Turing machines. We start with the first step, which captures most of the technical challenges we need to overcome; we address the second in Section 2.3.

At a high level, the core of the first step is to obfuscate circuits *gate-by-gate*, meaning that the obfuscated circuit consists of a collection of obfuscated gates. Independent of the use of obfuscation, it is crucial for security that (1) the contents of the original internal wires are kept private, and (2) the internal wires between obfuscated gates are authenticated, in order to prevent mix-and-match attacks that attempt to feed as input to an obfuscated gate an unrelated circuit wire, say coming from a non-adjacent obfuscated gate or an execution of the program on a different input. Thus, [JJ22] compiles every gate g from the original circuit into a circuit C_g, with input and output being encryptions of the original internal wires, along with authentications of these ciphertexts with respect to both the wire location and the execution at hand. These gates are then obfuscated using a general-purpose iO for circuits.

Their main idea is that if the size of these authentications is short, then so are the resulting circuits C_g and thus the obfuscated circuit $\{\mathsf{iO}(C_g)\}$. However, the authentication part must be of

size at least $\omega(\log(\lambda))$ for security (otherwise it could be broken by brute force). Thus, in [JJ22], *the input size of these obfuscated gates grows with the security parameter*.

The security proof of [JJ22] proceeds as follows. Assume for notational convenience that these obfuscated gates $\mathsf{iO}(C_g)$ have input size λ. The proof switches the behavior of an obfuscated gate using a standard puncturing technique, which goes through one hybrid per possible input to $\mathsf{iO}(C_g)$. This translates to the use of 2^λ-secure iO for circuits and puncturable PRFs.

Summing up so far, there are two sources behind the efficient unfalsifiability of assumptions used in the [JJ22] construction (for circuits):

(1) The puncturing technique, which is iterated over all possible 2^λ inputs to the obfuscated gates $\mathsf{iO}(C_g)$, which incurs a large security loss;
(2) The use of general-purpose iO for circuits with λ-bit inputs, in order to obfuscate C_g, which is not efficiently falsifiable in general.

2.2 One-Shot Programming via Shiftable PRFs

We first tackle issue (1). We provide a tighter security proof while keeping the construction for circuits nearly syntactically identical to the one of [JJ22]. To demonstrate our main technical tool for tighter reductions, we look at toy versions of encryption and authentication functionalities. Looking ahead, we introduce our new technique from the perspective of *shiftable PRFs*. We refer to Section 2.5 for how to make this technique rely on the weaker notion of function secret sharing instead.

Obfuscating Encryption. In order to introduce and illustrate our main technical idea, we first look at obfuscating a circuit implementing encryption. Suppose we wish to obfuscate the following circuit:

Circuit Enc_P

Hard-coded: A PRF key K, a program $P : \{0,1\}^n \to \{0,1\}$.
On input $x \in \{0,1\}^n$, output:

$$\mathsf{PRF}_K(x) + P(x).$$

In words, on input x, Enc_P computes a message $P(x)$, for an arbitrary hard-coded, private program P, and encrypts it under a symmetric encryption key K. Intuitively, in the [JJ22] construction, one can think of x as an authenticated encryption of an input wire under key (say) K', and $P(x)$ computing (given the key K') the next unencrypted output wire. Our goal is to prove that obfuscating Enc_P using iO hides P.

This is typically done using the celebrated puncturing technique of [SW14]. Namely, obfuscating Enc_P using an iO scheme and instantiating the PRF in Enc_P by a puncturable PRF, the standard proof that, for all (efficient) programs P, P', the obfuscated programs $\mathsf{iO}(\mathsf{Enc}_P)$ and $\mathsf{iO}(\mathsf{Enc}_{P'})$ are indistinguishable, goes as follows. For any fixed input x^*, one can puncture the PRF on x^* (by puncturable PRF security) and hard-code, on input x^*, a fresh random output r^* instead (by iO security). Doing so allows one to undetectably switch, for any input x^*, from outputting $\mathsf{PRF}_K(x^*) + P(x^*)$ to outputting r^*, and then to $\mathsf{PRF}_K(x^*) + P'(x^*)$. Iterating this argument over all possible input x^* shows the claimed indistinguishability.

A Tighter Proof using Shiftable PRFs. The central bottleneck in the proof above is that puncturable PRFs only allow changing the output on a single input x^* at a time, which in turn induces a $2^{|x|}$ security loss overall.

Our idea to avoid this loss is very natural: we would like to rely instead on a PRF which allows us to undetectably modify exponentially many outputs at once. Observe that Enc_P implements a "shifted PRF evaluation". From this perspective, in terms of security, we would like the ability to undetectably switch the "shift" from $P(x)$ to another program $P'(x)$, using only polynomial hardness.

This very object, that we call a shiftable PRF, turns out to have been previously studied! (In the literature, this was called a shift-hiding shiftable function [PS18, LV22].) In a nutshell, shiftable PRFs are a structured form of constraint-hiding constrained PRFs. In a shiftable PRF, one can generate a master PRF key K that can be used to compute $x \mapsto \mathsf{PRF}_K(x)$. Moreover, one can generate so-called shifted keys K_P with respect to shifts P, where $P : \{0,1\}^n \to \{0,1\}$ is an efficient program, which instead allow computing $x \mapsto \mathsf{PRF}_K(x) + P(x)$. The main security property of shiftable PRFs is that they are *shift-hiding*, that is, that a shifted key hides its underlying shift.

Importantly, shiftable PRFs for efficient shifts are known to exist assuming the *polynomial hardness* of LWE [PS18].[5] Another interpretation of the previous proof is that shiftable PRFs can be built from sub-exponentially secure iO and sub-exponentially secure one-way functions [LV22].

We now adapt the previous security proof that $\mathsf{iO}(\mathsf{Enc}_P)$ and $\mathsf{iO}(\mathsf{Enc}_{P'})$ are indistinguishable,[6] relying instead on shift-hiding PRFs:

- We first switch the obfuscated program $\mathsf{iO}(\mathsf{Enc}_P)$ to one that uses a shifted key $K_{\mathbf{0}}$ associated with the all-zero shift instead. This has the same functionality as the original program and is therefore indistinguishable from the original $\mathsf{iO}(\mathsf{Enc}_P)$ by iO security.
- We then switch the shift from computing the all-0 function to computing $-P + P'$; this change is undetectable by the shift-hiding property of the shiftable PRF. Note that the output of the obfuscated program on an input x is now $(\mathsf{PRF}_K(x) - P(x) + P'(x)) + P(x)$.
- We now compute the output using the master key as $\mathsf{PRF}_K(x) + P'(x)$, which is undetectable by iO security. This proves the indistinguishability of $\mathsf{iO}(\mathsf{Enc}_P)$ and $\mathsf{iO}(\mathsf{Enc}_{P'})$.

Crucially, we only used a constant number of hybrids, and in particular did not go through an "input-by-input" proof strategy. Instead we modified the behavior of $\mathsf{iO}(\mathsf{Enc}_P)$ on *all* 2^n *inputs*, using a single invocation of the shift-hiding property.

Overall, this allows us to argue that when the PRF is instantiated by a shiftable PRF, $\mathsf{iO}(\mathsf{Enc}_P)$ and $\mathsf{iO}(\mathsf{Enc}_{P'})$ are indistinguishable assuming only the polynomial hardness of iO and polynomial security of the shiftable PRF.

Obfuscating Authentication, and the Hidden Sparse Trigger Technique. Next, we turn to the case of authentication. On a technical level, we provide further evidence that shiftable PRFs are highly synergistic with iO, and in particular with the so-called "hidden sparse trigger" technique [SW14].

[5] We discuss in Section 2.5 how to generalize this proof technique to obtain more instantiations by relying instead on (polynomially-secure) function secret-sharing.

[6] The savvy reader might observe that in order to implement the functionality of Enc_P while preserving privacy of P, using a shiftable PRF on its own is sufficient, without the use of obfuscation. We present the obfuscated variant to keep the parallel to the puncturing technique. For our final construction, keeping the obfuscation does not cause additional assumptions; we require obfuscation for the authentication anyway.

Consider the following circuit to obfuscate. Here, the circuit has a PRF key K and an efficient program P hard-coded.

Circuit V_P

Hard-coded: A PRF key K, a program $P : \{0,1\}^n \to \{0,1\}$.
On input (x, y), output 1 if $P(x) = 1$ and $\mathsf{PRF}_K(x) = y$, and 0 otherwise.

In words, taking the convention that 1 means accepting, and denoting by $\mathbf{1}$ the "always accepting" predicate, $V_{\mathbf{1}}$ corresponds to a standard verification procedure for a signature. That is, it only checks whether $\mathsf{PRF}_K(x) = y$. On the other hand, V_P corresponds to a "constrained" verification procedure that only verifies signatures on "good" inputs x, i.e., if $P(x) = 1$. Our goal is to argue that $\mathsf{iO}(V_{\mathbf{1}})$ and $\mathsf{iO}(V_P)$ are indistinguishable, namely, that obfuscated "constrained verification keys" hide their underlying constraint P.

Intuitively, in the [JJ22] construction, one can roughly think of x as being an authenticated encryption of an input wire, and $P(x)$ verifying the authenticity of x using secret information. At a high level, the security property above allows one to *fully disable unauthorized inputs* from the obfuscated compiled gate $\mathsf{iO}(C_g)$, which is crucial to later rely on iO security; otherwise, there would be no guarantee of functional equivalence over unauthorized inputs, including ones that the adversary might use in a mix-and-match attack.

The standard proof of security for the above goes as follows. Start with $\mathsf{iO}(V_{\mathbf{1}})$. Fix any input x^* such that $P(x^*) = 0$. The first step is to use the puncturing technique: now, on any input of the form (x^*, y), output 1 if $y = r^*$ for a uniformly random hard-coded value r^*. We now disable inputs (x^*, y) using the hidden sparse trigger technique, as follows. We modify the check to output 1 on input (x^*, y) iff $G(y) = G(r^*)$ where G is an injective PRG; this is undetectable by iO security. Now, instead of checking $G(y) = G(r^*)$, we check $G(y) = s$ for a uniformly random s; this follows by PRG security. At this point, if G is sufficiently expanding, then the probability that there exists any y such that $G(y) = s$ is negligible over the random choice of s alone. Therefore one can now output 0 on every input of the form $(x^*, \cdot)$ instead, by relying on iO security. The proof that $\mathsf{iO}(V_{\mathbf{1}})$ and $\mathsf{iO}(V_P)$ are indistinguishable follows by iterating the argument over all x^* such that $P(x^*) = 0$.[7]

As before, this iteration over all possible bad inputs x^* induces in general an exponential loss in the input size. The key source of this loss is that the combination of puncturable PRFs and hidden sparse trigger technique allows us to disable only a single bad input at a time.

We show that via a careful use of shiftable PRFs, we can *disable all the bad inputs in one shot*, thus obtaining a tighter security proof in this setting as well. We modify the previous proof as follows:

- We first switch the obfuscated program $\mathsf{iO}(V_{\mathbf{1}})$ to one that uses a shifted key K_0 for the all 0 shift. Doing so is indistinguishable by iO security.
- We change the shifted key, from one computing the all 0 shift to one computing $x \mapsto (P(x)-1)\cdot R$ for a fixed, uniformly random R. In other words, the original PRF output is now shifted by $-R$ iff $P(x) = 0$. This is undetectable by the shift-hiding property of the PRF. The output of the obfuscated program on input (x, y) is now 1 iff $\mathsf{PRF}_K(x) - (1 - P(x)) \cdot R = y$.
- We change how the check is computed. The obfuscated program now uses the unshifted, master PRF key K, and outputs 1 iff $G(\mathsf{PRF}_K(x) - y) = G((1 - P(x)) \cdot R)$, where G is an injective PRG. This is undetectable by iO security.

[7] As mentioned in Footnote 6, for authentication we require the obfuscation. Here, if the adversary learned the output of the PRF — shifted or not — any security would break down.

- We now switch the check whenever $P(x) = 0$ to instead output 1 iff $G(\mathsf{PRF}_K(x) - y) = s$ for a uniformly random s hard-coded in the obfuscated program; this follows by PRG security with seed R and iO.
- The obfuscated program now always outputs 0 whenever $P(x) = 0$. This is indistinguishable by iO whenever s is not in the range of the PRG G, which holds with overwhelming probability over the random choice of s, as long as G is expanding enough. This program now corresponds to V_P, which finishes the proof.

Here again, relying on shift-hiding allowed us to change the behavior of the PRF on exponentially many inputs at once. An important additional mechanism that we use here is that, by carefully correlating the shift over all the different inputs, we managed to disable exponentially many outputs of the obfuscated circuit by arguing that a *single* random value s hard-coded in the program is outside of the range of the PRG.

In typical applications however, the indistinguishability statement above would not suffice, as adversaries would also have access to valid signatures. Thankfully, the proof above naturally extends when additionally giving out an associated constrained signing program, which provides signatures for any authorized inputs with respect to P, computed by the following circuit:

Circuit E_P

Hard-coded: A PRF key K, a program $P : \{0,1\}^n \to \{0,1\}$.
On input x, output $y = \mathsf{PRF}_K(x)$ if $P(x) = 1$ and $\perp$ otherwise.

Namely, one can show that $\mathsf{iO}(V_1)$ and $\mathsf{iO}(V_P)$ are indistinguishable even given $\mathsf{iO}(E_P)$. The proof is almost identical, the only differences being that (1) in the first step, all PRF keys get shifted, and (2) in the third step, we additionally rely on iO to switch back the shifted key used in $\mathsf{iO}(E_P)$ to the master PRF key (which is needed to argue that, at that point, the only information the adversary has on R is $G(R)$).

Overall, we abstract this construction as a constrained signature satisfying constraint-hiding for a rich class of predicates P, assuming the polynomial hardness of iO, of a shiftable PRF, and of an injective PRG. We refer to Section 4.2 for more details.

2.3 Wrapping Up the Construction of PViO

We describe here our improved construction of PViO from polynomial hardness assumptions, and further, how to rely only on efficiently falsifiable assumptions.

EFiO from Polynomial Hardness. Armed with our new technical tools, we now return to the [JJ22] template for circuits. We use shiftable PRFs instead of puncturable PRFs to instantiate the authenticated encryption components of the compiled gates C_g. Then, a direct combination of the techniques above and the original proof of [JJ22] yields a construction of iO for circuits with efficient proofs of equivalence, namely EFiO, assuming the polynomial hardness of iO, of shiftable PRFs, and of other building blocks implied by the polynomial hardness of LWE. This improves on [JJ22], which instead relied on sub-exponentially secure iO, sub-exponentially secure one-way functions, and the same other building blocks. As a side note, we stress that both results only require iO for circuits with small input size $\mathsf{poly}(\lambda)$ for a fixed poly (the polynomial depending on the other building blocks), independent of the actual input size of the original circuit (up to polylogarithmic factors). We refer to Section 5.1 for more details.

PViO from Polynomial Hardness via Time-Lock Puzzles. Next, we give a brief sketch on how to extend the above construction to Turing machines. Intuitively, in [JJ22], this is done by (implicitly) releasing super-polynomially many circuit obfuscations, one per possible input size. Evaluation is performed by obtaining, given an input, the circuit obfuscation corresponding to that input size (using the construction of EFiO above), and evaluating this obfuscated circuit. However, given two Turing Machines, security is argued by showing that *every pair* of circuit obfuscations, one pair per input size, are indistinguishable. This in turn results in [JJ22] relying on super-polynomially secure EFiO, and thus on super-polynomial hardness.

We observe that one can substantially reduce this loss using time-lock puzzles: instead of releasing the (implicit representations of) circuit obfuscations in the clear, we encrypt them using time-lock puzzles. In more detail, we encrypt (the implicit descriptions of) all the obfuscated circuits corresponding to input sizes up to 2^i in a single time-lock puzzle with time bound $T_i = 2^i$. This is done up to an input size bound $N = 2^\lambda$. Evaluation on input x naturally proceeds by first decrypting the circuit obfuscations supporting the input from the time-lock puzzle, which has time bound $T_{\lceil \log |x| \rceil} \leq 2|x|$ and is efficiently solvable for polynomial-size inputs, and then evaluating the resulting EFiO.

In the security proof, we can argue that for any adversary running in time T, the obfuscated circuits corresponding to input size larger than T are hidden by the time-lock puzzles. Only λ invocations of time-lock puzzle security are necessary for this proof step. Now, all but at most $\approx T$ obfuscated circuits are hidden from the adversary. We argue indistinguishability using $\approx T$ invocations of EFiO security. Since T is a polynomial, these steps only introduce a polynomial security loss.

As observed in [BS23] in a similar setting, it turns out the full power of TLPs is not required; a notion called relaxed TLP is sufficient. Relaxed TLPs place a bound on the size of the adversary instead of its depth. Further, [BS23] show how to build relaxed TLPs from iO and worst-case complexity assumptions.

Putting everything together, we obtain overall a construction of PViO (for Turing machines) assuming the polynomial hardness of iO (for small circuits), of LWE, and of (relaxed) time-lock puzzles. We refer to Section 5.2 for more details.

Tackling the Non-falsifiability of iO via Propositional Proofs, Again. So far, we spent most of this overview discussing how to resolve Issue (1), namely the loss of security induced by the puncturing arguments, along with the extension to Turing machines discussed above.

We now discuss Issue (2). Note that our construction of PViO discussed above relies on general-purpose iO for circuits of input size $\mathsf{poly}(\lambda)$, which is not an efficiently falsifiable assumption. We show how to replace iO with EFiO – still for circuits with small input size $\mathsf{poly}(\lambda)$ – whose security is efficiently falsifiable. To achieve this goal, it is necessary to ensure that we require iO security only for pairs of circuits that admit EF proofs of equivalence. In our context, for circuits hardcoding randomized elements (e.g., a ciphertext), we require a proof of equivalence for every possible randomness for these hardcoded elements.

To this end, we build upon observations from prior work ([JKLM25, Mat25]) that show how to formalize basic properties of many crypto systems in the $\mathcal{EF}$ proof system. A key challenge in our setting, however, is that the building blocks used in our one-shot programming technique are not known to admit EF proofs of their key properties. We focus on shiftable PRFs in this overview and defer discussion on other primitives to Section 6.

In our one-shot programming technique, it is crucial that the shifted evaluation is correct on *every input*: otherwise the resulting circuits wouldn't be functionally equivalent. Such a strong

correctness property is typically proven through a *counting argument*, which shows that "most" of the random setups provide correctness. Such a guarantee is insufficient for applying EFiO, where we need a proof for every fixed "valid setup". To tackle the issue, we provide a new construction of a shiftable PRF with a propositional proof of *perfect correctness* for shifted evaluation. Our construction relies on the notion of (somewhere statistically) *correlation-intractable* (CI) hash functions [CGH98, CCH+19, PS19]. Such schemes support hash keys hk_C hiding some relation C, with the guarantee that for all inputs x, $\mathsf{H}(\mathsf{hk}, x) \neq C(x)$.

Our starting point is a form of *approximately correct* shiftable PRF from [PS18], where instead of requiring the shifted evaluation to be exactly correct on most randomness, it only requires the evaluation output to be correct *up to some small additive error*, but on *all* randomness. As pointed out in [PS18], one can cast this form of approximate correctness to standard correctness via rounding, at the cost of introducing a possible rounding error whenever the evaluation is close to the rounding boundary.

We observe that one can avoid such rounding errors with the aid of somewhere statistically CI hash functions. In particular, we shift the evaluation by the CI hash value before rounding. In this case, rounding errors only occur for "bad" hash values that are close to the PRF evaluation modulo the rounding modulus. With appropriate parameters, the bad hash value can be uniquely computed for every input given the master PRF key. Therefore, if the hash is set up to be *statistically CI* in the actual construction over the above "bad" relation, no input will lead to rounding errors, resulting in a perfectly correct scheme. Even though the CI hash key depends on the master PRF key, security of the PRF is preserved as CI hash keys computationally hide their underlying statistical correlation. Finally, both this form of approximate correctness of the shiftable PRF from [PS18] and the statistical CI of the CI hash from [PS19] are provable in PV: they both boil down to the correctness of lattice homomorphic evaluations, and many recent works have shown how to give PV proofs of correctness for (different variants of) such homomorphism [JKLM25, Mat25].

2.4 More Applications of Shiftable PRFs and iO

We now shift gears, and depart from our previous goal of building iO for circuits or Turing machines with propositional proofs of equivalence. Rather, we propose shiftable PRFs as a powerful and possibly general tool to obtain tighter versions of several known applications of iO, typically in settings where known security proofs are loose due to an input-by-input argument.

Additional Application 1: Fully Homomorphic Encryption. We first turn to *fully homomorphic encryption* (FHE). It is currently known that FHE can be built from sub-exponentially secure iO, one-way functions, and (perfectly) rerandomizable encryption [CLTV15], which famously gives a possible alternative route to FHE that does not rely on lattices. We show how to make their security proof rely on polynomial assumptions, using shiftable PRFs as a core ingredient.

The [CLTV15] Construction for Leveled FHE. To illustrate our techniques, we first focus on the setting of *leveled* FHE. As we will heavily build on the template of [CLTV15], we start with a quick overview of their construction.

For clarity of the exposition, let us focus on building a homomorphic encryption supporting a *single homomorphic* NAND, starting with an iO scheme, a PRF, and a public-key encryption scheme with properties to be determined later. The homomorphic encryption scheme uses two public-secret key pairs, $(\mathsf{pk}_0, \mathsf{sk}_0), (\mathsf{pk}_1, \mathsf{sk}_1)$. The evaluation key for our homomorphic encryption consists of an obfuscation of the following circuit:

Circuit C_{ReEnc}

Hard-coded: $\mathsf{pk}_1, \mathsf{sk}_0$, and a PRF key K
On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, compute $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_0}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_0}(\mathsf{ct}^1))$, and output:

$$\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}_1, b_x \,;\, \mathsf{PRF}_K(x)).$$

In words, C_{ReEnc} parses its input as two ciphertexts under pk_0, decrypts them using its hard-coded key sk_0, and computes a NAND of the resulting plaintexts, resulting in a bit b_x. It then outputs an encryption of b_x under pk_1, using randomness derived by applying a PRF to its input. The secret key for the scheme is sk_1, the public key pk_0, and encryption consists of encrypting under pk_0. Homomorphic evaluation proceeds by evaluating an obfuscation of C_{ReEnc} on the input ciphertexts.

The crux of the security proof is to argue that semantic security holds even given the obfuscated circuit $\mathsf{iO}(C_{\mathsf{ReEnc}})$. To do so, it is sufficient to argue that $\mathsf{iO}(C_{\mathsf{ReEnc}})$ is indistinguishable (given pk_0) from a circuit that does not have sk_0 hard-coded: security would then follow by semantic security under pk_0. Suppose now that the encryption scheme with keys $(\mathsf{pk}_1, \mathsf{sk}_1)$ has a *perfectly lossy* mode, that is, public keys can be indistinguishably set up to a special mode, which we will denote $\overline{\mathsf{pk}}_1$, under which encryptions *perfectly hide the underlying message*. The security proof of [CLTV15] then goes as follows:

- First switch the public key pk_1 to a perfectly lossy key $\overline{\mathsf{pk}}_1$;
- Fix any input $x^* = (\mathsf{ct}^{0^*} \| \mathsf{ct}^{1^*})$ to C_{ReEnc}.
  - Puncture the PRF K on $x^* = (\mathsf{ct}^{0^*} \| \mathsf{ct}^{1^*})$, and modify the circuit C_{ReEnc}, on input $(\mathsf{ct}^{0^*} \| \mathsf{ct}^{1^*})$, to output instead a hard-coded ciphertext: $\mathsf{Enc}\left(\overline{\mathsf{pk}}_1, b_x \,;\, r^*\right)$, where r^* is uniformly random and b_x is computed as before using sk_0. This is indistinguishable assuming puncturable PRF security and iO.
  - By perfect lossiness, the distributions

    $$\mathsf{Enc}\left(\overline{\mathsf{pk}}_1, b_x \,;\, r^*\right), \quad \mathsf{Enc}\left(\overline{\mathsf{pk}}_1, 0 \,;\, r^*\right)$$

    are identical over the randomness of r^*. So hard-coding a sample for the right distribution instead of the left distribution yields identical distributions. Now the output on x^* does not depend on sk_0.
- After iterating the argument over all possible inputs $x^* = (\mathsf{ct}_0^*, \mathsf{ct}_1^*)$, the resulting circuit to be obfuscated does not depend on sk_0 anymore, as it no longer decrypts its inputs and always outputs encryptions of zeros. This finishes the proof.

As in the case of the security proof from [JJ22], the proof above uses an input-by-input argument, each invoking iO and puncturable PRF security, which results in the overall security relying on the sub-exponential security of both iO and puncturable PRF. Again, the technical culprit is the inability for puncturable PRFs to change the behavior of the underlying circuit on more than one input at once.

A Tighter Proof from Shiftable PRFs and Algebraically Rerandomizable Encryption. We naturally turn to shiftable PRFs to attempt to improve the proof above. Doing so allows us to

controllably switch, for all inputs x, the randomness used in the lossy encryption, from using $F_K(x)$ to using $F_K(x) + P(x)$ for an efficient function P, thus outputting $\mathsf{Enc}\left(\overline{\mathsf{pk}}_1, b_x\,;\, \mathsf{PRF}_K(x) + P(x)\right)$. At this stage, we would like to rely on iO to switch this ciphertext to an encryption of the form $\mathsf{Enc}\left(\overline{\mathsf{pk}}_1, 0\,;\, \mathsf{PRF}_K(x) + P'(x)\right)$. It would thus suffice to find efficient functions P and P' such that the two quantities above are equal for all x:

$$\mathsf{Enc}\left(\overline{\mathsf{pk}}_1, b_x\,;\, \mathsf{PRF}_K(x) + P(x)\right) \stackrel{?}{=} \mathsf{Enc}\left(\overline{\mathsf{pk}}_1, 0\,;\, \mathsf{PRF}_K(x) + P'(x)\right). \tag{1}$$

However, while the two encryptions are identically distributed when using uniformly random r^*, meaning that the equality holds for *some* functions P, P', it is not clear at all that such functions should be efficiently computable in general.

Instead, we rely on a special-purpose perfectly lossy encryption to argue that such efficient functions P, P' exist. In order to build such perfectly lossy encryption, we take inspiration from the instantiation in [CLTV15], who build such a standard perfectly lossy encryption starting from a perfectly rerandomizable encryption as follows:[8]

- The public key of the perfectly lossy encryption consists of two rerandomizable ciphertexts, one of message 0, $\mathsf{Rerand.ct}_0$, and one of 1, $\mathsf{Rerand.ct}_1$;
- Encryption of a bit b proceeds by rerandomizing the ciphertext $\mathsf{Rerand.ct}_b$. We will denote this by $\mathsf{Rerandomize}(\mathsf{Rerand.ct}_b)$.
- In lossy mode, both ciphertexts $\mathsf{Rerand.ct}_b$ are instead encryptions of 0.

Lossy public keys are indeed indistinguishable from standard public keys by semantic security of the rerandomizable encryption, and perfect rerandomizability translates to perfect lossiness of the construction.

We observe that in this construction, structure on the rerandomization procedure translates to structure on the resulting lossy encryption. Suppose that the rerandomization procedure works by *shifting* the underlying ciphertext randomness by the randomness of the rerandomization procedure: writing $\mathsf{Rerand.ct} = \mathsf{Rerand.Enc}(b; r)$, this means:

$$\mathsf{Rerandomize}(\mathsf{Rerand.ct}; r') = \mathsf{Rerand.Enc}(b; r + r'). \tag{2}$$

We call rerandomizable encryption schemes satisfying Equation (2) *algebraically rerandomizable*, and observe that they can be built assuming the (polynomial hardness of) DDH: ElGamal satisfies such a property. Using such a rerandomizable encryption to build a perfectly lossy encryption à la [CLTV15], and writing the lossy public key as

$$\overline{\mathsf{pk}}_1 \;=\; (\mathsf{Rerand.ct}_0 = \mathsf{Rerand.Enc}(0; r_0),\; \mathsf{Rerand.ct}_1 = \mathsf{Rerand.Enc}(0; r_1))\,,$$

Equation (1) then becomes:

$$\mathsf{Rerand.Enc}(\mathsf{pk}_1, 0\,;\, r_{b_x} + \mathsf{PRF}_K(x) + P(x)) \stackrel{?}{=} \mathsf{Rerand.Enc}(\mathsf{pk}_1, 0\,;\, r_0 + \mathsf{PRF}_K(x) + P'(x)).$$

We can now define P as follows: given sk_0 and r_0, r_1 hard-coded, on input x, compute b_x using sk_0 and output $r_0 - r_{b_x}$, while P' is defined as the 0 function. This makes Equation (1) hold for all x, which in turn makes the security proof go through assuming only the polynomial hardness of iO, of shiftable PRFs, and of an algebraically rerandomizable encryption (namely, one satisfying Equation (2)).

[8] A perfectly rerandomizable encryption is an encryption scheme, such that any ciphertext can be rerandomized into a perfectly distributed encryption of the same message.

Unleveled FHE via Time-Lock Puzzles, Again. Next, we briefly describe how to extend the leveled construction above to the unleveled setting, as done in [BS23]. First, the construction above directly extends to support any bounded depth d of homomorphism: this is done by considering for all $i < d$ fresh keys $(\mathsf{pk}_i, \mathsf{sk}_i, K_i)$ and an associated circuit C^i_{ReEnc} with $(\mathsf{pk}_{i+1}, \mathsf{sk}_i, K_i)$ hard-coded. The proof proceeds similarly, removing the secret keys sk_i from $C^{d-1}_{\mathsf{ReEnc}}, \ldots, C^0_{\mathsf{ReEnc}}$, in order.

To extend this construction to support an unbounded amount of homomorphism, [CLTV15] compresses the circuits C^i_{ReEnc}, by giving out an obfuscated circuit which on input $i \leq 2^\lambda$ outputs C^i_{ReEnc}. This is extremely similar in spirit to how [JJ22] extended their construction of iO for circuits to iO for Turing machines. Accordingly, the original proof of [CLTV15] proceeded by arguing security for *all* 2^λ (pairs of) circuits.

Unsurprisingly, similar to how we removed the related loss in [JJ22], we show how to remove this one using (relaxed) time-lock puzzles. Summing up, this yields an unleveled FHE assuming the polynomial hardness of iO, of shiftable PRFs, of an algebraically rerandomizable encryption, and of (relaxed) time-lock puzzles. To obtain instead leveled FHE, the time-lock puzzle assumption can be removed.

Application 2: Adaptively Sound SNARGs for Trapdoor Languages. We briefly discuss our second application related to SNARGs. We say that a family of languages in NP is *trapdoored* if one can sample a language along with an efficient trapdoor program that decides it. We show that the SNARG construction of Sahai-Waters [SW14] is, without any modification, adaptively sound if the underlying language is trapdoored and if the underlying PRF is a shiftable PRF. Moreover, adaptive soundness only relies on the polynomial security of iO and of the shiftable PRF.

The construction actually follows directly from our construction of constraint-hiding constrained signature, sketched in Section 2.2: the SNARG prover runs an obfuscation of the circuit E_P, where P authorizes (x, w) only if w is a valid witness for x. The verification algorithm corresponds to an obfuscation of the circuit $V_{\mathbf{1}}$ for the all-accepting constraint (which only verifies the signature, by only taking x as input). Observe that this almost exactly corresponds to the construction of [SW14] instantiated with a shiftable PRF, up to the fact that their construction adds an additional layer of one-way function to verify signatures. We show that the construction is adaptively sound even without this additional layer (but remains sound with it).[9]

The proof of adaptive soundness goes as follows: given a trapdoor decider for the language at hand, we switch how P is computed, relying on the trapdoor instead; this is indistinguishable by iO security. Now, constraint hiding of the signature states that the view of the adversary is indistinguishable from one where no valid proof for unauthorized statements exists, namely ones that are not in the language, and adaptive soundness follows.[10]

2.5 Relaxing Shiftable PRFs to Function Secret Sharing

Last, we observe that the main techniques we presented in this overview do not actually need the full power of shiftable PRFs, and that function secret sharing (FSS) [BGI15] suffices. We first recall the notion of FSS.

In a two-party FSS with additive reconstruction, Alice and Bob are respectively given function shares $\mathsf{sh}_{A,f}, \mathsf{sh}_{B,f}$ with respect to some function f. Given these shares, and any input x, they can

[9] This is essentially because we use injective PRGs *in the proof* to implement a similar check.

[10] We previously only showed how to build constraint-hiding constrained signatures from the polynomial hardness of iO, of shiftable PRFs, and of *injective PRGs*. We show in Appendix A that injective PRGs follow from the polynomial hardness of iO and of one-way functions, and we thus do not require any additional assumptions here.

respectively compute some output $y_A(x), y_B(x)$ locally, such that $y_B(x) - y_A(x) = f(x)$.[11] Security of FSS states that the share $\mathsf{sh}_{A,f}$ (resp. $\mathsf{sh}_{B,f}$) hides the function f.

The notion of FSS is very similar to shiftable PRFs: interpreting the master key of a shiftable PRF and a shifted key as two respective shares directly gives an FSS. Conversely, shiftable PRFs is a strengthening of FSS: the master key of a shiftable PRF is generated independently of the shift f, which is not always possible in FSS.

We observe here that this strong latter property is superfluous for our one-shot programming technique, as long as the "master key" hides the shift, which exactly corresponds to FSS. We illustrate this with the same circuit Enc_P used in Section 2.2: we prove that $\mathsf{iO}(\mathsf{Enc}_P)$ and $\mathsf{iO}(\mathsf{Enc}_{P'})$ are indistinguishable, where PRF evaluation is replaced by local evaluation of an FSS supporting sufficiently expressive computations. We highlight the differences in red:

Circuit Enc_P

Hard-coded: An FSS share $\mathsf{sh}_{A,0}$ for Alice, a program $P : \{0,1\}^n \to \{0,1\}$.
On input $x \in \{0,1\}^n$, output:

$$y_A(x) + P(x).$$

- We first switch the obfuscated program $\mathsf{iO}(\mathsf{Enc}_P)$ to one that uses the Bob share $\mathsf{sh}_{B,0}$ (and the local evaluation for Bob) instead. This has the same functionality as the original program by correctness of the FSS, and is therefore indistinguishable from the original $\mathsf{iO}(\mathsf{Enc}_P)$ by iO security.
- We then switch the function in Bob's share from computing the all-0 function to computing $-P + P'$; this change is undetectable by security of the FSS. So now, the obfuscated program uses the share $\mathsf{sh}_{B,-P+P'}$. The output of the obfuscated program on an input x is now $y_B(x) + P(x) = (y_A(x) - P(x) + P'(x)) + P(x)$.
- We now compute the output using the Alice share $\mathsf{sh}_{A,-P+P'}$, and computing $y_A(x) + P'(x)$ instead, which is undetectable by iO security.
- (New Hybrid) We switch back the Alice share from using $\mathsf{sh}_{A,-P+P'}$ to $\mathsf{sh}_{A,0}$. This follows by security of the FSS, and finishes the proof of indistinguishability of $\mathsf{iO}(\mathsf{Enc}_P)$ and $\mathsf{iO}(\mathsf{Enc}_{P'})$.

One can readily adapt the argument above to all our uses of shiftable PRFs discussed previously. Alternatively, another interpretation of the proof above is that one can build shiftable PRFs from the *polynomial security* of iO and of FSS (for a related class of functions). In any case, this allows us to weaken the assumptions in our theorem statements, from relying on shiftable PRFs to relying on FSS.

We stress that, in order to instantiate our one-shot programming technique, we crucially rely on both (1) additive reconstruction of the FSS (over some group) and (2) correctness on *all* inputs. We do not know how to leverage FSS without these specific properties. Still, in applications where the shift can be computed in NC^1, this allows us to instantiate our one-shot programming technique, assuming the polynomial hardness of DCR [OSY21, RS21], or of DDH over class groups [ADOS22].

Our use of FSS is perhaps not so surprising in hindsight. Above, we improved the punctured programming technique of [SW14] relying on punctured PRFs, by developing a one-shot programming technique relying on shiftable PRFs. Both notions of PRFs are examples of *constrained PRFs*

[11] We use subtractive reconstruction for convenience of exposition and analogy to shiftable PRFs here, but everything readily extends to standard additive reconstruction by introducing appropriate signs.

[BW13, KPTZ13, BGI14]: punctured PRFs are constrained PRFs for (the negation of) point function constraints, and shiftable PRFs are a strong form of *constraint-hiding* constrained PRFs. Meanwhile, FSS and constrained PRFs were previously observed to be related [CMPR23] – constrained PRFs can roughly be thought as a "staged" version of FSS. Similarly, shiftable PRFs are a strengthening of FSS, where one of the shares, namely the master PRF key, does not depend on the function to be computed. One critical difference is that, in our context, shiftable PRFs and FSS are *constraint/function hiding*, whereas previous works on staged FSS heavily leverage that the shared function is public. Still, we find it intriguing that FSS can technically replace shiftable PRFs in the context of iO.

On the Necessary Assumptions for Our One-Shot Technique. One interpretation of our work is that we obtain significantly tighter reductions in the context of iO, by leveraging additional structure out of the underlying PRF. Intuitively, this requires the use of "algebraically structured" assumptions, as illustrated by our instantiations of FSS and shiftable PRFs from (the polynomial hardness of) LWE or DCR.

Curiously, it is also unlikely that such FSS and shiftable PRFs *formally require* structure: as noted in Section 2.2 and by [LV22], they can be built under the *sub-exponential* hardness of iO and of one-way functions, via a standard puncturing argument. While this latter construction is mostly uninteresting for the main purposes of this paper (our objective being constructions from polynomial hardness assumptions), it is quite insightful in terms of complexity of the primitives. Indeed, because this construction is black-box in the sense of Asharov-Segev [AS15], FSS and shiftable PRFs consequently do not imply in a black-box manner either collision resistant hashing [AS15], nor hardness of the complexity class SZK [BDV17]. On the other hand, most concrete "structured" assumptions, such as LWE or DCR, imply either (and most often both).

We stress again that our one-shot programming technique currently relies both on a strong form of correctness ensuring correct evaluation on *all possible inputs*, and on additive reconstruction. We leave the construction of such an FSS from DDH (say over a subgroup of $\mathbb{Z}_q^*$ or elliptic curves) as an open problem.

2.6 Related Work

While iO is now famous for enabling constructions of almost all known cryptographic primitives, many constructions rely on sub-exponential assumptions (e.g. [BZ14, CLTV15, BPR15, BPW16, DHRW16, MT19, SLM+23, WW25]).

Consequently, a parallel line of work explored the possibility of relaxing the use of (sub-exponential) iO to *functional encryption* (FE), for which constructions from efficiently falsifiable assumptions are now known [JLS21, JLS22, RVV24]. These works typically leveraged that, in some applications, iO was only used to obfuscate specific, structured computations [GPS16, GS16, GPSZ17]. This line of work culminated in the works [LZ21, KLMR18], which show how to build a relaxed form of iO from FE, where iO security only holds whenever the circuits share some specific common structure.[12] However, this common structure is very specific and tied to the technical transformation from FE to iO [AJ15, BV15]. In contrast, we believe our one-shot programming technique opens the way to reduce this sub-exponential dependence for *unstructured programs*, such as deterministic encryption of an arbitrary function of the input, or constrained signatures for arbitrary constraints, as discussed in Section 2.2.

[12] In a nutshell, this structure corresponds to natural decision tree representations of the two circuits having large overlaps. This is notably the case whenever the circuits behave *identically* conditioned on small prefixes of the input.

The work of [ACH20] also focused on weakening this required sub-exponential hardness, by relying on extremely lossy functions (ELFs) [Zha16] (and polynomially secure $i\mathcal{O}$). However, it is currently unknown whether ELFs can be instantiated without *exponential* assumptions – currently, the only known construction of ELFs relies on the exponential hardness of DDH.

Prior and concurrent works, e.g. [KWZ22, NW26], also attempted to remove this reliance on sub-exponential security by using constrained PRFs, a relaxation of shiftable PRFs. In our setting, shiftable PRFs offer significantly more expressive control over the programming, which we crucially use in all our applications.

3 Preliminaries

3.1 Function Secret Sharing

Definition 3.1 (Secret-key FSS [BGI15, BGI16, BGI19])**.** *A (two-party) secret-key FSS scheme for a function class $\mathcal{F} = \{F_\lambda\}$ with input space $\{0,1\}^{n(\lambda)}$ and output space $R^{m(\lambda)}$ for some ring R consists of the following algorithms:*

- $\mathsf{Share}(1^\lambda, f \in F_\lambda) \to (\mathsf{sh}_0, \mathsf{sh}_1)$ *takes as input the security parameter and a function f, and outputs two shares $\mathsf{sh}_0, \mathsf{sh}_1$. We use $\mathsf{Share}_0, \mathsf{Share}_1$ to denote the partial functions that output the first and second share respectively.*
- $\mathsf{Eval}(\mathsf{sh}_b, x \in \{0,1\}^n) \to y_b \in R^m$ *takes as input a share sh_b and an input x, and deterministically outputs an evaluation share $y_b \in R^m$.*

We require the FSS to satisfy the following properties:

- **Everywhere Correctness:** *There exists a negligible function* negl *such that for every function $f \in F_\lambda$, it holds that*

$$\Pr\left[\forall x \in \{0,1\}^n : \; y_0 + y_1 = f(x) \;\middle|\; \begin{array}{c} (\mathsf{sh}_0, \mathsf{sh}_1) \leftarrow \mathsf{Share}(1^\lambda, f) \\ \forall b \in \{0,1\}, \; y_b \leftarrow \mathsf{Eval}(\mathsf{sh}_b, x) \end{array}\right] \geq 1 - \mathsf{negl}(\lambda).$$

  We say that the FSS scheme is perfectly correct *if the above holds with probability 1.*

- **Security:** *For every efficient adversary $\mathcal{A}$, there exists a negligible function* negl *such that for every $\lambda \in \mathbb{N}$, every function $f \in F_\lambda$ and every bit $b \in \{0,1\}$,*

$$\left| \begin{array}{l} \Pr\left[\mathcal{A}(1^\lambda, \mathsf{sh}_b) = 1 : (\mathsf{sh}_0, \mathsf{sh}_1) \leftarrow \mathsf{Share}(1^\lambda, f)\right] \\ - \Pr\left[\mathcal{A}(1^\lambda, \mathsf{sh}_b) = 1 : (\mathsf{sh}_0, \mathsf{sh}_1) \leftarrow \mathsf{Share}(1^\lambda, \mathbf{0})\right] \end{array} \right| \leq \mathsf{negl}(\lambda),$$

  where $\mathbf{0}$ is a canonical representation of the all-zeros function.

Remark 3.1 (HSS and FSS)**.** Function secret sharing can be viewed as the dual version of homomorphic secret sharing, where the input is shared and the evaluation takes in a function. For schemes supporting sufficiently expressive functionalities, the two notions can be converted to each other using universal circuits with a slight asymptotic loss in expressiveness. As a direct corollary, any HSS scheme for all bounded-size circuits implies an FSS scheme for all bounded-size circuits, and vice versa.

Remark 3.2 (Everywhere Correctness)**.** We stress that we require a strong form of correctness in Definition 3.1, where (with high probability over the shares) FSS evaluation is correct for *all* inputs x. This is crucial in our applications, where we need the circuit $\mathsf{Eval}(\mathsf{sh}_0, \cdot)$ and $f(\cdot) - \mathsf{Eval}(\mathsf{sh}_1, \cdot)$ to be equivalent in order to invoke iO security.

While correctness can be amplified to everywhere correctness using parallel repetition, this does not preserve the additive reconstruction of the share, which is crucial for our applications.

Therefore, in this work, we only consider FSS schemes which inherently satisfy everywhere correctness.

Theorem 3.1 ([DHRW16, OSY21, RS21, ADOS22])**.** *Assuming the polynomial hardness of LWE (with sub-exponential modulus-to-noise ratio) (resp. of DCR, or of DDH over appropriate class groups), there exists a secret-key, everywhere correct FSS scheme for the function class of all bounded-size circuits (resp.* NC^1*).*

3.2 Puncturable PRF

Definition 3.2 (Puncturable PRF)**.** *A puncturable PRF of key length λ, input length $n = n(\lambda)$, and output length $m = m(\lambda)$ consists of the following efficient algorithms:*

- $\mathsf{Punc}(1^\lambda, k, S = \{x_i\})$ *takes as input a master key $k \in \{0,1\}^\lambda$ and a set of points $S = \{x_i\} \subseteq \{0,1\}^n$ in the input space of the PRF, and outputs a punctured key k_S.*
- $\mathsf{Eval}(k, x)$ *takes as input a key k (either master key or punctured key) and an input $x \in \{0,1\}^n$, and outputs PRF evaluation $y \in \{0,1\}^m$.*

We require the puncturable PRF to satisfy the following properties:

- **Efficiency:** *The size of the punctured key k_S is $|S| \cdot \mathsf{poly}(\lambda)$.*
- **Correctness:** *For every master key k, every set $S = \{x_i\}$, and every punctured key $k_S \leftarrow \mathsf{Punc}(1^\lambda, k, S)$, it holds that*

$$\forall x \notin S, \quad \mathsf{Eval}(k_S, x) = \mathsf{Eval}(k, x).$$

- **Security:** *For every efficient adversary $\mathcal{A}$ and every polynomial $s = s(\lambda)$, there exists a negligible function negl such that for every sequence of polynomial-size sets $\{S_\lambda = \{x_{\lambda,i}\}_{i\in[s]}\}$ of size at most $s(\lambda)$, it holds that for all $\lambda \in \mathbb{N}$,*

$$\left|\Pr\left[\mathcal{A}(1^\lambda, k_S, \{\mathsf{PRF}(k, x_{\lambda,i})\}_{i\in[s]}) = 1\right] - \Pr\left[\mathcal{A}(1^\lambda, k_S, \{y_{\lambda,i}\}_{i\in[s]}) = 1\right]\right| \leq \mathsf{negl}(\lambda),$$

where $k \leftarrow \{0,1\}^\lambda$ is a uniform master key, $k_S \leftarrow \mathsf{Punc}(1^\lambda, k, S_\lambda)$ is the punctured key, and $y_{\lambda,i} \leftarrow \{0,1\}^m$ are uniform random values from the PRF output space.

Following the observations in [BW13, BGI14, KPTZ13], the GGM construction [GGM84] of PRF from one-way functions gives a puncturable PRF.

Theorem 3.2 ([GGM84, BW13, BGI14, KPTZ13])**.** *Assuming the existence of one-way functions, there exists a puncturable PRF with key length λ and any polynomial input and output length.*

3.3 Shiftable PRF

In this section, we revisit the notion of shiftable PRF, originally introduced in [PS18] under the name *Shift-hiding Shiftable Function*. To align with its role as an iO-friendly pseudorandom source in our applications, we adopt the term *Shiftable PRF* in this work.

Definition 3.3 (Shiftable PRF)**.** *A shiftable PRF for a function class $\mathcal{F} = \{F_\lambda\}$ with input space $\{0,1\}^{n(\lambda)}$ and output space $R^{m(\lambda)}$ for some ring R consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{pp}, k)$ *takes as input the security parameter and outputs the public parameter pp and the master key k.*
- $\mathsf{Eval}(\mathsf{pp}, k, x \in \{0,1\}^n) \to y \in R^m$ *takes as input the public parameter pp, the master key k and an input x, outputs PRF evaluation y.*
- $\mathsf{Shift}(\mathsf{pp}, k, f \in F_\lambda) \to k_f$ *takes as input the public parameter pp, the master key k and a function $f : \{0,1\}^n \to R^m$ from the function class F_λ, outputs a shifted key k_f.*

- $\mathsf{EvalShifted}(\mathsf{pp}, k_f, x \in \{0,1\}^n) \to y \in R^m$ *takes as input the public parameter* pp*, the shifted key* k_f *and an input* x*, outputs PRF evaluation* y*.*

We require the shiftable PRF to satisfy the following properties:

- **Everywhere Correctness:** *There exists a negligible function* negl *such that for every* $\lambda \in \mathbb{N}$ *and every function* $f \in F_\lambda$*, it holds that*

$$\Pr\left[\forall x \in \{0,1\}^n,\ \mathsf{EvalShifted}(\mathsf{pp}, k_f, x) = \mathsf{Eval}(\mathsf{pp}, k, x) + f(x)\right] \geq 1 - \mathsf{negl}(\lambda),$$

  where the probability is taken over the randomness of $(\mathsf{pp}, k) \leftarrow \mathsf{Setup}(1^\lambda)$ *and* $k_f \leftarrow \mathsf{Shift}(\mathsf{pp}, k, f)$*. We say that the shiftable PRF scheme is* perfectly correct *if the above holds with probability 1.*

- **Shift Hiding:** *For every efficient adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that for every* $\lambda \in \mathbb{N}$ *and any two functions* $f_1, f_2 \in F_\lambda$*, it holds that*

$$\left|\Pr\left[\mathcal{A}(1^\lambda, \mathsf{pp}, k_{f_1}) = 1\right] - \Pr\left[\mathcal{A}(1^\lambda, \mathsf{pp}, k_{f_2}) = 1\right]\right| \leq \mathsf{negl}(\lambda),$$

  where the probabilities are taken over the randomness of $(\mathsf{pp}, k) \leftarrow \mathsf{Setup}(1^\lambda)$ *and* $k_{f_i} \leftarrow \mathsf{Shift}(\mathsf{pp}, k, f_i)$ *for* $i = 1, 2$*.*

For output ring R with distance metrics (e.g., $\mathbb{Z}_q$), we also define the following relaxed correctness notion from [PS18], where the shifted evaluation only provides a noisy evaluation.

Definition 3.4 (Approximate Correctness)**.** *A shiftable PRF with output space* $\mathbb{Z}_q^{m(\lambda)}$ *satisfies approximate correctness with error bound* $B(\lambda)$ *if for every* $\lambda \in \mathbb{N}$*, every function* $f \in F_\lambda$*, and every input* $x \in \{0,1\}^{n(\lambda)}$*, it holds that*

$$\left\| \left(\mathsf{Eval}(\mathsf{pp}, k, x) + f(x)\right) - \mathsf{EvalShifted}(\mathsf{pp}, k_f, x) \right\|_\infty \leq B(\lambda)$$

where $(\mathsf{pp}, k) \leftarrow \mathsf{Setup}(1^\lambda)$ *and* $k_f \leftarrow \mathsf{Shift}(\mathsf{pp}, k, f)$*.*

Remark 3.3 (Shiftable PRFs are (Punctured) PRFs)**.** Observe that if the function class $\mathcal{F}$ supports evaluation of scaled point functions $x \mapsto R \cdot \mathbb{1}_y$, where $\mathbb{1}_y$ is the point function defined by y, then a shiftable PRF is a punctured PRF. This justifies our name "shiftable PRF".

Remark 3.4 (Shiftable PRFs imply FSS)**.** A shiftable PRF for a function class $\mathcal{F}$ trivially implies an FSS scheme for the same function class $\mathcal{F}$ by setting sh_0 to be the shifted key $k_{\mathbf{0}}$ of the zero function and sh_1 to be the shifted key k_f for the shared function f. Each individual share hides f by the shift hiding property.

Theorem 3.3 ([PS18])**.** *Assuming polynomially hard LWE (with sub-exponential modulus-to-noise ratio), there exists an everywhere correct shiftable PRF for all bounded-size circuits. Furthermore, there exists a shiftable PRF with approximate correctness (Definition 3.4) with error bound* $B(\lambda)$ *for all bounded-size circuits where* B/q *is negligible.*

3.4 Somewhere Extractable Hash

Definition 3.5 (Somewhere Extractable Hash (SEH) [HW15])**.** *A somewhere extractable hash with prefix consistency proof is a tuple of PPT algorithms* $(\mathsf{Gen}, \mathsf{TGen}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Ver}, \mathsf{Ext}, \mathsf{ProveCons}, \mathsf{VerCons})$ *defined as follows:*

- $\mathsf{Gen}(1^\lambda, N, 1^\ell) \to \mathsf{hk}$*. On input the security parameter* 1^λ*, message length* 1^N*, and extraction set size bound* 1^ℓ*, outputs a hash key* hk*.*

- $\mathsf{TGen}(1^\lambda, N, 1^\ell, E \subseteq [N]) \to (\mathsf{hk}^*, \mathsf{td})$. *On input the security parameter* 1^λ*, message length* 1^N*, extraction set size bound* 1^ℓ *and an extraction set* $E \subseteq [N]$ *of size at most* ℓ*, outputs a hash key* hk^* *along with a trapdoor* td*.*

- $\mathsf{Hash}(\mathsf{hk}, \mathbf{x} \in \{0,1\}^N) \to \tau$*. On input a hash key* hk *and a* N*-bit message* $\mathbf{x}$*, outputs a hash value* τ*.*

- $\mathsf{Open}(\mathsf{hk}, \mathbf{x}, i) \to \rho$*. On input the hash key* hk*, a message* $\mathbf{x} \in \{0,1\}^N$ *and an index* $i \in [N]$*, outputs a local opening* ρ*.*

- $\mathsf{Ver}(\mathsf{hk}, \tau, i, y, \rho) \to 0/1$*. On input the hash key* hk*, a hash value* τ*, an index* $i \in [N]$*, a bit* $y \in \{0,1\}$ *and an opening* ρ*, the verification algorithm decides to accept or reject the local opening.*

- $\mathsf{Ext}(\mathsf{hk}^*, \mathsf{td}, \tau) \to \{\mathbf{x}_i\}_{i \in E}$*. On input the hash key* hk^**, the trapdoor* td*, and a hash value* τ*, the extraction algorithm outputs* $\{\mathbf{x}_i\}_{i \in E}$ *(the extraction set was chosen during the* TGen *algorithm).*

The SEH should satisfy the following properties:

- **Succinctness:** *The sizes of hash key* hk*, hash value* τ*, and local opening* ρ*, along with the running times of* Gen*,* TGen*,* Ver*, and* Ext*, are all* $\mathsf{poly}(\lambda, \ell, \log N)$*.*

- **Opening Correctness:** *For every* $\lambda, N, \ell \in \mathbb{N}$*, every hash key* hk *generated by* $\mathsf{Gen}(1^\lambda, 1^N, 1^\ell)$ *or* $\mathsf{TGen}(1^\lambda, 1^N, 1^\ell, E)$*, every message* $\mathbf{x} \in \{0,1\}^N$*, and every index* $i \in [N]$*, it holds that*

$$\Pr\left[\mathsf{Ver}(\mathsf{hk}, \mathsf{Hash}(\mathsf{hk}, \mathbf{x}), i, \mathbf{x}[i], \mathsf{Open}(\mathsf{hk}, \mathbf{x}, i)) = 1\right] = 1.$$

- **Key Indistinguishability:** *For every efficient adversary* $\mathcal{A}$ *and all polynomials* $N = N(\lambda)$*,* $\ell = \ell(\lambda)$*, there exists a negligible function* negl *such that for every sequence of extraction sets* $E_\lambda \subseteq [N]$ *of size at most* ℓ *and every* $\lambda \in \mathbb{N}$*, it holds that*

$$\left| \begin{array}{l} \Pr\left[\mathcal{A}(1^\lambda, \mathsf{hk}) = 1 \mid \mathsf{hk} \leftarrow \mathsf{Gen}(1^\lambda, 1^N, 1^\ell)\right] \\ - \Pr\left[\mathcal{A}(1^\lambda, \mathsf{hk}^*) = 1 \mid (\mathsf{hk}^*, \mathsf{td}) \leftarrow \mathsf{TGen}(1^\lambda, 1^N, 1^\ell, E_\lambda)\right] \end{array} \right| \leq \mathsf{negl}(\lambda).$$

- **Somewhere Extractability:** *For every* $\lambda, N, \ell \in \mathbb{N}$*, every extraction set* $E \subseteq [N]$ *of size at most* ℓ*, every hash key and trapdoor pair* $(\mathsf{hk}^*, \mathsf{td})$ *generated by* $\mathsf{TGen}(1^\lambda, 1^N, 1^\ell, E)$*, every index* $i \in E$*, every opening* ρ^**, and every value* $y \in \{0,1\}$*, it holds that*

$$\mathsf{Ver}(\mathsf{hk}^*, \tau, i, y, \rho^*) = 1 \implies y = \mathsf{Ext}(\mathsf{hk}^*, \mathsf{td}, \tau)[i],$$

where the notation $\mathsf{Ext}(\mathsf{hk}^*, \mathsf{td}, \tau)[i]$ *denotes the* i*-th entry of the extracted set* $\{y_i\}_{i \in E}$*.*

SEH with prefix consistency proof. We also recall the SEH schemes with proof of prefix consistency introduced in [MDS25]. Such schemes allow proving $x_1[:i] = x_2[:i]$ for any prefix length i using just the hash values $\tau_1 = \mathsf{SEH}(x_1)$ and $\tau_2 = \mathsf{SEH}(x_2)$, along with a succinct proof π_i.

Definition 3.6 (SEH with prefix consistency proof [MDS25])**.** *A somewhere extractable hash with prefix consistency proof is an SEH scheme with the following additional algorithms:*

- $\mathsf{ProveCons}(\mathsf{hk}, \mathbf{x}_1, \mathbf{x}_2, i) \to \pi_i$*. On input the hash key* hk*, two messages* $\mathbf{x}_1, \mathbf{x}_2 \in \{0,1\}^N$ *and a prefix length* $i \in [N]$*, output a proof* π_i *of prefix consistency.*

- $\mathsf{VerCons}(\mathsf{hk}, \tau_1, \tau_2, i, \pi_i) \to 0/1$. *On input the hash key* hk*, two hash values* τ_1, τ_2*, a prefix length* $i \in [N]$ *and a proof* π_i*, the verification algorithm decides to accept or reject the prefix consistency proof.*

The SEH with prefix consistency proof should satisfy the following additional property:

- **Succinctness:** *The size of the prefix consistency proof* π_i *and the time of the verification algorithm* $\mathsf{VerCons}$ *are both* $\mathsf{poly}(\lambda, \ell, \log N)$.
- **Consistency Completeness:** *For every* $\lambda, N, \ell \in \mathbb{N}$*, any hash key* hk *generated by* $\mathsf{Gen}(1^\lambda, 1^N, 1^\ell)$ *or* $\mathsf{TGen}(1^\lambda, 1^N, 1^\ell, E)$*, and any two strings* $\mathbf{x}_1, \mathbf{x}_2 \in \{0,1\}^N$ *such that* $\mathbf{x}_1[1:i] = \mathbf{x}_2[1:i]$ *for some index* $i \in [N]$*, it holds that*

$$\Pr\left[\mathsf{VerCons}(\mathsf{hk}, \mathsf{Hash}(\mathsf{hk}, \mathbf{x}_1), \mathsf{Hash}(\mathsf{hk}, \mathbf{x}_2), i, \mathsf{ProveCons}(\mathsf{hk}, \mathbf{x}_1, \mathbf{x}_2, i)) = 1\right] = 1.$$

- **Somewhere Statistical Soundness:** *For every* $\lambda, N, \ell \in \mathbb{N}$*, every set* $E \subseteq [N]$ *of size at most* ℓ*, any hash key and trapdoor pair* $(\mathsf{hk}^*, \mathsf{td})$ *generated by* $\mathsf{TGen}(1^\lambda, 1^N, 1^\ell, E)$*, every index* $i \in [N]$*, and any two hash values* τ_1, τ_2*, and every proof* π_i*, it holds that*

$$\mathsf{VerCons}(\mathsf{hk}^*, \tau_1, \tau_2, i, \pi_i) = 1 \implies \forall j \in E \cap [i],\ \mathsf{Ext}(\mathsf{hk}^*, \mathsf{td}, \tau_1)[j] = \mathsf{Ext}(\mathsf{hk}^*, \mathsf{td}, \tau_2)[j].$$

Theorem 3.4 ([MDS25])**.** *Assuming polynomially (resp. sub-exponentially) secure FHE, there exists a polynomially (resp. sub-exponentially) secure somewhere extractable hash with prefix consistency proof.*

3.5 Indistinguishability Obfuscation

We start by recalling the definition of indistinguishability obfuscation for both circuits and Turing machines.

Definition 3.7 (Indistinguishability Obfuscation for Circuits)**.** *A PPT algorithm* iO *is an indistinguishability obfuscator for a class of circuits* $\mathcal{C} = \{\mathcal{C}_\lambda\}$ *if it satisfies the following properties:*

- **Correctness***: There exists a negligible function* negl *such that for every* $\lambda \in \mathbb{N}$*, every circuit* $C \in \mathcal{C}_\lambda$*, and every input* x *in the domain of* C*, it holds that*

$$\Pr[\widetilde{C}(x) = C(x) \,|\, \widetilde{C} \leftarrow \mathsf{iO}(1^\lambda, C)] \geq 1 - \mathsf{negl}(\lambda).$$

- **Security***: For every efficient adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that for every* $\lambda \in \mathbb{N}$ *and any two circuits* $C_0, C_1 \in \mathcal{C}_\lambda$ *that are functionally equivalent, it holds that*

$$\left|\Pr[\mathcal{A}(1^\lambda, \mathsf{iO}(1^\lambda, C_0)) = 1] - \Pr[\mathcal{A}(1^\lambda, \mathsf{iO}(1^\lambda, C_1)) = 1]\right| \leq \mathsf{negl}(\lambda).$$

Definition 3.8 (Indistinguishability Obfuscation for Turing Machines)**.** *A PPT algorithm* iO *is an indistinguishability obfuscator for a class of Turing machines* $\mathcal{M}$ *if it satisfies the following properties:*

- **Correctness***: There exists a negligible function* negl *such that for every* $\lambda \in \mathbb{N}$*, every efficient Turing machine* $M \in \mathcal{M}$*, any input-length bound* N *and every* $x \in \{0,1\}^*$ *with length at most* N*, it holds that*

$$\Pr[\widetilde{M}(x) = M(x) \,|\, \widetilde{M} \leftarrow \mathsf{iO}(1^\lambda, M, N)] \geq 1 - \mathsf{negl}(\lambda).$$

- **Efficiency***: There exists a polynomial* p *such that for every* $\lambda \in \mathbb{N}$*, every Turing machine* $M \in \mathcal{M}$ *with runtime* $T(\cdot)$*, and every input* $x \in \{0,1\}^*$ *of length at most* N*, the Turing machine* $\mathsf{iO}(1^\lambda, M, N)$ *on input* x *runs in time at most* $p(\lambda, \log N, T(|x|))$.

- **Security**: *For every efficient adversary $\mathcal{A}$ and every polynomial T, there exists a negligible function* negl *such that for every $\lambda \in \mathbb{N}$, any time bound $N(\lambda) \leq 2^\lambda$, and any two Turing machines $M_0, M_1 \in \mathcal{M}$ with a runtime $T(\cdot)$ satisfying $M_0(x) = M_1(x)$ for every input $x \in \{0,1\}^*$, it holds that*

$$\left|\Pr[\mathcal{A}(1^\lambda, \mathsf{iO}(1^\lambda, M_0, N(\lambda))) = 1] - \Pr[\mathcal{A}(1^\lambda, \mathsf{iO}(1^\lambda, M_1, N(\lambda))) = 1]\right| \leq \mathsf{negl}(\lambda),$$

ΔiO and EFiO for circuits To avoid the widely-conjectured exponential security loss in constructing iO, the work [JJ22] considered the weakening EFiO, where the security requirement only needs to hold for pairs of circuits where functionality equivalence can be efficiently verified given a polynomial size $\mathcal{EF}$ proof. As a core step of the construction, Jain and Jin [JJ22] show that every pair of circuits with a polynomial size $\mathcal{EF}$ proof of equivalence can be transformed into a pair of Δ-equivalent circuits, which are two circuits that can be connected via a sequence of small local changes preserving functionality.

We recall the definition of Δ-equivalent circuits and EFiO below.

Definition 3.9 (Δ-Equivalent Circuits)**.** *Two families of circuits $\{C^0_\lambda\}$ and $\{C^1_\lambda\}$ are said to be Δ-equivalent with subcircuit size $s(\lambda) = O(\log \lambda)$ if there exists some polynomial $\ell(\lambda)$ such that for any $\lambda \in \mathbb{N}$, there exists a sequence of circuits $\{C_1, C_2, \ldots, C_{\ell(\lambda)}\}$ such that*

- *$C_1 = C^0_\lambda$ and $C_{\ell(\lambda)} = C^1_\lambda$.*
- *For any $i \in [\ell(\lambda) - 1]$, C_i and C_{i+1} have the same circuit topology. Furthermore, there exists a subcircuit S consisting of at most $s(\lambda)$ gates such that*
  - *For every gate outside of S, the corresponding gates in C_i and C_{i+1} are identical.*
  - *The subcircuits of C_i and C_{i+1} induced by S are functionally equivalent, i.e.,*

$$\forall y \in \{0,1\}^{|\mathsf{in}(S)|},\ C_{i,S}(y) = C_{i+1,S}(y).$$

We say that the two families are δ-equivalent if they are Δ-equivalent with $\ell(\lambda) = 2$.

Lemma 3.1 ($\mathcal{EF}$ Proofs of Equivalence imply Δ-Equivalence [JJ22])**.** *There exists an efficient padding algorithm* Pad *satisfying the following property:*

- *For any circuit C of size at most s_{cir} and any integer s_{pf}, $\mathsf{Pad}(C, s_{\mathrm{cir}}, s_{\mathrm{pf}})$ outputs a circuit of size $\mathsf{poly}(|C|, s_{\mathrm{cir}}, s_{\mathrm{pf}})$ that is functionally equivalent to C. Furthermore, for any two circuits C_0 and C_1 of size at most s_{cir}, $\mathsf{Pad}(C_0, s_{\mathrm{cir}}, s_{\mathrm{pf}})$ and $\mathsf{Pad}(C_1, s_{\mathrm{cir}}, s_{\mathrm{pf}})$ have the same circuit topology, and every gate in $\mathsf{Pad}(C, s_{\mathrm{cir}}, s_{\mathrm{pf}})$ has at most 2 input wires and at most 2 output wires.*
- *For any two family of circuits $\{C^0_\lambda\}$ and $\{C^1_\lambda\}$ where for every $\lambda \in \mathbb{N}$, the circuit C^b_λ has size at most $s_{\mathrm{cir}}(\lambda)$, and where there exists a $\mathcal{EF}$ proof of size at most $s_{\mathrm{pf}}(\lambda)$ of the statement that C^0_λ and C^1_λ are functionally equivalent, the two family $\{\mathsf{Pad}(C^0_\lambda, s_{\mathrm{cir}}, s_{\mathrm{pf}})\}$ and $\{\mathsf{Pad}(C^1_\lambda, s_{\mathrm{cir}}, s_{\mathrm{pf}})\}$ are Δ-equivalent with subcircuit size $s(\lambda) = O(\log s_{\mathrm{cir}} + \log s_{\mathrm{pf}}(\lambda))$ and $\ell(\lambda) = \mathsf{poly}(\lambda, s_{\mathrm{cir}}(\lambda) + s_{\mathrm{pf}}(\lambda))$.*

As a corollary, if there exists a PV *proof of equivalence between two families of circuits $\{C^0_\lambda\}$ and $\{C^1_\lambda\}$ with a polynomial bounding value (i.e., every element in the proof needs at most polynomially many digits), then there exists some suitable polynomial $s_{\mathrm{pf}}(\lambda)$ such that the two family $\{\mathsf{Pad}(C^0_\lambda, s_{\mathrm{cir}}, s_{\mathrm{pf}})\}$ and $\{\mathsf{Pad}(C^1_\lambda, s_{\mathrm{cir}}, s_{\mathrm{pf}})\}$ are Δ-equivalent.*

Definition 3.10 (ΔiO and EFiO for Circuits)**.** *A PPT algorithm* iO *is a* ΔiO *for a class of circuits* $\mathcal{C} = \{C_\lambda\}$ *with subcircuit size parameter* $s(\lambda)$ *if it satisfies the correctness property in Definition 3.7, and that for any two family of circuits* $\{C^0_\lambda\}$ *and* $\{C^1_\lambda\}$ *that are* Δ*-equivalent with subcircuit size* $s(\lambda)$*, the security property in Definition 3.7 restricted to these circuits holds.*

Similarly, a PPT algorithm iO *is an* EFiO *for a class of circuits* $\mathcal{C} = \{C_\lambda\}$ *with proof size* $\ell(\lambda)$ *if it satisfies the correctness property in Definition 3.7, and that for any two family of circuits* $\{C^0_\lambda\}$ *and* $\{C^1_\lambda\}$ *admitting a* $\ell(\lambda)$ *size* $\mathcal{EF}$ *proof of equivalence on each security parameter , the security property in Definition 3.7 restricted to these circuits holds.*

As a direct corollary of Lemma 3.1, every ΔiO is also an EFiO.

ΔiO and PViO for Turing machines. One important observation in [JJ22] is that the power of proofs of equivalence allows one to extend the construction for obfuscating circuits to obfuscating Turing machines with unbounded input length. They observed that two Turing machines with a PV proof of equivalence can be transformed into two Δ-equivalent Turing machines, i.e., two Turing machines where the circuit representations on every input length are Δ-equivalent.

We recall the circuit representation of Turing machines and the definition of ΔiO and PViO for Turing machines below.

Lemma 3.2 (Circuit representation of Turing machines)**.** *For any Turing machine* M *that runs in polynomial time* $T(\cdot)$ *and any length* n*, one can efficiently construct a circuit* M_n *of size* $T(n) \cdot \mathsf{poly}(\log T(n))$ *that simulates* M *on inputs of length* n*. Furthermore, there exists an efficient algorithm that, given as input a length bound* N*, outputs a circuit* $[\![M]\!]_N(\cdot,\cdot)$ *of size* $\mathsf{poly}(\log T(N), |M|)$ *with the format*

$$(\mathsf{in}_g, \mathsf{out}_g, f_g) \leftarrow [\![M]\!]_N(n \in [N], g \in [|M_n|]),$$

which takes as input an input length n *and a gate index* g*, and outputs the gate description of gate* g *in the circuit* $|M_n|$*. Here,* in_g *is the list of input wires of gate* g*,* out_g *is the list of output wires of gate* g*, and* f_g *is the function* $\{0,1\}^{|\mathsf{in}_g|} \to \{0,1\}^{|\mathsf{out}_g|}$ *computed by gate* g*.*

In other words, the circuit $[\![M]\!]_N(\cdot,\cdot)$ *is a succinct description of the family of circuits* $\{M_n\}_{n \le N}$*, which allows uniform generation of the circuit descriptions.*

Definition 3.11 (Δ-Equivalent Turing Machines)**.** *Two Turing machines* M_0 *and* M_1 *that run in time* $T(\cdot)$ *are said to be* Δ*-equivalent with subcircuit size* $s(n) = O(\log n)$ *if there exists a function* $\ell = \mathsf{poly}(n, \log N)$ *and a bound* $D(N) = \mathsf{poly}(\log N)$ *such that for any input length bound* N *and any* $n \le N$*, there exists a sequence of circuits* $\{C'_1, C'_2, \ldots, C'_{\ell(n,N)}\}$ *such that*

- C'_1 *is functionally equivalent to* $[\![M_0]\!]_N(n,\cdot)$ *and* $C'_{\ell(n,N)}$ *is functionally equivalent to* $[\![M_1]\!]_N(n,\cdot)$*.*
- *Each circuit* C'_i *has size at most* $D(N)$*.*
- *For any* $i \in [\ell(n,N) - 1]$*, let* C_i *and* C_{i+1} *be the circuit described by the succinct description* C'_i *and* C'_{i+1} *respectively. Then* C_i *and* C_{i+1} *are* δ*-equivalent with subcircuit size* $s(n)$*. In other words,* C'_i *and* C'_{i+1} *differ on at most* $s(n)$ *inputs, and the subcircuits induced by these outputted gate descriptions are functionally equivalent.*

Definition 3.12 (Uniform-$\mathcal{EF}$ proof of equivalence)**.** *We say that two Turing machines* M_0 *and* M_1 *admit a uniform-*$\mathcal{EF}$ *proof of equivalence of description size* D_{pf} *if there exists a Turing machine* $M_{\mathcal{EF}}$ *of size* D_{pf} *with polynomial runtime such that* $M_{\mathcal{EF}}(1^n)$ *outputs a proof of equivalence between the circuits induced by* M_0 *and* M_1 *on fixed input length* n*.*

Following [JJ22], if there exists a PV *proof of equivalence between two Turing machines M_0 and M_1 with a polynomial bounding value, then there exists some suitable polynomial $D_{\mathrm{pf}} = \mathsf{poly}(|M_0| + |M_1|)$ such that M_0 and M_1 admit a uniform-$\mathcal{EF}$ proof of equivalence of description size D_{pf}.*

Lemma 3.3 (PV Proofs of Equivalence Imply Δ-Equivalence [JJ22])**.** *There exists an efficient padding algorithm* sPad *satisfying the following property:*

- *For any machine M that runs in time $T(\cdot)$ with description size bound D, and any length bound N, $\mathsf{sPad}(M, D, N)$ outputs a circuit of size $\mathsf{poly}(\log T(N), D)$ that is functionally equivalent to the circuit $[\![M]\!]_N(\cdot,\cdot)$. Furthermore, for any two machines M_0 and M_1 that both run in time $T(\cdot)$ with description size bound D, for every length bound N and any $n < N$, the circuits described by the succinct description $\mathsf{sPad}(M_b, D, N)(n,\cdot)$ for $b \in \{0,1\}$ have the same circuit topology, and every gate outputted $\mathsf{sPad}(M, D, N)(n,\cdot)$ has at most 2 input wires and at most 2 output wires.*
- *For any two Turing machines M_0 and M_1 that run in polynomial time $T(\cdot)$ with description size bound D, and any length bound N, if there exists a uniform-$\mathcal{EF}$ proof of equivalence of description size at most D_{pf} between M_0 and M_1, then M_0 and M_1 are Δ-equivalent with subcircuit size $s(n) = O(\log n)$. Furthermore, each of the intermediate circuits in the Δ-equivalence guarantee can be generated uniformly.*

Definition 3.13 (ΔiO and PViO for Turing machines)**.** *A PPT algorithm* iO *is a Δ*iO *for a class of Turing machines $\mathbb{M}$ with subcircuit size parameter $s(n)$ if it satisfies the correctness property in Definition 3.8, and if for any two Δ-equivalent Turing machines M_0 and M_1 with subcircuit size $s(n)$, the security property in Definition 3.8 restricted to these Turing machines holds.*

Similarly, a PPT algorithm is a PViO *for a class of Turing machines $\mathbb{M}$ of proof length L if it satisfies the correctness property in Definition 3.8, and that for any two machines M_0 and M_1 admitting a length L* PV *proof of equivalence with a polynomial bounding value, the security property in Definition 3.8 restricted to these Turing machines holds.*

As a direct corollary of Lemma 3.3, every ΔiO for efficient Turing machines is also a PViO.

3.6 Time-Lock Puzzles

Definition 3.14 (Relaxed Time-Lock Puzzles)**.** *A relaxed time-lock puzzle scheme with message length $m = m(\lambda)$ consists of the following two efficient algorithms:*

- $\mathsf{Gen}(1^\lambda, T, x \in \{0,1\}^m) \to Z$ *takes as input the security parameter, a time parameter T, and a message x, and outputs a puzzle Z.*
- $\mathsf{Solve}(1^\lambda, 1^T, Z) \to x \in \{0,1\}^m$ *takes as input the security parameter, a time parameter T encoded in unary, and a puzzle Z, and outputs the solution x.*

A time-lock puzzle scheme satisfies the following properties:

- **Efficiency***: For every $\lambda \in \mathbb{N}$, every time parameter T, and every message $x \in \{0,1\}^m$, $\mathsf{Gen}(1^\lambda, T, x)$ runs in time $\mathsf{poly}(\lambda, \log T)$, and $\mathsf{Solve}(1^\lambda, 1^T, Z)$ runs in time $T \cdot \mathsf{poly}(\lambda)$.*
- **Correctness***: For every $\lambda \in \mathbb{N}$, every time parameter T, and every message $x \in \{0,1\}^m$, it holds that*

$$\Pr[\mathsf{Solve}(1^\lambda, 1^T, Z) = x \mid Z \leftarrow \mathsf{Gen}(1^\lambda, T, x)] = 1.$$

- **Security**: *The puzzle* $(\mathsf{Gen}, \mathsf{Solve})$ *is a time-lock puzzle with gap* $\epsilon < 1$ *if there exists a polynomial* $t_{\mathsf{min}}(\cdot)$*, such that for every time parameter* $2^\lambda \geq T(\lambda) \geq t_{\mathsf{min}}(\lambda)$*, every polynomial* p*, and every adversary* $\mathcal{A}$ *running in time at most* $\min(p(\lambda), T^\epsilon(\lambda))$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *so that for every* $\lambda \in \mathbb{N}$ *and any two messages* $x_0, x_1 \in \{0,1\}^{m(\lambda)}$*, it holds that*

$$\left|\Pr[\mathcal{A}(1^\lambda, Z_0) = 1] - \Pr[\mathcal{A}(1^\lambda, Z_1) = 1]\right| \leq \mathsf{negl}(\lambda),$$

where $Z_b \leftarrow \mathsf{Gen}(1^\lambda, T(\lambda), x_b)$ *for* $b \in \{0,1\}$*.*

Remark 3.5 (Relaxed Time-lock Puzzles with Large Time Parameter)**.** In this work, we only need a time-lock puzzle secure against *time-bounded* adversaries as opposed to depth-bounded adversaries. We therefore define the notion of *relaxed* time-lock puzzles following [BGJ+16, BS23]. Note that the security definition we provide in Definition 3.14 is slightly stronger than the ones in prior works [RSW96, BGJ+16, MT19]. Prior definitions argue security for ciphertexts with polynomial time/depth parameters, while we require *polynomial hardness* for ciphertexts with *superpolynomial time parameters*.

Implicitly, [BS23] proves that such a time-lock puzzle exists if there exists a non-parallelizable language and a decomposable obfuscation [LZ21].[13] Decomposable obfuscation is a strictly weaker primitive than EFiO: a decomposable obfuscation is an iO for circuit pairs with "identical partial evaluations", i.e., there exists a polynomial set of prefixes covering the whole input space, such that on the partial evaluation of each prefix, the pair simplifies to two circuits with identical descriptions. Such a prefix set naturally transfers to a proof of equivalence of the circuit pair.

Theorem 3.5. *If non-parallelizable languages and (polynomially secure) EFiO exist, then relaxed time-lock puzzles as in Definition 3.14 exist.*

In fact, [BS23] showed that the non-parallelizable language can be replaced with the following complexity assumption on time versus circuit size.

Assumption 3.1. *For any polynomial* $q(\cdot)$ *there exists a polynomial* $Q(\cdot)$ *and a language* $\mathcal{L} \in \mathbf{DTIME}(Q)$ *such that any family* $C = \{C_\lambda\}$ *of size-*$q(\lambda)$ *circuits fails to decide* $\mathcal{L}_\lambda = \mathcal{L} \cap \{0,1\}^\lambda$ *for all large enough* λ*.*

Theorem 3.6. *Assuming Assumption 3.1 and (polynomially secure) EFiO, then relaxed time-lock puzzles as in Definition 3.14 exist.*

3.7 Somewhere Statistically Correlation Intractable Hash

In this section, we recall the notion of somewhere statistically correlation intractable hash family (CIH) from [CCH+19]. For simplicity, we only focus on the function class of efficiently searchable relations, where CIHs are known from LWE [PS19].

Definition 3.15 (Somewhere Statistically CIH for Searchable Relations [CCH+19])**.** *Let* $C_{m,d,s} = \{C_\lambda\}$ *be the family of all circuits of arbitrary input length, output length* $m(\lambda)$*, depth at most* $d(\lambda)$ *and size at most* $s(\lambda)$*. A CIH family for* $C_{m,d,s}$ *is a tuple of PPT algorithms* $(\mathsf{Gen}, \mathsf{SGen}, \mathsf{Hash})$ *defined as follows:*

- $\mathsf{Gen}(1^\lambda) \to \mathsf{hk}$ *takes as input the security parameter* λ *and outputs a hash key* hk*.*
- $\mathsf{SGen}(1^\lambda, C \in C_\lambda) \to \mathsf{hk}$ *takes as input the security parameter* λ *and a circuit* C *of output length at least* $m(\lambda)$ *and size at most* $s(\lambda)$*, and outputs a hash key* hk*.*

[13] We thank the anonymous reviewers of Crypto 2026 for pointing this out.

- $\mathsf{Hash}(\mathsf{hk}, x) \to h$ *takes as input the hash key* hk*, and an input* x *of arbitrary length, and outputs a hash value* h *of length* $m(\lambda)$*.*

The CIH family should satisfy the following properties:

- **Statistical Correlation Intractability***: For every* $C \in \mathcal{C}_\lambda$ *with input size* n*, every* $\mathsf{hk} \leftarrow \mathsf{SGen}(1^\lambda, C)$*, and every input* $x \in \{0,1\}^n$*, it holds that* $\mathsf{Hash}(\mathsf{hk}, x) \neq C(x)$*.*
- **Relation Hiding***: For every efficient adversary* $\mathcal{A}$*, there exists negligible function* negl *such that for every sequence of circuits* $\{C_\lambda \in \mathcal{C}_\lambda\}$*, it holds that*

$$\left|\Pr\left[\mathcal{A}(1^\lambda, \mathsf{hk}) = 1 : \mathsf{hk} \leftarrow \mathsf{Gen}(1^\lambda)\right] - \Pr\left[\mathcal{A}(1^\lambda, \mathsf{hk}) = 1 : \mathsf{hk} \leftarrow \mathsf{SGen}(1^\lambda, C_\lambda)\right]\right| \leq \mathsf{negl}(\lambda).$$

Theorem 3.7. *For every polynomial* $d = d(\lambda)$*,* $s = s(\lambda)$*, and (sufficiently large)* $m = m(\lambda)$*, assuming LWE, there exists a somewhere statistically correlation intractable hash family for the class of efficiently searchable relations* $\mathcal{C}_{m,d,s}$*.*

4 Building Blocks

4.1 Shiftable PRF

In this section, we explore the relationship between function secret sharing and shiftable PRFs. It was observed in [CMPR23] that using a special form of FSS, namely staged homomorphic secret sharing, one can build a weaker form of shiftable PRFs called constrained PRFs. Here we use iO to build shiftable PRFs from FSS, thus removing the requirement for the HSS/FSS to be staged, and resulting in a construction of the stronger notion of shiftable PRFs.

Construction 4.1. Let FSS be a function secret sharing scheme with everywhere correctness (Definition 3.1) and iO an indistinguishability obfuscation. We construct a shiftable PRF as follows:

- $\mathsf{Setup}(1^\lambda)$: Run $\mathsf{sh}_0 \leftarrow \mathsf{FSS.Share}_0(1^\lambda, \mathbf{0})$ to get a share of the all zeros function. Output $(\mathsf{pp} = \perp, k := \mathsf{sh}_0)$. Recall that Share_0 is the algorithm that generates the first share of the function secret sharing scheme.
- $\mathsf{Eval}(\mathsf{pp}, k, x \in \{0,1\}^n)$: Output $\mathsf{FSS.Eval}(\mathsf{sh}_0, x \in \{0,1\}^n)$.
- $\mathsf{Shift}(\mathsf{pp}, k, f)$: Output $k_f = \mathsf{iO}(E_{\mathsf{sh}_0,f})$, where $E_{\mathsf{sh}_0,f}$ is defined as follows:

  > $E_{\mathsf{sh}_0,f}(x \in \{0,1\}^n)$: Output $\mathsf{FSS.Eval}(\mathsf{sh}_0, x) + f(x)$.

- $\mathsf{EvalShifted}(\mathsf{pp}, k_f, x \in \{0,1\}^n)$: Evaluate the circuit k_f on x, and output the result.

Theorem 4.1. *If* FSS *is (polynomially) secure and has a negligible correctness error and* iO *is a (polynomially-secure) indistinguishability obfuscation scheme then Construction 4.1 is a shiftable PRF.*

Proof. Perfect correctness follows from the correctness of the obfuscation scheme.

To prove shift-hiding, we show that a shifted key with respect to a function f is computationally indistinguishable from one that is independent of the shift. We do so using the following sequence of hybrids:

- Hybrid 0: This is the normal distribution, namely the view of the adversary is $(\mathsf{pp} = \perp, k_f = \mathsf{iO}(E_{\mathsf{sh}_{0,\mathbf{0}},f}(x)))$, where $\mathsf{sh}_{0,\mathbf{0}} \leftarrow \mathsf{FSS.Share}_0(\mathbf{0})$.

  > $E_{\mathsf{sh},f}(x \in \{0,1\}^n)$: Output $\mathsf{FSS.Eval}(\mathsf{sh}, x) + f(x)$.

- Hybrid 1: We replace the share of the all-zeros function with a share of $-f$. We generate the view of the adversary by computing $(\mathsf{sh}_{0,f}, \mathsf{sh}_{1,f}) \leftarrow \mathsf{FSS.Share}(-f)$ and outputting $(\mathsf{pp} = \perp, k_f = \mathsf{iO}(E_{\mathsf{sh}_{0,f},f}(x)))$.

  Claim 4.1. Assuming FSS is (polynomially) secure, Hybrid 0 and Hybrid 1 are computationally indistinguishable.

- Hybrid 2: We replace the evaluation of the 0 share of $-f$, by the evaluation of the 1 share of $-f$, and subtracting $f(x)$. We generate the view of the adversary by computing $(\mathsf{sh}_{0,f}, \mathsf{sh}_{1,f}) \leftarrow \mathsf{FSS.Share}(-f)$ and outputting $(\mathsf{pp} = \perp, k_f = \mathsf{iO}(E'_{\mathsf{sh}_{1,f}}(x)))$.

  > $E'_{\mathsf{sh}}(x \in \{0,1\}^n)$: Output $\mathsf{FSS.Eval}(\mathsf{sh}, x)$.

Claim 4.2. Assuming FSS is everywhere correct (Definition 3.1) and iO is secure then Hybrid 1 and Hybrid 2 are computationally indistinguishable.

Proof. Everywhere correctness of the FSS guarantees that $\mathsf{Eval}(\mathsf{sh}_{0,f}, \cdot)$ and $\mathsf{Eval}(\mathsf{sh}_{1,f}, \cdot) - f(\cdot)$ are functionally equivalent with overwhelming probability, and thus so are $\mathsf{Eval}(\mathsf{sh}_{0,f}, \cdot) + f(\cdot)$ and $\mathsf{Eval}(\mathsf{sh}_{1,f}, \cdot)$. iO security then ensures that no efficient adversary can distinguish obfuscations of these circuits with more than negligible probability. □

- Hybrid 3: We replace the 1 share of $-f$ with a share of the all-zeros function. We generate the view of the adversary by computing $\mathsf{sh}_{1,\mathbf{0}} \leftarrow \mathsf{FSS.Share}_1(\mathbf{0})$ and outputting $(\mathsf{pp}, k_f = \mathsf{iO}(E'_{\mathsf{sh}_{1,\mathbf{0}}}(x)))$.

  Claim 4.3. Assuming FSS is secure, Hybrid 2 and Hybrid 3 are computationally indistinguishable.

In Hybrid 3, the view of the adversary is independent of f, and shift-hiding follows. □

Remark 4.1 (Expressivity of the Shiftable PRF)**.** Looking at our proof, the construction is secure for constraints f as long as the FSS supports evaluation of the function $-f$ (where $-$ is defined over the ring R).

Remark 4.2 (Our One-Shot Puncturing Technique, Directly from FSS)**.** In the technical body, we present our constructions using the abstraction of shiftable PRFs. We observe here that one can directly replace our use of shiftable PRFs with FSS evaluation, without going through Construction 4.1 (which adds an additional layer of iO). This is because all our calls to shiftable PRFs in our work are within an "outer" iO, and so we can remove the layer of "inner" iO from Construction 4.1. Then, the proof of Theorem 4.1 would still go through, using the "outer iO" to invoke iO security.

4.2 Deterministic Constrained Signatures

In this section, we define a notion of *deterministic constrained signature* scheme, which is a crucial building block for applications of iO from polynomial assumptions.

Definition 4.1. *A deterministic constrained signature scheme is defined with message space* $\{0,1\}^{\ell(\lambda)}$ *and constraint space* $\mathcal{F} = \{F_\lambda\}$ *consisting of functions* $f : \{0,1\}^{\ell(\lambda)} \to \{0,1\}$. *Without loss of generality, we assume that the all-accepting function is in* $\mathcal{F}_\lambda$ *for all* λ.

The signature scheme consists of the following PPT algorithms:

- $\mathsf{Setup}(1^\lambda) \to \mathsf{msk}$ *takes as input the security parameter* λ*, and outputs a master signing key* msk.
- $\mathsf{Constrain}(\mathsf{msk}, f) \to (\mathsf{vk}_f, \mathsf{sk}_f)$ *takes as input the master secret key* msk *and a constraint function* $f \in \mathcal{F}_\lambda$*, and outputs a constrained key pair* $(\mathsf{vk}_f, \mathsf{sk}_f)$.
- $\mathsf{Sign}(\mathsf{sk}_f, x) \to \sigma$ *takes as input the constrained signing key* sk_f *and a message* $x \in \{0,1\}^\ell$*, and outputs a signature* σ.
- $\mathsf{Ver}(\mathsf{vk}_f, x, \sigma^*) \to \{0,1\}$ *takes as input a constrained verification key* vk_f*, a message* $x \in \{0,1\}^\ell$ *and a signature* σ^**, outputs 1 if the signature is valid and 0 otherwise.*

The deterministic constrained signature scheme should satisfy the following properties:

- **Correctness:** *For any* $\lambda \in \mathbb{N}$*, any* $f \in \mathcal{F}_\lambda$*, and any* $x \in \{0,1\}^{\ell(\lambda)}$ *such that* $f(x) = 1$*, it holds that*

$$\Pr\left[\mathsf{Ver}(\mathsf{vk}_f, x, \mathsf{Sign}(\mathsf{sk}_f, x)) = 1 \,\middle|\, \mathsf{msk} \leftarrow \mathsf{Setup}(1^\lambda),\ (\mathsf{vk}_f, \mathsf{sk}_f) \leftarrow \mathsf{Constrain}(\mathsf{msk}, f)\right] = 1.$$

- **Deterministic Signing:** *For any* $\lambda \in \mathbb{N}$*, any* $f_0, f_1 \in \mathcal{F}_\lambda$*, and any* $x \in \{0,1\}^{\ell(\lambda)}$ *such that* $f_0(x) = f_1(x) = 1$*, it holds that*

$$\Pr\left[\mathsf{Sign}(\mathsf{sk}_{f_0}, x) = \mathsf{Sign}(\mathsf{sk}_{f_1}, x) \,\middle|\, \mathsf{msk} \leftarrow \mathsf{Setup}(1^\lambda),\ (\mathsf{vk}_{f_b}, \mathsf{sk}_{f_b}) \leftarrow \mathsf{Constrain}(\mathsf{msk}, f_b)\right] = 1.$$

- **Statistical Soundness:** *There exists a negligible function* negl *such that for any* $\lambda \in \mathbb{N}$ *and any* $f \in \mathcal{F}_\lambda$*, it holds that*

$$\Pr\left[\forall x \in f^{-1}(0),\ \mathsf{Ver}(\mathsf{vk}_f, x, \cdot) = \mathsf{Null} \,\middle|\, (\mathsf{vk}_f, \mathsf{sk}_f) \leftarrow \mathsf{Constrain}(\mathsf{Setup}(1^\lambda), f)\right] \geq 1 - \mathsf{negl}(\lambda).$$

  where $f^{-1}(0)$ *denotes the set* $\{x \in \{0,1\}^\ell \mid f(x) = 0\}$. *We say that the signature scheme is* perfectly sound *if the above probability is 1.*

- **Constraint hiding:** *For any* $\lambda \in \mathbb{N}$*, any function* $f \in \mathcal{F}_\lambda$*, the following two distributions are computationally indistinguishable:*

$$\left\{(\mathsf{vk}_f, \mathsf{sk}_f) \,\middle|\, \begin{array}{c} \mathsf{msk} \leftarrow \mathsf{Setup}(1^\lambda) \\ (\mathsf{vk}_f, \mathsf{sk}_f) \leftarrow \mathsf{Constrain}(\mathsf{msk}, f) \end{array}\right\} \approx \left\{(\mathsf{vk}_{\mathbf{1}}, \mathsf{sk}_f) \,\middle|\, \begin{array}{c} \mathsf{msk} \leftarrow \mathsf{Setup}(1^\lambda) \\ (\mathsf{vk}_{\mathbf{1}}, \mathsf{sk}_{\mathbf{1}}) \leftarrow \mathsf{Constrain}(\mathsf{msk}, \mathbf{1}) \\ (\mathsf{vk}_f, \mathsf{sk}_f) \leftarrow \mathsf{Constrain}(\mathsf{msk}, f) \end{array}\right\}$$

  where $\mathbf{1}$ *is the all-one function.*

Remark 4.3 (Comparison with [BZ14])**.** Readers might find the above definition similar to the constrained signature scheme defined in [BZ14]. Indeed, both definitions provide constrained signing and verification operations, where the verification never accepts messages disallowed by the constraint. However, our definition requires the following two strengthened properties.

- **Constraint Hiding given a Secret Key:** In [BZ14], the hiding property of the constraint holds against adversaries with *oracle queries* to the (constrained) signing algorithm. In our definition, we require the constraint to be hidden even when the constrained signing key sk_f is given to the adversary. This stronger hiding property is essential for our application, as we would hardcode the constrained signing key into an obfuscated program.
- **Consistency of Signatures across Constraints:** In [BZ14], every constrained key pair is generated independently. For our applications, it is crucial to switch between different constrained signing keys while fixing the verification key. We realize such a switch through iO security, by requiring signatures generated under different constrained signing keys to be equal.

Also, on the construction side, the constrained signatures in [BZ14] are built from *sub-exponentially secure* constrained PRF and iO. In Construction 4.2, we show how to build deterministic constrained signatures from *polynomially secure* shiftable PRF and iO.

4.2.1 Construction

In this section, we show how to construct a deterministic constrained signature scheme from polynomially-secure shiftable PRF and indistinguishability obfuscation.

Construction 4.2. Let iO be an indistinguishability obfuscation scheme, and let $\mathsf{sPRF} = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{Shift}, \mathsf{EvalShifted})$ be a shiftable PRF scheme for all bounded-size circuits with input space $\{0,1\}^{\ell(\lambda)}$ and output space $R^{m(\lambda)}$ for a ring R. We construct a deterministic constrained signature scheme with message space $\{0,1\}^{\ell(\lambda)}$ and signature space $R^{m(\lambda)}$ as follows:

- $\mathsf{Setup}(1^\lambda)$: Run $(\mathsf{pp}, k) \leftarrow \mathsf{sPRF.Setup}(1^\lambda)$, output $\mathsf{msk} = (\mathsf{pp}, k)$.
- $\mathsf{Constrain}(\mathsf{msk}, f)$: Parse $\mathsf{msk} = (\mathsf{pp}, k)$, and define the following two circuits:

  $E_{\mathsf{pp},k,f}(x \in \{0,1\}^\ell)$: If $f(x) = 0$, output $\perp$. Else, output $\mathsf{sPRF.Eval}(\mathsf{pp}, k, x)$.

  $V_{\mathsf{pp},k,f}(x \in \{0,1\}^\ell, y \in R^m)$: If $f(x) = 1$ and $\mathsf{sPRF.Eval}(\mathsf{pp}, k, x) = y$, output 1. Else, output 0.

  Output $\mathsf{vk}_f = \mathsf{iO}(V_{\mathsf{pp},k,f})$ and $\mathsf{sk}_f = \mathsf{iO}(E_{\mathsf{pp},k,f})$.
- $\mathsf{Sign}(\mathsf{sk}_f, x)$: Parse $\mathsf{sk}_f = \widetilde{E}_{\mathsf{pp},k,f}$ as an obfuscated program. Output $\sigma = \widetilde{E}_{\mathsf{pp},k,f}(x)$.
- $\mathsf{Ver}(\mathsf{vk}_f, x, \sigma^*)$: Parse $\mathsf{vk}_f = \widetilde{V}_{\mathsf{pp},k,f}$ as an obfuscated program. Output $\widetilde{V}_{\mathsf{pp},k,f}(x, \sigma^*)$.

It is straightforward from the construction that, if the obfuscation scheme is perfectly correct, then the deterministic constrained signature scheme is perfectly correct, perfectly sound, and has deterministic signing. We now show that the scheme is constraint hiding.

Theorem 4.2. *If iO is a polynomially secure indistinguishability obfuscation scheme, sPRF is a polynomially-secure shiftable PRF scheme for all bounded-size circuits with input space $\{0,1\}^{\ell(\lambda)}$ and output space $R^{m(\lambda)}$, and if there exists a polynomially secure injective PRG PRG with input space $R^{m(\lambda)}$ and output space $\{0,1\}^{m'(\lambda)}$ with λ bit stretch (i.e., $m'(\lambda) - |R|m(\lambda) \geq \lambda$), then Construction 4.2 is a constraint hiding deterministic constrained signature scheme.*

Proof. We argue through the following sequence of hybrids:

- Hybrid 0: This is the left distribution in the constraint hiding definition, where the output consists of $(\mathsf{vk}_f, \mathsf{sk}_f) = (\mathsf{iO}(V_{\mathsf{pp},k,f}), \mathsf{iO}(E_{\mathsf{pp},k,f}))$.

- Hybrid 1: We switch the underlying keys to be shifted by the all-zero function. Output $(\mathsf{vk}'_f, \mathsf{sk}'_f) = (\mathsf{iO}(V'_{\mathsf{pp},k_{\mathbf{0}},f}), \mathsf{iO}(E'_{\mathsf{pp},k_{\mathbf{0}},f}))$, where $k_{\mathbf{0}} \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}, k, \mathbf{0})$ is the shiftable PRF key shifted by the all-zero circuit, and the circuits $E'_{\mathsf{pp},k_{\mathbf{0}},f}$ and $V'_{\mathsf{pp},k_{\mathbf{0}},f}$ are obtained by replacing $\mathsf{sPRF.Eval}(\mathsf{pp}, k, x)$ with the shifted evaluation $\mathsf{sPRF.EvalShifted}(\mathsf{pp}, k_{\mathbf{0}}, x)$ in the definitions of $E_{\mathsf{pp},k,f}$ and $V_{\mathsf{pp},k,f}$ respectively.

  By the correctness of the shiftable PRF, the new circuits are functionally equivalent to the ones in Hybrid 0, and so a direct reduction to iO security shows the following.

  Claim 4.4. Assuming the security of iO and the everywhere correctness of sPRF, Hybrid 0 and Hybrid 1 are computationally indistinguishable.

- Hybrid 2: We change the way shifted keys are computed. We first sample a random vector $v \leftarrow R^{m(\lambda)}$, and use a shifted key $k_{C_{f,v}}$ instead of $k_{\mathbf{0}}$, where the shift circuit is defined by $C_{f,v}(x) = (1 - f(x)) \cdot v$. The hybrid outputs $(\mathsf{vk}'_f, \mathsf{sk}'_f) = (\mathsf{iO}(V'_{\mathsf{pp},k_{C_{f,v}},f}), \mathsf{iO}(E'_{\mathsf{pp},k_{C_{f,v}},f}))$.

  Indistinguishability follows from a direct reduction to shift hiding.

  Claim 4.5. Assuming sPRF is shift hiding, Hybrid 1 and Hybrid 2 are computationally indistinguishable.

- Hybrid 3: We change how the circuits compute their outputs. Compute $(\mathsf{vk}'_f, \mathsf{sk}'_f) = (\mathsf{iO}(V'_{\mathsf{pp},k,f,C_{f,v}}), \mathsf{iO}(E'_{\mathsf{pp},k,f,C_{f,v}}))$, where the circuit $E'_{\mathsf{pp},k,f,C_{f,v}}$ and $V'_{\mathsf{pp},k,f,C_{f,v}}$ are obtained by replacing the $\mathsf{sPRF.EvalShifted}(\mathsf{pp}, k_{C_{f,v}}, x)$ in the definitions of $E'_{\mathsf{pp},k_{C_{f,v}},f}$ and $V'_{\mathsf{pp},k_{C_{f,v}},f}$ respectively, with $\mathsf{sPRF.Eval}(\mathsf{pp}, k, x) + C_{f,v}(x)$.

  By the correctness of the shiftable PRF, the new circuits are functionally equivalent to the ones in Hybrid 2.

  Claim 4.6. Assuming the security of iO and the everywhere correctness of sPRF, Hybrid 2 and Hybrid 3 are computationally indistinguishable.

- Hybrid 4: Compute $(\mathsf{vk}'_f, \mathsf{sk}_f) = (\mathsf{iO}(V'_{\mathsf{pp},k,f,C_{f,v}}), \mathsf{iO}(E_{\mathsf{pp},k,f}))$, where the circuit $E_{\mathsf{pp},k,f}$ is the same as in Hybrid 0.

  Note that the circuit $E_{\mathsf{pp},k,f,C_{f,v}}$ and $E_{\mathsf{pp},k,f}$ are functionally equivalent since for any x such that $f(x) = 1$, we have $C_{f,v}(x) = 0$.

  Claim 4.7. Assuming the security of iO and the everywhere correctness of sPRF, Hybrid 3 and Hybrid 4 are computationally indistinguishable.

- Hybrid 5: Compute $(\mathsf{vk}'_f, \mathsf{sk}_f) = (\mathsf{iO}(V''_{\mathsf{pp},k,f,T_0,T_1}), \mathsf{iO}(E_{\mathsf{pp},k,f}))$, where the circuit $V''_{\mathsf{pp},k,f,T_0,T_1}$ is defined by replacing the equality check in $V'_{\mathsf{pp},k,f,C_{f,v}}$:

$$\mathsf{sPRF.Eval}(\mathsf{pp}, k, x) + C_{f,v}(x) = y$$

with the following check:

$$\mathsf{PRG}(y - \mathsf{sPRF.Eval}(\mathsf{pp}, k, x)) = T_{f(x)}, \quad \text{where } T_0 = \mathsf{PRG}(v), T_1 = \mathsf{PRG}(0^m).$$

where PRG is the injective PRG.

By the injectivity of PRG, the two equality checks are functionally equivalent.

Claim 4.8. Assuming the security of iO and the injectivity of PRG, Hybrid 4 and Hybrid 5 are computationally indistinguishable.

Note that in this hybrid, T_0 is the only variable that depends on v.

- Hybrid 6: Same as Hybrid 5, but replace T_0 with a random value in $\{0,1\}^{m'(\lambda)}$.

  Claim 4.9. Assuming the security of PRG, Hybrid 5 and Hybrid 6 are computationally indistinguishable.

  Note that since $m'(\lambda) > \lambda + |R|m(\lambda)$, with overwhelming probability, T_0 falls outside the image of PRG and the check $\mathsf{PRG}(y - \mathsf{sPRF.Eval}(\mathsf{pp}, k, x)) = T_0$ is not satisfiable.

- Hybrid 7: Compute $(\mathsf{vk}_1', \mathsf{sk}_f) = (\mathsf{iO}(V''_{\mathsf{pp},k,1,T_0,T_1}), \mathsf{iO}(E_{\mathsf{pp},k,f}))$, where the circuit $V''_{\mathsf{pp},k,1,T_0,T_1}$ is defined by replacing the initial check $f(x) = 1$ in $V''_{\mathsf{pp},k,f,T_0,T_1}$ with the always-true check. The two circuits differ only in the case where there exists some (x, y) such that $f(x) = 0$ but $\mathsf{PRG}(y - \mathsf{sPRF.Eval}(\mathsf{pp}, k, x)) = T_0$. With overwhelming probability over the random choice of T_0, such (x, y) does not exist.

  Claim 4.10. Assuming the security of iO, and that PRG has λ bit stretch, Hybrid 6 and Hybrid 7 are computationally indistinguishable.

The rest of the proof is symmetric to the first half, switching vk_1' back to vk_1. The indistinguishability between each pair of hybrids follows from the exact same arguments from Hybrids 0 to 6.

- Hybrid 8: This is the same as Hybrid 7, but replace T_0 back to $\mathsf{PRG}(v)$.
- Hybrid 9: Compute $(\mathsf{vk}_1', \mathsf{sk}_f) = (\mathsf{iO}(V'_{\mathsf{pp},k,1,C_{f,v}}), \mathsf{iO}(E_{\mathsf{pp},k,f}))$, where the PRG equation check in $V''_{\mathsf{pp},k,1,T_0,T_1}$ is replaced back to the equality check:

  $$\mathsf{sPRF.Eval}(\mathsf{pp}, k, x) + C_{f,v}(x) = y.$$

- Hybrid 10: Compute $(\mathsf{vk}_1', \mathsf{sk}_f') = (\mathsf{iO}(V'_{\mathsf{pp},k,1,C_{f,v}}), \mathsf{iO}(E'_{\mathsf{pp},k,f,C_{f,v}}))$, where the circuit $E'_{\mathsf{pp},k,f,C_{f,v}}$ is defined as in Hybrid 3.
- Hybrid 11: Compute $(\mathsf{iO}(V'_{\mathsf{pp},k_{C_{f,v}},f}), \mathsf{iO}(E'_{\mathsf{pp},k_{C_{f,v}},f}))$, where the circuits are obtained by replacing $\mathsf{sPRF.Eval}(\mathsf{pp}, k, x) + C_{f,v}(x)$ by $\mathsf{sPRF.EvalShifted}(\mathsf{pp}, k_{C_{f,v}}, x)$ from the circuits in Hybrid 10.
- Hybrid 12: Same as Hybrid 11, but uses the shifted-by-zero key $k_{\mathbf{0}}$ instead of $k_{C_{f,v}}$.
- Hybrid 13: This is the right distribution in the constraint hiding definition, where the output consists of $(\mathsf{vk}_1, \mathsf{sk}_f) = (\mathsf{iO}(V_{\mathsf{pp},k,1}), \mathsf{iO}(E_{\mathsf{pp},k,f}))$.

□

Combining with the observation that injective PRGs can be constructed from one-way functions plus iO (see Construction A.1), we have the following corollary.

Corollary 4.1. Assuming the existence of a polynomially-secure one-way function, a polynomially-secure FSS for all bounded-size circuits, and a polynomially-secure indistinguishability obfuscation scheme, there exists a constraint hiding deterministic constrained signature scheme.

Remark 4.4 (Expressivity of the Constrained Signature)**.** Looking at our proof, the construction is secure for constraints f as long as the shiftable PRF supports shifts of the form $x \mapsto (1 - f(x)) \cdot v$ for a hard-coded v in the range of the shiftable PRF. Given Remark 4.1, this translates to the FSS supporting evaluation of $x \mapsto (f(x) - 1) \cdot v$.

Remark 4.5 (Our One-Shot Hidden Sparse Trigger Technique, Directly from FSS/Shiftable PRF)**.** Similar to the observation from Remark 4.2, whenever *both* the signing key and the verification key are within an "outer" iO, we can remove the "inner" iO layer from Construction 4.2 and directly use the signing circuit $E_{\mathsf{pp},k,f}$ and the verification circuit $V_{\mathsf{pp},k,f}$. The proof of security would still go through, using the "outer iO" to invoke iO security.

5 PViO from iO for Small Circuits

We present here our construction of PViO from iO for circuits with inputs of size $\mathsf{poly}(\lambda)$, along with (the polynomial hardness) of LWE and time-lock puzzles (Theorem 5.5). We start with a construction for circuits, namely EFiO (Definition 3.10), in Section 5.1. The construction for Turing machines, namely PViO (Definition 3.13), can be found in Section 5.2.

5.1 Construction of EFiO

In this section, we show how to construct EFiO (Definition 3.10) for polynomial-size circuits from (fixed-size) polynomial assumptions. Following the observation from [JJ22] (restated in Lemma 3.1), it suffices to construct a ΔiO scheme for circuits with sub-circuit size $s(\lambda) = O(\log \lambda)$.

Our construction largely follows the blueprint of [JJ22]. At a high level, they construct ΔiO by providing an obfuscation for each gate of the original circuit. To hide the intermediate computation, which is essential for iO security, the input and output to each gate should be encrypted and authenticated. The [JJ22] construction of gate obfuscation takes as input encryptions of input wires, along with MAC tags of these ciphertexts, and outputs a ciphertext of output wires along with MAC tags only if the inputs are valid. The encryptions and MACs are all instantiated with puncturable PRFs, resulting in the need of 2^λ hybrids when arguing security using iO.

Overview of the Construction. The construction from [JJ22] involves some additional components to help prove security, which were skimmed in our technical overview. We provide some intuition here. To reduce the reduction loss to polynomial, we essentially replace the puncturable PRFs in the [JJ22] construction with shiftable PRFs (Definition 3.3). Additionally, we also leverage ideas from [MDS25] to further simplify the construction by removing the need for BARGs. Overall, Construction 5.1 can be viewed as the construction from [JJ22] with the following modifications:

- Using somewhere extractable hash with inherent prefix consistency proof [MDS25], we remove the need for BARG proofs in the gate obfuscation. As a trade-off, the wire values have to be sequentially evaluated following their indices.
- The ciphertext for each wire value $\mathsf{ct}_i = x_i \oplus \mathsf{PRF}(k_i, h_i)$ is replaced with a shiftable PRF $\mathsf{ct}_i = x_i + \mathsf{sPRF}(k_i, h_i)$.
- Instead of providing MAC tags for each gate output, we use a constrained signature scheme (Definition 4.1) to sign and verify each wire ciphertext.

As explained in the overview, two internal, connected gates share encryption and authentication keys; we need to break this dependence to argue semantic security of the encryption. To do so, the construction additionally involves, for every internal wire, both a (somewhere-extractable) hash (SEH), which encodes all "previously-seen" ciphertexts and authentications, and an FHE encryption of a dummy ciphertext. In the proof, these components will help implement trapdoors to decrypt the input wire ciphertexts without the corresponding secret key.

First, for every wire, we switch how decryption is performed to rely instead on the FHE and the SEH. The dummy FHE ciphertext is first switched to be an encryption of all the SEH-extracted input wires of the (sub)circuit. This corresponds to $\mathcal{H}_1^b$ and $\mathcal{H}_2^b$. Then, this dummy ciphertext can be used to compute the real internal wires using the FHE secret key. So for honestly generated ciphertexts, this new trapdoor decryption computes a functionally equivalent decryption. However, this is not true for maliciously generated ciphertexts. We then rely on our authentication component to argue

that for authenticated ciphertexts, the decryption procedures are functionally equivalent. Here, we crucially use our constrained signatures: it allows us to remove all unauthorized authentications from the picture, without using an input-by-input argument. This overall allows us to ensure that the trapdoor decryption using FHE and SEH is functionally equivalent to the original decryption, and thus switch standard decryption to trapdoor decryption using iO. This corresponds to $\mathcal{H}^b_{3,t,0}$ to $\mathcal{H}^b_{3,t,2}$.

At this point, encryption keys are only used to encrypt messages, and we wish to argue semantic security. We now face the problem described in the technical overview, namely, we want to switch from an obfuscation encrypting $P(x)$ to one encrypting $P'(x)$ for some P, P'. We do so by relying on shiftable PRFs, which save us from using an input-by-input argument. This corresponds to $\mathcal{H}^b_4$ to $\mathcal{H}^b_7$.

We now provide the formal construction of $\Delta\mathsf{iO}$.

Construction 5.1. Consider the following building blocks:

- A fully homomorphic encryption scheme $\mathsf{FHE} = (\mathsf{FHE.KeyGen}, \mathsf{FHE.Enc}, \mathsf{FHE.Dec}, \mathsf{FHE.Eval})$.
- A somewhere extractable hash with prefix consistency proof scheme $\mathsf{SEH} = (\mathsf{SEH.Gen}, \mathsf{SEH.TGen}, \mathsf{SEH.Hash}, \mathsf{SEH.Open}, \mathsf{SEH.Ver}, \mathsf{SEH.Ext}, \mathsf{SEH.ProveCons}, \mathsf{SEH.VerCons})$. (Definition 3.5)
- A shiftable PRF scheme $\mathsf{sPRF} = (\mathsf{sPRF.Setup}, \mathsf{sPRF.Eval}, \mathsf{sPRF.Shift}, \mathsf{sPRF.EvalShifted})$ with output over a ring R (Definition 3.3).
- A deterministic constrained signature scheme $\mathsf{Sig} = (\mathsf{Sig.Setup}, \mathsf{Sig.Constrain}, \mathsf{Sig.Sign}, \mathsf{Sig.Ver})$ (Definition 4.1).
- An indistinguishability obfuscation scheme iO secure for circuits of size at most $p(\lambda)$, where p will be instantiated later.

When clear from context, we sometimes omit the scheme name prefix for brevity. The obfuscation for subcircuit size $s(\lambda) = O(\log \lambda)$ (where the size is counted as the number of wires in the subcircuit) is defined by the following procedure.

$\underline{\Delta\mathsf{iO}(1^\lambda, C)}$:

- Sample FHE key pair $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{FHE.KeyGen}(1^\lambda)$.
- For each wire i of the circuit, sample shiftable PRF keys $(\mathsf{pp}_i, k_i) \leftarrow \mathsf{sPRF.Setup}(1^\lambda)$ and the master constrained signature key $\mathsf{msk}_i \leftarrow \mathsf{Sig.Setup}(1^\lambda)$. Compute the constrained key pair $(\mathsf{vk}_i, \mathsf{sk}_i) \leftarrow \mathsf{Sig.Constrain}(\mathsf{msk}_i, \mathbf{1})$, where $\mathbf{1}$ denotes the all-one function.
- Define the block size B as the ciphertext length of FHE when encrypting $s(\lambda)$-bit messages, plus the encoding length of one PRF output. Compute $\mathsf{hk} \leftarrow \mathsf{SEH.Gen}(1^\lambda, 1^{|C|\cdot B}, 1^{s(\lambda)\cdot B})$.
- Let $|\mathsf{td}|$ denote the size of the SEH trapdoor generated by $\mathsf{TGen}(1^\lambda, 1^{|C|\cdot B}, 1^{s(\lambda)\cdot B}, \cdot)$, and $|\mathsf{sPRF}|$ denote the size of shiftable PRF master output $(\mathsf{pp}, k) \leftarrow \mathsf{sPRF.Setup}(1^\lambda)$. Compute the dummy ciphertexts $\mathsf{ct}_{\mathsf{FHE}} \leftarrow \mathsf{FHE.Enc}(\mathsf{pk}, 0^{|\mathsf{td}|+|\mathsf{sPRF}|\cdot s(\lambda)})$.
- For each gate g of the circuit C with description $\langle g \rangle = (\mathsf{in}, \mathsf{out}, f_{\mathsf{out}})$, compute the obfuscated gate $\widetilde{\mathsf{Gate}_g} = \mathsf{iO}(1^\lambda, \mathsf{Gate}_{[\langle g \rangle, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{vk}_i\}_{i \in \mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{sk}_o\}_{o \in \mathsf{out}}, \mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}]})$ where the circuit Gate is defined below in Figure 1.
- Output the final obfuscated circuit $\widetilde{C}$ described as follows.

- The circuit has hardcoded: the hash key hk, shiftable PRF keys (pp_i, k_i) and constrained signature keys sk_i for each input wire i of the circuit, shiftable PRF keys (pp_o, k_o) for each output wire o of the circuit, and all the obfuscated gates $\{\widetilde{\mathsf{Gate}}_g\}$.
- Evaluate the circuit by the wire index as follows:
  * Maintain the string $m_i = \{(\mathsf{ct}_j, h_j)\}_{j<i}$, which denotes the concatenation of all previous wire ciphertexts and compressed hashes (ct_j, h_j), padded with zero blocks to length $|C| \cdot B$, as follows:
  * Compute $H_i \leftarrow \mathsf{SEH.Hash}(\mathsf{hk}, m_i)$. Compute the compressed hash $h_i = \mathsf{FHE.Eval}(\mathsf{pk}, \mathsf{Ext\&Dec}(\mathsf{hk}, H_i, \cdot), \mathsf{ct}_{\mathsf{FHE}})$ as in the circuit Gate, and where the function $\mathsf{Ext\&Dec}$ is defined in Figure 2.
  * If i is an input wire of the original circuit C, compute the ciphertext $\mathsf{ct}_i = x_i + \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i)$ and the signature $\sigma_i \leftarrow \mathsf{Sign}(\mathsf{sk}_i, (\mathsf{ct}_i, h_i))$.
  * If i is an output wire of some gate g, then for every input wire j of g, compute the opening $\rho_j \leftarrow \mathsf{SEH.Open}(\mathsf{hk}, m_i, [(j-1)B, jB-1])$, i.e. the opening for the j-th block of m_i, and the prefix consistency proof $\pi_j \leftarrow \mathsf{SEH.ProveCons}(\mathsf{hk}, m_i, m_j, (j-1)B)$, i.e., the proof that the non-zero parts of m_j are consistent with the prefix of m_i. Run the obfuscated gate $\widetilde{\mathsf{Gate}}_g(H_i, i, \{H_j, \mathsf{ct}_j, \rho_j, \pi_j, \sigma_j\}_{j\in\mathsf{in}})$ to obtain $(\mathsf{ct}_i, \sigma_i)$.
- Finally, for each output wire o of the circuit, decrypt the wire ciphertext ct_o by $x_o = \mathsf{ct}_o - \mathsf{Eval}(\mathsf{pp}_o, k_o, h_o)$ and output the wire values $\{x_o\}$.

$\underline{\mathsf{Gate}_{[\langle g\rangle, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{vk}_i\}_{i\in\mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{sk}_o\}_{o\in\mathsf{out}}, \mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}]}(H_o, o, \{H_i, \mathsf{ct}_i, \rho_i, \pi_i, \sigma_i\}_{i\in\mathsf{in}})}$

Hardcoded values:

- The description $\langle g\rangle$ of the gate g with input wire set in and output wire set out, along with the computed function $f_{\mathsf{out}} = \left\{f_o : \{0,1\}^{|\mathsf{in}|} \to \{0,1\}\right\}$ for each output wire $o \in \mathsf{out}$.
- Hash key hk for SEH.
- For each input wire $i \in \mathsf{in}$, a shiftable PRF setup (pp_i, k_i) and a constrained signature verification key vk_i; for each output wire $o \in \mathsf{out}$, a shiftable PRF setup (pp_o, k_o) and a constrained signature signing key sk_o.
- FHE public key pk and FHE ciphertexts $\mathsf{ct}_{\mathsf{FHE}}$ of $|\mathsf{td}| + |\mathsf{sPRF}| \cdot s(\lambda)$ bits.

Inputs:

- The index o of the output wire to be computed, and the hash value H_o which is supposed to be the SEH hash of $\{(\mathsf{ct}_j, h_j)\}_{j<o}$.
- For each input wire $i \in \mathsf{in}$, the following values:
  - SEH hash value H_i which is supposed to be the SEH hash of $\{(\mathsf{ct}_j, h_j)\}_{j<i}$.
  - Ciphertext ct_i supposed to be $x_i + \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i)$ where x_i is the input wire value.
  - Opening ρ_i for H_o at block i. For honest input H_o, ρ_i should open to (ct_i, h_i) where h_i is a compressed hash computed from H_i.
  - Prefix consistency proof π_i showing that H_i is consistent with H_o for the first $i-1$ blocks.
  - Signature σ_i of message (ct_i, h_i) under signing key sk_i.

Evaluation Procedure:

1. Compute the compressed hash h_o, which is an FHE ciphertext obtained by homomorphically evaluating the function $\mathsf{Ext\&Dec}(\mathsf{hk}, H_o, \cdot)$ on the hardcoded ciphertext $\mathsf{ct}_{\mathsf{FHE}}$:

$$h_o \leftarrow \mathsf{FHE.Eval}(\mathsf{pk}, \mathsf{Ext\&Dec}(\mathsf{hk}, H_o, \cdot), \mathsf{ct}_{\mathsf{FHE}}),$$

where the function $\mathsf{Ext\&Dec}$ is defined in Figure 2. For each $i \in \mathsf{in}$, similarly compute

$$h_i \leftarrow \mathsf{FHE.Eval}(\mathsf{pk}, \mathsf{Ext\&Dec}(\mathsf{hk}, H_i, \cdot), \mathsf{ct}_{\mathsf{FHE}}).$$

2. If $o \notin \mathsf{out}$, output $\perp$. Otherwise, for each input wire $i \in \mathsf{in}$, run the following checks and output $\perp$ if any of them fails:
   - Run $\mathsf{SEH.Ver}(\mathsf{hk}, H_o, \big[(i-1)B : iB - 1\big], (\mathsf{ct}_i, h_i), \rho_i)$ to verify the opening ρ_i, which certifies that the hash H_o opens to (ct_i, h_i) on the specified block interval. Here we abuse the notation to denote opening verification on block intervals.
   - Run $\mathsf{SEH.VerCons}(\mathsf{hk}, H_i, H_o, (i-1)B, \pi_i)$ to verify the prefix consistency proof π_i for the first $(i-1)B$ blocks, which certifies that H_i is consistent with H_o up to the first $(i-1)B$ bits / $(i-1)$ blocks.
   - Run $\mathsf{Sig.Ver}(\mathsf{vk}_i, (\mathsf{ct}_i, h_i), \sigma_i)$ to verify the signature σ_i on message (ct_i, h_i) under verification key vk_i.
3. Decrypt each input wire value $x_i = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i)$ for each $i \in \mathsf{in}$. If $x_i \notin \{0,1\}$, output $\perp$. Otherwise, compute the output wire value $x_o = f_o(\{x_i\}_{i \in \mathsf{in}})$.
4. Compute the output ciphertext $\mathsf{ct}_o = x_o + \mathsf{Eval}(\mathsf{pp}_o, k_o, h_o)$ and compute the signature $\sigma_o \leftarrow \mathsf{Sign}(\mathsf{sk}_o, (\mathsf{ct}_o, h_o))$. Here $x_o \in \{0,1\}$ is treated as an element in the ring R.
5. Output $(\mathsf{ct}_o, \sigma_o)$.

Figure 1: Definition of the Gate circuit

$$\underline{\mathsf{Ext\&Dec}\big(\mathsf{hk}, H, (\mathsf{td}, \{(\mathsf{pp}_i, k_i)\}_{i \in S.\mathsf{in}})\big)}$$

1. Extract $\{(\mathsf{ct}_i, h_i)\}_{i \in S.\mathsf{in}} \leftarrow \mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H)$.
2. Decrypt each ciphertext by $x_i = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i)$ for each $i \in S.\mathsf{in}$. If $x_i \notin \{0,1\}$, set $x_i = 0$.
3. Output a length $s(\lambda)$ bit string $\{x_i\}_{i \in S.\mathsf{in}}$.

Figure 2: Definition of the Extract-and-Decrypt function $\mathsf{Ext\&Dec}$

The correctness of the above construction follows directly from the correctness of the underlying primitives. Therefore, it suffices to prove the security against Δ-equivalent circuits. Similar to the proof in [JJ22], we start by the following core theorem regarding δ-equivalent circuits.

Theorem 5.1 (δ-security of Construction 5.1)**.** *Let $\{C_\lambda^0\}$ and $\{C_\lambda^1\}$ be two ensembles of topologically identical circuits, where every C_λ^0 and C_λ^1 differs only on a sub-circuit S_λ of size at most $s(\lambda) = O(\log \lambda)$. We suppress the subscript λ in the notation for simplicity. Let $S.\mathsf{in}$ and $S.\mathsf{out}$ denote the input and output wires of the sub-circuit S. Then, under polynomial security of iO, of FHE, of SEH with prefix consistency proof, of shiftable PRF, and of constrained signature, the distribution $\mathcal{D}_0, \mathcal{D}_1$ defined as follows are (polynomially) indistinguishable:*

$$\mathcal{D}_b \ : \ (\mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{sk}_i\}_{i \in S.\mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{vk}_o\}_{o \in S.\mathsf{out}}, \{\widetilde{\mathsf{Gate}}_g^b\}_{g \in S}),$$

where the elements $(\mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{sk}_i\}_{i \in S.\mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{vk}_o\}_{o \in S.\mathsf{out}})$ are generated as in the obfuscation procedure of Construction 5.1 (which has identical distribution when obfuscating C^0 or C^1), and each obfuscated gate

$$\widetilde{\mathsf{Gate}}_g^b = \mathsf{iO}(1^\lambda, \mathsf{Gate}_{[\langle g^b \rangle, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{vk}_i\}_{i \in \mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{sk}_o\}_{o \in \mathsf{out}}, \mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}]}),$$

is the gate obfuscation of gate g (with description $\langle g^b \rangle$ in circuit C^b) computed as in Construction 5.1 when obfuscating circuit C^b.

Before proving the theorem, we first note that the distribution of $\Delta\mathsf{iO}(1^\lambda, C^b)$ can be perfectly simulated given $\mathcal{D}_b$, as every element in $\mathcal{D}_b$ does not depend on the rest of the components of the $\Delta\mathsf{iO}$ computation. Therefore, Theorem 5.1 directly implies the indistinguishability of $\Delta\mathsf{iO}(1^\lambda, C^0)$ and $\Delta\mathsf{iO}(1^\lambda, C^1)$. Following a direct hybrid argument of $\ell(\lambda)$ steps of subcircuit replacements, we immediately conclude that Construction 5.1 is a secure $\Delta\mathsf{iO}$ scheme for circuits (Definition 3.10).

Proof of Theorem 5.1. Consider the following sequence of hybrids:

- $\mathcal{H}_0^b$: This is the distribution $\mathcal{D}_b$ as defined in the theorem statement.
- $\mathcal{H}_1^b$: Same as $\mathcal{H}_0^b$, but the SEH hash key is generated with a trapdoor, i.e., $(\mathsf{hk}, \mathsf{td}) \leftarrow \mathsf{SEH.TGen}(1^\lambda, 1^{|C|\cdot B}, 1^{s(\lambda)\cdot B}, E)$, where the extraction set is $E = \bigcup_{i\in S.\mathsf{in}}\big[(i-1)B : iB-1\big]$. In other words, the extraction set E contains all the blocks corresponding to the input wires of the sub-circuit S.

  Claim 5.1. Assuming SEH satisfies key-indistinguishability, Hybrids $\mathcal{H}_0^b$ and $\mathcal{H}_1^b$ are indistinguishable.

- $\mathcal{H}_2^b$: Same as $\mathcal{H}_1^b$, but generate the FHE ciphertext $\mathsf{ct}_{\mathsf{FHE}}$ to be an encryption of the SEH trapdoor td along with all the shiftable PRF keys $\{(\mathsf{pp}_i, k_i)\}_{i\in S.\mathsf{in}}$ for input wires of the sub-circuit S.

  Claim 5.2. Assume FHE is semantically secure, Hybrids $\mathcal{H}_1^b$ and $\mathcal{H}_2^b$ are indistinguishable.

  Note that in $\mathcal{H}_2^b$, for honest SEH hash values $H_i \leftarrow \mathsf{Hash}(\mathsf{hk}, \{(\mathsf{ct}_j, h_j)\}_{j<i})$ where each ciphertext ct_j encrypts a bit $x_j = \mathsf{ct}_j - \mathsf{Eval}(\mathsf{pp}_j, k_j, h_j)$, the compressed hash

  $$h_i \leftarrow \mathsf{FHE.Eval}(\mathsf{pk}, \mathsf{Ext\&Dec}(\mathsf{hk}, H_i, \cdot), \mathsf{ct}_{\mathsf{FHE}}) \in \mathsf{FHE.Enc}(\mathsf{pk}, \{\overline{x}_j\}_{j\in S.\mathsf{in}})$$

  is a ciphertext of some bit string $\{\overline{x}_j\}_{j\in S.\mathsf{in}}$ where $\overline{x}_j = x_j$ if $j < i$. More generally, for any SEH hash value H^*, for any index $j \in S.\mathsf{in}$, if the j-th block of the extraction $\mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H^*)$ is (ct_j, h_j), and that $x_j = \mathsf{ct}_j - \mathsf{Eval}(\mathsf{pp}_j, k_j, h_j)$ is a bit, then $\overline{x}_j = x_j$.

Next, we consider the following sequence of sub-hybrids for each gate $\{g_t\}$ in the sub-circuit S in topological order. For every wire $i \in S$, let $C^b_{S,i}$ denote the sub-circuit of C^b_S that takes the input bits of the subcircuits $\{x_j\}_{j\in S.\mathsf{in}}$ and outputs the value on wire i of the evaluation $C^b_S(\{x_j\}_{j\in S.\mathsf{in}})$.

- $\mathcal{H}^b_{3,t,0}$: This hybrid is the same as $\mathcal{H}^b_{3,t-1,2}$ (or $\mathcal{H}^b_2$ if $t = 1$), but compute the obfuscation of gate $\widetilde{\mathsf{Gate}}_{g_t}$ using the modified gate circuit Gate' defined below in Figure 3.

  $\mathsf{Gate}'_{[\langle g\rangle, \mathsf{hk}, \{\mathsf{pp}_i, \mathsf{vk}_i\}_{i\in\mathsf{in}}, \{\mathsf{pp}_o, k_o, \mathsf{sk}_o\}_{o\in\mathsf{out}}, \mathsf{pk}, \mathsf{FHE.sk}, \mathsf{ct}_{\mathsf{FHE}}]}(H_o, o, \{H_i, \mathsf{ct}_i, \rho_i, \pi_i, \sigma_i\}_{i\in\mathsf{in}})$

  Same as the original Gate function defined in Figure 1, except that it additionally hardcodes the FHE secret key $\mathsf{FHE.sk}$, while removing the hardcoded shiftable PRF key k_i for its input wires. In the evaluation procedure, after all the checks pass, instead of decrypting each input wire value $x_i = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i)$, it computes the output wire value by:
  - Decrypting h_o using the FHE secret key $\mathsf{FHE.sk}$ to obtain

    $$\{\overline{x}_j\}_{j\in S.\mathsf{in}} \leftarrow \mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o).$$

  - Computing the output wire value $x_o = C^b_{S,o}(\{\overline{x}_j\}_{j\in S.\mathsf{in}})$ where $C^b_{S,o}$ is defined above.

  The rest of the evaluation procedure remains unchanged.

Figure 3: Definition of the Gate' circuit with extracted computation

We claim that the circuit Gate' defined above is functionally equivalent to the original circuit Gate in $\mathcal{H}^b_{3,t-1,2}$. To do so, we rely on the following lemma stating that in $\mathcal{H}^b_{3,t-1,2}$, the gate obfuscation $\widetilde{\mathsf{Gate}}_{g_t}$ enforces input soundness for wires that are outputs of other gates in the sub-circuit S.

Lemma 5.1 (Input soundness of $\mathcal{H}^b_{3,t-1,2}$)**.** *Assuming the statistical soundness property of* Sig*, in* $\mathcal{H}^b_{3,t-1,2}$*, for every input wire* $i \in \mathsf{in}$ *of gate* g_t*, if* $i \notin S.\mathsf{in}$*, i.e., i is an output wire of some other gate* $g_{t'}$ *in the sub-circuit S where* $t' < t$*, then if signature verification* $\mathsf{Sig.Ver}(\mathsf{vk}_i, (\mathsf{ct}_i, h_i), \sigma_i)$ *passes, then it holds that*

$$C^b_{S,i}(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_i)) = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i).$$

We defer the proof of the lemma to the introduction of $\mathcal{H}^b_{3,t,2}$ below.

With the above claim, we can now argue the functional equivalence between Gate and Gate'. It suffices to show that, for any input $(H_o, o, \{H_i, \mathsf{ct}_i, \rho_i, \pi_i, \sigma_i\}_{i\in\mathsf{in}})$ to gate g_t, if all checks pass, the decrypted input values computed in the original circuit Gate are identical to the decrypted-and-computed values in the modified circuit Gate'. Formally, we need to show that for every input wire $i \in \mathsf{in}$ of gate g_t where all check passes, it holds that

$$x_i = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i) = C^b_{S,i}(\{\overline{x}_j\}), \quad \text{where } \{\overline{x}_j\}_{j\in S.\mathsf{in}} \leftarrow \mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o)$$

We separate the argument into two cases:

- Case 1: $i \in S.\mathsf{in}$. In this case, the subcircuit $C^b_{S,i}$ is simply the identity function on input wire i, i.e., $C^b_{S,i}(\{\overline{x}_j\}) = \overline{x}_i$. By the definition of $\mathsf{Ext\&Dec}$ and the correctness of FHE evaluation, $\overline{x}_i$ is equal to the output of the following computation:

  $$(\mathsf{ct}^*_i, h^*_i) \leftarrow \mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H_o)\big[(i-1)B : iB-1\big], \quad \overline{x}_i = \mathsf{ct}^*_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h^*_i).$$

  On the other hand, since the circuit Gate executes the opening check $\mathsf{Ver}(\mathsf{hk}, H_o, \big[(i-1)B : iB-1\big](\mathsf{ct}_i, h_i), \rho_i)$, by the somewhere extractability of SEH, we have $(\mathsf{ct}_i, h_i) = (\mathsf{ct}^*_i, h^*_i)$, and therefore $x_i = \overline{x}_i$.
- Case 2: $i \notin S.\mathsf{in}$. In this case, by Lemma 5.1, we have

  $$x_i = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i) = C^b_{S,i}(\{\overline{x}'_j\}), \quad \text{where } \{\overline{x}'_j\}_{j\in S.\mathsf{in}} \leftarrow \mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_i)$$

  It remains to show that the FHE decryption outcome $\{\overline{x}'_j\}_{j\in S.\mathsf{in}}$ of h_i is consistent with $\{\overline{x}_j\}_{j\in S.\mathsf{in}}$ of h_o. Using the same argument from Case 1, for every subcircuit input wire $j \in S.\mathsf{in}$, $\overline{x}'_j$ is equal to the output of the following computation:

  $$(\mathsf{ct}^*_j, h^*_j) \leftarrow \mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H_i)\big[(j-1)B : jB-1\big], \quad \overline{x}_j = \mathsf{ct}^*_j - \mathsf{Eval}(\mathsf{pp}_j, k_j, h^*_j).$$

  By the fact that prefix consistency proof check $\mathsf{VerCons}(\mathsf{hk}, H_i, H_o, (i-1)B, \pi_i)$ passes, and the somewhere statistical soundness of the SEH prefix consistency proof, we have that for every $j \in S.\mathsf{in}$ with index $j < i$, it holds that

  $$\mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H_i)\big[(j-1)B : jB-1\big] = \mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H_o)\big[(j-1)B : jB-1\big].$$

  Therefore we have $\overline{x}'_j = \overline{x}_j$ for every $j \in S.\mathsf{in}$ with index $j < i$. Finally, since the wires are evaluated in topological order, the subcircuit $C^b_{S,i}$ only depends on input wires $j \in S.\mathsf{in}$ with index $j < i$. Therefore, we have $C^b_{S,i}(\{\overline{x}'_j\}) = C^b_{S,i}(\{\overline{x}_j\})$.

Overall, we have shown the following.

Claim 5.3. Assuming the security of iO, the correctness of FHE, the somewhere extractability and somewhere statistical soundness of SEH, and that the input soundness (Lemma 5.1) holds for gate g_t in $\mathcal{H}^b_{3,t-1,2}$, Hybrids $\mathcal{H}^b_{3,t,0}$ and $\mathcal{H}^b_{3,t-1,2}$ are indistinguishable.

- $\mathcal{H}^b_{3,t,1}$: Same as $\mathcal{H}^b_{3,t,0}$, but for every output wire o of gate g_t, if $o \notin S.\mathsf{out}$, i.e., o is an inner wire taken as input by some other gate $g_{t'}$ in the sub-circuit S, change the constrained signature signing key sk_o, which was originally constrained to the all-one function, to be constrained to the function that only signs messages (ct_o, h_o) satisfying the relation

$$C^b_{S,o}(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o)) = \mathsf{ct}_o - \mathsf{Eval}(\mathsf{pp}_o, k_o, h_o).$$

  Since every message the circuit Gate' would sign on output wire o already satisfies the above relation, and that the deterministic signing property of constrained signature guarantees that the signatures generated with the all-one constraint are identical to those generated with this new constraint, we conclude that the circuit Gate' in $\mathcal{H}^b_{3,t,0}$ and $\mathcal{H}^b_{3,t,1}$ are functionally equivalent.

  Claim 5.4. Assuming the security of iO and the deterministic signing property of Sig, Hybrids $\mathcal{H}^b_{3,t,1}$ and $\mathcal{H}^b_{3,t,0}$ are indistinguishable.

- $\mathcal{H}^b_{3,t,2}$: Same as $\mathcal{H}^b_{3,t,1}$, but for every output wire o of gate g_t, if $o \notin S.\mathsf{out}$, i.e., o is an inner wire taken as input by some other gate $g_{t'}$ in the sub-circuit S, change the constrained signature public key vk_o hardcoded in gate $g_{t'}$ to be constrained to the function that only verifies signatures on messages (ct_o, h_o) satisfying the relation

$$C^b_{S,o}(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o)) = \mathsf{ct}_o - \mathsf{Eval}(\mathsf{pp}_o, k_o, h_o).$$

  Claim 5.5. Assuming the constraint hiding property of Sig, Hybrids $\mathcal{H}^b_{3,t,2}$ and $\mathcal{H}^b_{3,t,1}$ are indistinguishable.

  With this change, the input soundness lemma (Lemma 5.1) of gate g_{t+1} now follows directly from the statistical soundness of Sig.

  Proof of Lemma 5.1. Consider any input wire $i \in \mathsf{in}$ of gate g_{t+1} where $i \notin S.\mathsf{in}$, i.e., i is the output of gate $g_{t'}$ for some $t' \leq t$. From the change by $\mathcal{H}^b_{3,t',2}$, the signature verification key vk_i hardcoded in gate $\widetilde{\mathsf{Gate}}_{g_{t+1}}$ is constrained. Therefore, by the statistical soundness of constrained signature, if the signature verification $\mathsf{Sig.Ver}(\mathsf{vk}_i, (\mathsf{ct}_i, h_i), \sigma_i)$ passes, it must hold that

$$C^b_{S,i}(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_i)) = \mathsf{ct}_i - \mathsf{Eval}(\mathsf{pp}_i, k_i, h_i).$$

  which is exactly the statement of the lemma. □

With these sub-hybrids, we now reach a distribution where every obfuscated gate does not compute gate-evaluation directly, but instead computes from the (extracted) subcircuit inputs. We continue the proof with the following hybrids.

- $\mathcal{H}_4^b$: This is $\mathcal{H}_{3,t_S,2}^b$, where t_S is the number of gates in the sub-circuit S.

  For every obfuscated gate $\widetilde{\mathsf{Gate}}_g$ in the sub-circuit S, the output ciphertext ct_o is always computed as

  $$\mathsf{ct}_o = C_{S,o}^b(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o)) + \mathsf{Eval}(\mathsf{pp}_o, k_o, h_o).$$

- $\mathcal{H}_5^b$: Same as $\mathcal{H}_4^b$, but for every gate g with output wire $o \notin S.\mathsf{out}$, compute the shifted-by-zero PRF key

  $$k_{o,\mathbf{0}} \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}_o, k_o, \mathbf{0}).$$

  Modify the computation of Gate'_g to compute ct_o as

  $$\mathsf{ct}_o = C_{S,o}^b(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o)) + \mathsf{EvalShifted}(\mathsf{pp}_o, k_{o,\mathbf{0}}, h_o).$$

  Claim 5.6. Assuming the security of iO and the correctness of sPRF, Hybrids $\mathcal{H}_5^b$ and $\mathcal{H}_4^b$ are indistinguishable.

  Note that in $\mathcal{H}_5^b$, the shifted key $k_{o,\mathbf{0}}$ is the only variable depending on the master key k_o, as the gate g' taking wire o as input is now obfuscated using the circuit Gate' which does not hardcode PRF keys for its input wires. (In contrast, for wires $o \in S.\mathsf{out}$, the key k_o is part of the output distribution.)

- $\mathcal{H}_6^b$: Same as $\mathcal{H}_5^b$, but for every gate g with output wire $o \notin S.\mathsf{out}$, compute the shifted PRF key

  $$k_{o,f_o^b} \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}_o, k_o, f_o^b).$$

  Modify the computation of Gate'_g to compute ct_o as

  $$\mathsf{ct}_o = \mathsf{EvalShifted}(\mathsf{pp}_o, k_{o,f_o^b}, h_o),$$

  where the function f_o^b is defined by $f_o^b(h) = C_{S,o}^b(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h))$.

  Claim 5.7. Assuming the security of iO and the correctness of shiftable PRF, Hybrids $\mathcal{H}_6^b$ and $\mathcal{H}_5^b$ are indistinguishable.

- $\mathcal{H}_7^b$: Same as $\mathcal{H}_6^b$, but for every gate g with output wire $o \notin S.\mathsf{out}$, compute the shifted-by-zero PRF key

  $$k_{o,\mathbf{0}} \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}_o, k_o, \mathbf{0}).$$

  Modify the computation of Gate'_g to compute ct_o as

  $$\mathsf{ct}_o = \mathsf{EvalShifted}(\mathsf{pp}_o, k_{o,\mathbf{0}}, h_o).$$

  Claim 5.8. Assuming the shift-hiding property of sPRF, Hybrids $\mathcal{H}_7^b$ and $\mathcal{H}_6^b$ are indistinguishable.

Finally, we argue that $\mathcal{H}_7^0$ and $\mathcal{H}_7^1$ are computationally indistinguishable.

Claim 5.9. Assuming the security of iO, Hybrids $\mathcal{H}_7^0$ and $\mathcal{H}_7^1$ are indistinguishable.

Note that in $\mathcal{H}_7^b$, the only dependence on b is in the obfuscated gates $\widetilde{\mathsf{Gate}}_g$, when computing ct_o for output wires $o \in S.\mathsf{out}$ that are also output wires of the sub-circuit S. Note that by definition the circuit C_S^0 and C_S^1 are functionally equivalent, i.e., for every output wire $o \in S.\mathsf{out}$, the function $C_{S,o}^0$ and $C_{S,o}^1$ are functionally equivalent. Therefore, the computation of ct_o defined by

$$\mathsf{ct}_o = C_{S,o}^b(\mathsf{FHE.Dec}(\mathsf{FHE.sk}, h_o)) + \mathsf{Eval}(\mathsf{pp}_o, k_o, h_o).$$

is functionally equivalent for $b = 0$ and $b = 1$. The indistinguishability thus follows from iO.

Combining all the hybrids above, we conclude that $\mathcal{D}_0$ and $\mathcal{D}_1$ are computationally indistinguishable, which completes the proof. □

Remark 5.1 (On the choice of circuit size bound p of iO)**.** In the above proof of Theorem 5.1, we only obfuscate the circuits Gate or Gate' using iO. Both circuits have size $p_0(\lambda, |C|) = \mathsf{poly}(\lambda, \log|C|)$, where $|C|$ is the size of the original circuit being obfuscated. Therefore, it suffices to assume polynomial security of iO for circuits of size $p(\lambda) = p_0(\lambda, 2^\lambda)$to achieve security for all polynomial-size circuits.

Combining Theorem 5.1 with the fact that all circuits with $\mathcal{EF}$ proof of equivalence are Δ-equivalent (Lemma 3.1), we immediately conclude the main theorem of this section.

Theorem 5.2 (EFiO for Circuits)**.** *There exists a fixed polynomial p such that, assuming polynomial security of iO for $p(\lambda)$-size circuits, of one-way functions, of FHE, and of FSS for bounded-size circuits, for all polynomial $S(\lambda)$, $\ell(\lambda)$, Construction 5.1 with subcircuit size $s = O(\log S + \log \ell)$ is an* EFiO *for all circuits of size at most $S(\lambda)$ with $\mathcal{EF}$ proof of equivalence of length at most $\ell(\lambda)$.*

5.2 Construction of PViO

In this section, we show how to build PViO, i.e., iO for Turing machines with PV proofs of equivalence, from polynomial assumptions. We follow the blueprint of [JJ22] and [MDS25]. We obfuscate the machine M by obfuscating the circuits M_N, which emulate the machine M on inputs of size $n \le N$, where $N = 2^m$ and $m \in [\lambda]$. Because the size of the circuit M_{2^λ} can be exponential, we cannot afford to obfuscate these circuits directly. Instead, we leverage the fact that the machine M is a succinct description of M_N. We do so by obfuscating a circuit $\mathsf{UGate}_{M,N}$, which, on input n, outputs an obfuscation of M_N gate by gate. In our case, the gate-by-gate obfuscations are the circuits Gate we used in Construction 5.1 to build $\Delta\mathsf{iO}$ for circuits.

We unfortunately don't know how to prove security of this construction from polynomial hardness assumptions. Indeed, to prove indistinguishability of $\mathsf{iO}(\mathsf{UGate}_{M,N})$ as is, we seemingly need to invoke iO security (at least) once for every gate in M_N, which can be exponential in λ when $N = 2^\lambda$. To prevent this security loss, we encrypt, for all $m \in [\lambda]$, all obfuscations of $\mathsf{UGate}_{M,N}$, where $N \le N_m = 2^m$ under a time-lock puzzle that opens after time $\ge N_m$. This way, given any polynomial-time adversary $\mathcal{A}$ running in time $\le T$, *all* obfuscations of $\mathsf{UGate}_{M,N}$, where $N \gtrsim T$, will be hidden from $\mathcal{A}$ using at most λ invocations of time-lock-puzzle security, one per index m. Thus, this step only requires the polynomial hardness of the time-lock puzzle. Then, there only remains to argue indistinguishability of at most $\approx T$ obfuscations $\mathsf{iO}(\mathsf{UGate}_{M,N})$, which we do directly by paying the polynomial security loss.

Construction 5.2 ($\Delta\mathsf{iO}$ for Turing Machines)**.** Let $\mathsf{TLP} = (\mathsf{TLP.Gen}, \mathsf{TLP.Solve})$ be a relaxed time-lock puzzle (Definition 3.14), and let $\mathsf{pPRF} = (\mathsf{pPRF.Punc}, \mathsf{pPRF.Eval})$ be a puncturable PRF. The $\Delta\mathsf{iO}$ for Turing machines is constructed as follows.

The obfuscation for subcircuit size parameter $s(n) = O(\log n)$ is defined as follows.

$\Delta\mathsf{iO}(1^\lambda, M)$:

- For each $m \in [\lambda]$, set an input length bound $N_m = 2^m$. Sample a master puncturable PRF key $K^m \leftarrow \{0,1\}^\lambda$, and compute the obfuscated universal gate

$$\widetilde{\mathsf{UGate}}_m = \mathsf{iO}(1^\lambda, \mathsf{UGate}_{[M,N_m,K^m]}),$$

  where the circuit UGate is defined below in Figure 4.

- Encrypt each obfuscated universal gate under a time-lock puzzle:

$$\widetilde{\mathsf{ct}}_m \leftarrow \mathsf{TLP.Gen}(1^\lambda, N_m, \widetilde{\mathsf{UGate}}_m),$$

- Output the final obfuscated Turing machine $\widetilde{M}$ described as follows.
  - On input x, compute $m = \lceil \log |x| \rceil$ and solve the time-lock puzzle to obtain $\widetilde{\mathsf{UGate}}_m \leftarrow \mathsf{TLP.Solve}(\widetilde{\mathsf{ct}}_m)$.
  - Evaluate the obfuscated universal gate $\widetilde{\mathsf{UGate}}_m(|x|, i, 0)$ for every input/output wire i of the circuit $M_{|x|}$ and $\widetilde{\mathsf{UGate}}_m(|x|, 0, g)$ for every gate g in the circuit $M_{|x|}$ to obtain the obfuscated circuit description of $M_{|x|}$ of the form

$$(\mathsf{hk}, \mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}, \{(\mathsf{pp}_i, k_i), \mathsf{sk}_i\}_{i\in\mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{vk}_o\}_{o\in\mathsf{out}}, \{\widetilde{\mathsf{Gate}}_g\}_{g\in M_{|x|}}).$$

  - Evaluate the circuit $M_{|x|}$ on input x following the evaluation procedure defined in Construction 5.1.

$\underline{\mathsf{UGate}_{[M,N_m,K^m]}(n,i,g)}$

Hardcoded values: Turing machine M, input length bound N_m, and puncturable PRF key K^m.
Inputs: Input length $n \le N_m$, the wire-mode index i, and the gate-mode index g.
Evaluation Procedure:

1. If $n > N_m$, output $\perp$. Otherwise, let M_n be the circuit described by the succinct circuit description $[\![M]\!]_{N_m}(n,\cdot)$ on length n. We assume that its topology is fixed.
2. Generate the following setups as in Construction 5.1 for obfuscating M_n using randomness $y_0 \leftarrow \mathsf{pPRF.Eval}(K^m, (n,0,0))$:
   - FHE key $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{FHE.KeyGen}(1^\lambda)$.
   - SEH hash key $\mathsf{hk} \leftarrow \mathsf{SEH.Gen}(1^\lambda, 1^{|M_n|\cdot B}, 1^{s(n)\cdot B})$, where B is the block size determined by (ct_i, h_i) as in Construction 5.1.
   - Dummy FHE ciphertext $\mathsf{ct}_{\mathsf{FHE}} \leftarrow \mathsf{FHE.Enc}(\mathsf{pk}, 0^{|\mathsf{td}|+|\mathsf{sPRF}|\cdot s(n)})$.
3. The circuit then decides whether the input is requesting information for input/output wires or requesting gate obfuscation.
   - If $g = 0$, i.e., in wire-mode:
     (a) Check whether i is an input or output wire of M_n. If not, output $\perp$.
     (b) Compute randomness $y_i \leftarrow \mathsf{pPRF.Eval}(K^m, (n,i,0))$ for input wire i or $y_i \leftarrow \mathsf{pPRF.Eval}(K^m, (n, 2^\lambda + i, 0))$ for output wire i, then sample the shiftable PRF setup (pp_i, k_i) and constrained signature keys $(\mathsf{vk}_i, \mathsf{sk}_i)$ as in Construction 5.1 using the randomness y_i.

(c) Output

$$\begin{aligned}&(\mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}, \mathsf{hk}, (\mathsf{pp}_i, k_i), \mathsf{sk}_i), && \text{if } i \text{ is an input wire of the circuit,}\\ &(\mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}, \mathsf{hk}, (\mathsf{pp}_i, k_i), \mathsf{vk}_i), && \text{if } i \text{ is an output wire of the circuit.}\end{aligned}$$

Note that by offsetting the PRF evaluation index for the output wires by 2^λ, we ensure that any polynomial number of dummy gates padded to the circuit do not affect the keys sampled for the output wires.

- If $g > 0$, i.e., in gate-mode:
  (a) Obtain the description $\langle g \rangle$ of gate g in circuit M_n by

$$\langle g \rangle = (\mathsf{in}, \mathsf{out}, f_g) \leftarrow [\![M]\!]_{N_m}(n, g).$$

  (b) For each input wire $i \in \mathsf{in}$ and output wire $o \in \mathsf{out}$, using randomness $y_i \leftarrow \mathsf{pPRF.Eval}(K^m, (n, i, 0))$ and $y_o \leftarrow \mathsf{pPRF.Eval}(K^m, (n, o, 0))$ (or $y_o \leftarrow \mathsf{pPRF.Eval}(K^m, (n, 2^\lambda + o, 0))$ if o is also an output wire of the full circuit) respectively to sample the shiftable PRF setups (pp_i, k_i) and constrained signature keys $(\mathsf{vk}_i, \mathsf{sk}_i)$ on each wire, as in Construction 5.1.
  (c) Using the randomness $\mathsf{pPRF.Eval}(K^m, (n, 0, g))$, compute the obfuscated gate

$$\widetilde{\mathsf{Gate}}_g = \mathsf{iO}(1^\lambda, \mathsf{Gate}_{[\langle g \rangle, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{vk}_i\}_{i \in \mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{sk}_o\}_{o \in \mathsf{out}}, \mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}]}),$$

  where the circuit Gate is defined in Figure 1.
  (d) Finally, output $\widetilde{\mathsf{Gate}}_g$.

Figure 4: Definition of the universal gate UGate

The correctness of the above construction follows directly from the correctness of Construction 5.1 and the time-lock puzzle.

Theorem 5.3 (Efficiency of Construction 5.2)**.** *For any Turing machine M with polynomial running time $T(\cdot)$, the obfuscation $\Delta\mathsf{iO}(1^\lambda, M)$ defined in Construction 5.2 runs in time $\mathsf{poly}(\lambda)$, and for any input x, the evaluation $\widetilde{M}(x)$ runs in time $\mathsf{poly}(\lambda, T(|x|))$.*

Proof. Note that the universal gate circuit UGate computes gate obfuscations locally, and its size is poly-logarithmic in the size of the circuit M_n. Therefore, by the efficiency of iO, computing the obfuscation $\widetilde{\mathsf{UGate}}_m$ takes time $\mathsf{poly}(\lambda, \log T(N_m)) = \mathsf{poly}(\lambda)$. The time-lock puzzle generation also runs in time $\mathsf{poly}(\lambda, \log N_m) = \mathsf{poly}(\lambda)$. Therefore, the obfuscation $\Delta\mathsf{iO}(1^\lambda, M)$ runs in time $\mathsf{poly}(\lambda)$.

For the evaluation efficiency, note that on input x, solving the time-lock puzzle $\mathsf{TLP.Solve}(\widetilde{\mathsf{ct}}_m)$ runs in time $\mathsf{poly}(\lambda, N_m) = \mathsf{poly}(\lambda, |x|)$. Evaluating the obfuscated universal gate $\widetilde{\mathsf{UGate}}_m$ for every input/output wire and gate in circuit $M_{|x|}$ runs in time $\mathsf{poly}(\lambda) \cdot |M_{|x|}| = \mathsf{poly}(\lambda, T(|x|))$. Finally, evaluating the obfuscated circuit on input x also runs in time $\mathsf{poly}(\lambda, T(|x|))$. Therefore, the overall evaluation time $\widetilde{M}(x)$ runs in time $\mathsf{poly}(\lambda, T(|x|))$. □

Theorem 5.4 (Δ-iO security of Construction 5.2)**.** *Under the polynomial security of relaxed time-lock puzzle, of puncturable PRF, of FHE, of SEH with proof of prefix consistency, of shiftable PRF, of constrained signature, and of iO for circuits, Construction 5.2 is a secure $\Delta\mathsf{iO}$ for Turing machines.*

Proof. Let M^0 and M^1 be two Δ-equivalent Turing machines with subcircuit size $s(n)$. Recalling from Definition 3.11, there exists $\ell = \mathsf{poly}(n, \log N)$ and $D = \mathsf{poly}(\log N)$ such that for every input length bound N and every input length $n \le N$, there exists a sequence of circuits $\{C'_1, C'_2, \ldots, C'_{\ell(n,N)}\}$ such that

- C'_1 is functionally equivalent to $[\![M_0]\!]_N(n, \cdot)$ and $C'_{\ell(n,N)}$ is functionally equivalent to $[\![M_1]\!]_N(n, \cdot)$.
- Each circuit C'_i has size at most $D(N)$.
- For any $i \in [\ell(n, N) - 1]$, let C_i and C_{i+1} be the circuit described by the succinct description C'_i and C'_{i+1} respectively. Then C_i and C_{i+1} are δ-equivalent with subcircuit size $s(n)$.

At a high level, we prove the security by a hybrid argument that gradually replaces the succinct circuit description $[\![M^0]\!]_N(n, \cdot)$ with $[\![M^1]\!]_N(n, \cdot)$ using the sequence of circuits $\{C'_i\}$. However, since we have $N_\lambda = 2^\lambda$ in the obfuscation construction, the potential input length n can be exponential, and so is the number of hybrids $\ell(n, N_\lambda)$. To avoid the issue, we first rely on the security of time-lock puzzle to restrict the accessible input lengths for any polynomial-time adversary.

Formally, consider the following hybrids:

- $\mathcal{H}_0^b$: This hybrid outputs the obfuscation $\Delta\mathsf{iO}(1^\lambda, M^b)$ as defined in Construction 5.2.
- $\mathcal{H}_0^{b,t}$: For every polynomial $t(\lambda)$, this hybrid is the same as $\mathcal{H}_0^b$, except that for every m such that $N_m = 2^m > t(\lambda)$, the time-lock puzzle ciphertext $\widetilde{\mathsf{ct}}_m$ is generated to be an encryption of zeros instead of the obfuscated universal gate $\widetilde{\mathsf{UGate}}_m$.

Claim 5.10. Assuming the polynomial security of relaxed time-lock puzzle with gap ϵ, for every adversary $\mathcal{A}$ running in time $t(\lambda)$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\left[\mathcal{A}(\mathcal{H}_0^0) = 1\right] - \Pr\left[\mathcal{A}(\mathcal{H}_0^1) = 1\right]\right| \le \left|\Pr\left[\mathcal{A}(\mathcal{H}_0^{0,t'}) = 1\right] - \Pr\left[\mathcal{A}(\mathcal{H}_0^{1,t'}) = 1\right]\right| + \mathsf{negl}(\lambda).$$

where $t'(\lambda) = t(\lambda)^{1/\epsilon}$ is a polynomial.

Notice that the above claim only argues security against bounded *time* adversaries. The claim therefore follows from a direct invocation of the relaxed time-lock puzzle security definition (Definition 3.14).

Now, it remains to prove the indistinguishability between $\mathcal{H}_0^{0,t}$ and $\mathcal{H}_0^{1,t}$ for every fixed polynomial $t(\lambda)$. We now consider the following intermediate hybrids indexed by a length-bound parameter $m_0 = 0, 1, \ldots, \lceil \log t \rceil$, a length parameter $n_0 \le 2^{m_0} \le \mathsf{poly}(\lambda)$, and $0 \le \ell_0 \le \ell(n_0, N_{m_0}) + 1$.

- $\mathcal{H}^t_{m_0,n_0,\ell_0}$: Let $\{C'_1, C'_2, \ldots, C'_{\ell(n_0,N_{m_0})}\}$ be the sequence of circuits defined between $[\![M^0]\!]_{N_{m_0}}(n_0, \cdot)$ and $[\![M^1]\!]_{N_{m_0}}(n_0, \cdot)$ as in the definition of Δ-equivalence for Turing machines. To simplify the notation, we let $C'_0 = [\![M^0]\!]_{N_{m_0}}(n_0, \cdot)$ and $C'_{\ell(n_0,N_{m_0})+1} = [\![M^1]\!]_{N_{m_0}}(n_0, \cdot)$.

  The hybrid computes each obfuscated universal gate $\widetilde{\mathsf{UGate}}_m$ by:

  - If $m < m_0$, compute $\widetilde{\mathsf{UGate}}_m \leftarrow \mathsf{iO}(1^\lambda, \mathsf{UGate}_{[M^1,N_m,K^m]})$.
  - If $m > m_0$, compute $\widetilde{\mathsf{UGate}}_m \leftarrow \mathsf{iO}(1^\lambda, \mathsf{UGate}_{[M^0,N_m,K^m]})$.
  - If $m = m_0$, compute $\widetilde{\mathsf{UGate}}_m \leftarrow \mathsf{iO}(1^\lambda, \mathsf{UGate}'_{[M^0,M^1,n_0,C'_{\ell_0},N_{m_0},K^{m_0}]})$, where the circuit UGate' is defined below in Figure 5.

$$\overline{\mathsf{UGate}'_{[M^0,M^1,n_0,C'_{\ell_0},N_{m_0},K^{m_0}]}(n,i,g)}$$

Same as the universal gate $\mathsf{UGate}_{[M,N_{m_0},K^{m_0}]}$ defined in Figure 4, except that it uses the succinct circuit description from either M^0, M^1, or C'_{ℓ_0} depending on the input length n as follows:

- If $n < n_0$, use the succinct description $[\![M^1]\!]_{N_{m_0}}(n,\cdot)$ to obtain the description of M^1_n.
- If $n > n_0$, use the succinct description $[\![M^0]\!]_{N_{m_0}}(n,\cdot)$ to obtain the description of M^0_n.
- If $n = n_0$, use the succinct description C'_{ℓ_0} to obtain the description of C_{ℓ_0}.

Note that the indices of input/output wires are identical across M^0_n, M^1_n, and C_{ℓ_0}, therefore the functionality of UGate' only differs from UGate on gate-mode inputs.

Figure 5: Definition of the intermediate universal gate UGate'

In short, the hybrid uses M^1_n to compute the obfuscation for every index "before" (m_0, n_0), and uses M^0_n for everything "after" (m_0, n_0). On the m_0-th universal gate, and the input length n_0, it uses the ℓ_0-th intermediate circuit C'_{ℓ_0} in the equivalence sequence. Furthermore C'_0 and $C'_{\ell(n_0,N_{m_0})+1}$ correspond to the circuits constructed from M^0 and M^1 in the real scheme.

Claim 5.11. By definition, the following pairs of hybrids are identical.

- $\mathcal{H}^{0,t}_0 \equiv \mathcal{H}^t_{0,0,0}$
- $\mathcal{H}^{1,t}_0 \equiv \mathcal{H}^t_{\lceil\log(t)\rceil,2^{\lceil\log(t)\rceil},\ell(2^{\lceil\log(t)\rceil},N_{\lceil\log(t)\rceil})+1}$
- For every m_0, $\mathcal{H}^t_{m_0,2^{m_0},\ell(2^{m_0},N_{m_0})+1} \equiv \mathcal{H}^t_{m_0+1,0,0}$
- For every m_0 and every $n_0 < 2^{m_0}$, $\mathcal{H}^t_{m_0,n_0,\ell(n_0,N_{m_0})+1} \equiv \mathcal{H}^t_{m_0,n_0+1,0}$

Therefore, to prove the indistinguishability between $\mathcal{H}^{0,t}_0$ and $\mathcal{H}^{1,t}_0$, it suffices to prove the indistinguishability between each pair of neighboring hybrids $\mathcal{H}^t_{m_0,n_0,\ell_0}$ and $\mathcal{H}^t_{m_0,n_0,\ell_0+1}$.

First, we consider the neighboring hybrids where the functionality of the obfuscated universal gate $\widetilde{\mathsf{UGate}}$ is unchanged.

Claim 5.12. Assuming the polynomial security of iO, for all m_0, n_0, $\mathcal{H}^t_{m_0,n_0,0}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,1}$, and $\mathcal{H}^t_{m_0,n_0,\ell(n_0,N_{m_0})}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell(n_0,N_{m_0})+1}$.

The claim follows from the guarantee of Δ-equivalence (Definition 3.11). Let $\{C'_1, \ldots, C'_{\ell(n_0,N_{m_0})}\}$ be the sequence of circuits between $[\![M^0]\!]_{N_{m_0}}(n_0,\cdot)$ and $[\![M^1]\!]_{N_{m_0}}(n_0,\cdot)$, we have the guarantee that C'_1 is functionally equivalent to $C'_0 = [\![M^0]\!]_{N_{m_0}}(n_0,\cdot)$, and $C'_{\ell(n_0,N_{m_0})}$ is functionally equivalent to $C'_{\ell(n_0,N_{m_0})+1} = [\![M^1]\!]_{N_{m_0}}(n_0,\cdot)$. Therefore, by the security of iO, these neighboring hybrids are computationally indistinguishable.

Next, we consider the neighboring hybrids where the functionality of the obfuscated universal gate $\widetilde{\mathsf{UGate}}$ changes from using C'_{ℓ_0} to C'_{ℓ_0+1} for some $\ell_0 \in [\ell(n_0, N_{m_0})]$, where the induced circuit C_{ℓ_0} and C_{ℓ_0+1} are guaranteed to be δ-equivalent with subcircuit size $s(n_0)$. We rely on the fact that the output of $\widetilde{\mathsf{UGate}}$ forms a $\Delta\mathsf{iO}$ output for the circuit C_{ℓ_0} and C_{ℓ_0+1} respectively, and invoke the

core lemma in Theorem 5.1 to argue that one can "hardcode" the difference between C_{ℓ_0} and C_{ℓ_0+1} into $\widetilde{\mathsf{UGate}}$ and argue indistinguishability.

Let S be the differing sub-circuit between C_{ℓ_0} and C_{ℓ_0+1}, where the two circuits are the circuits defined by the succinct descriptions C'_{ℓ_0} and C'_{ℓ_0+1} respectively. By the definition of δ-equivalence, the sub-circuits C_S^0 and C_S^1 defined by C_{ℓ_0} and C_{ℓ_0+1} respectively are functionally equivalent. We now consider the following sub-hybrids between $\mathcal{H}^t_{m_0,n_0,\ell_0}$ and $\mathcal{H}^t_{m_0,n_0,\ell_0+1}$:

- $\mathcal{H}^t_{m_0,n_0,\ell_0,1}$: This hybrid is identical to $H^t_{m_0,n_0,\ell_0}$ except for the hardcoded puncturable PRF key. The hybrid computes and hardcodes the punctured key $K_R^{m_0}$, where the set R with size $O(|S|)$ consists of input $(n_0, 0, 0)$, all $(n_0, i, 0)$ and $(n_0, 2^\lambda + i, 0)$ for every wire i in the sub-circuit S, and all $(n_0, 0, g)$ for every gate g in the sub-circuit S, and it computes the output values that are punctured away via the master evaluation $y_R \leftarrow \mathsf{pPRF.Eval}(K^{m_0}, R)$. The hybrid then computes the obfuscated universal gate $\widetilde{\mathsf{UGate}}_m \leftarrow \mathsf{iO}(1^\lambda, \mathsf{UGate}'_{[M^0,M^1,n_0,C'_{\ell_0},N_{m_0},(K_R^{m_0},y_R)]})$, where the circuit UGate' is identical, except that it samples puncturable PRF outputs using the punctured key $K_R^{m_0}$ when the input is not in the set R, and uses the hardcoded value y_R when the input is in the set R.

  We immediately have the following claim.

  Claim 5.13. Assuming the polynomial security of iO and the correctness of pPRF, $\mathcal{H}^t_{m_0,n_0,\ell_0}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0,1}$.

- $\mathcal{H}^t_{m_0,n_0,\ell_0,2}$: Same as $\mathcal{H}^t_{m_0,n_0,\ell_0,1}$, but samples the punctured PRF outputs y_R at random.

  Claim 5.14. Assuming the security of pPRF, $\mathcal{H}^t_{m_0,n_0,\ell_0,1}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0,2}$.

  The claim is immediate from the security of puncturable PRF, which guarantees that the punctured outputs y_R are pseudorandom given the punctured key $K_R^{m_0}$.

- $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$: Instead of hardcoding the punctured outputs y_R, the hybrid now hardcodes all the elements computed based on y_R. Before computing the obfuscated universal gate, the hybrid first computes the following elements as in Figure 4:
  - Using randomness $y_{(n_0,0,0)}$, compute the FHE key $(\mathsf{pk}, \mathsf{sk})$, SEH hash key hk, and dummy ciphertext $\mathsf{ct}_{\mathsf{FHE}}$.
  - For every wire i in the sub-circuit S, using randomness $y_{(n_0,i,0)}$, compute the shiftable PRF setup (pp_i, k_i) and constrained signature keys $(\mathsf{vk}_i, \mathsf{sk}_i)$.
  - For every gate g in the sub-circuit S, using randomness $y_{(n_0,0,g)}$, compute the obfuscated gate
    $$\widetilde{\mathsf{Gate}}_g = \mathsf{iO}(1^\lambda, \mathsf{Gate}_{[\langle g\rangle,\mathsf{hk},\{(\mathsf{pp}_i,k_i),\mathsf{vk}_i\}_{i\in\mathsf{in}},\{(\mathsf{pp}_o,k_o),\mathsf{sk}_o\}_{o\in\mathsf{out}},\mathsf{pk},\mathsf{ct}_{\mathsf{FHE}}]}),$$
    where $\langle g\rangle = (\mathsf{in}, \mathsf{out}, f_g) \leftarrow C'_{\ell_0}(n_0, g)$ is the description of gate g in the circuit C_{ℓ_0}.

  The hybrid then collects the relevant elements
  $$\widetilde{S} = \left(\mathsf{pk}, \mathsf{ct}_{\mathsf{FHE}}, \mathsf{hk}, \{(\mathsf{pp}_i, k_i), \mathsf{sk}_i\}_{i\in S.\mathsf{in}}, \{(\mathsf{pp}_o, k_o), \mathsf{vk}_o\}_{o\in S.\mathsf{out}}, \{\widetilde{\mathsf{Gate}}_g\}_{g\in S}\right),$$
  and computes the obfuscated universal gate $\widetilde{\mathsf{UGate}}_m \leftarrow \mathsf{iO}(1^\lambda, \mathsf{UGate}''_{[M^0,M^1,n_0,C'_{\ell_0},N_{m_0},K_R^m,\widetilde{S}]})$, where the circuit UGate'' is identical to UGate', except that when the input length is $n = n_0$, it uses the elements in $\widetilde{S}$ directly instead of computing them using puncturable PRF randomness.

Note that $\widetilde{S}$ covers all the elements that directly depend on the punctured PRF outputs y_R, therefore all outputs of UGate'' are computed identically as in UGate' given $\widetilde{S}$ and K_R^m, leading to the following claim.

Claim 5.15. Assuming the polynomial security of iO, $\mathcal{H}^t_{m_0,n_0,\ell_0,2}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$.

- $\mathcal{H}^t_{m_0,n_0,\ell_0,4}$: Same as $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$, but when computing $\widetilde{S}$, replace the obfuscated gates $\{\widetilde{\mathsf{Gate}}_g\}_{g\in S}$ by obfuscated gate descriptions computed using the circuit C'_{ℓ_0+1} instead of C'_{ℓ_0}.

  Note that the distribution of $\widetilde{S}$ in $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$ and $\mathcal{H}^t_{m_0,n_0,\ell_0,4}$ are exactly the distributions $\mathcal{D}_0$ and $\mathcal{D}_1$ defined in Theorem 5.1 when obfuscating circuit C_{ℓ_0} and C_{ℓ_0+1} respectively, where the two circuits differ on a subcircuit of size $s(n_0) \le O(\log t(\lambda)) = O(\log \lambda)$. By Theorem 5.1, we have the following claim.

  Claim 5.16. Assuming the polynomial security of FHE, SEH, shiftable PRF, constrained signature, and iO, $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0,4}$.

- Finally, note that $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$ and $\mathcal{H}^t_{m_0,n_0,\ell_0,4}$ are symmetric. Therefore, following the same argument of $\mathcal{H}^t_{m_0,n_0,\ell_0}$ being indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0,3}$, we can also claim that $\mathcal{H}^t_{m_0,n_0,\ell_0,4}$ is indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0+1}$.

  Claim 5.17. Assuming polynomial security of iO and the correctness and security of pPRF, $\mathcal{H}^t_{m_0,n_0,\ell_0,4}$ is computationally indistinguishable from $\mathcal{H}^t_{m_0,n_0,\ell_0+1}$.

Combining the indistinguishability between the polynomially many sub-hybrids, we conclude that for every fixed polynomial t, $\mathcal{H}_0^{0,t} \equiv H^t_{0,0,0}$ is computationally indistinguishable from $\mathcal{H}_1^t \equiv H^t_{\lceil\log(t)\rceil,2^{\lceil\log(t)\rceil},\ell(2^{\lceil\log(t)\rceil},N_{\lceil\log(t)\rceil})+1}$, which concludes the proof. □

Combined with the observation from [JJ22] that PV proof of equivalence implies Δ-equivalence for Turing machines (Lemma 3.3), we conclude the main theorem of this section.

Theorem 5.5. *Assuming the polynomial security of iO for circuits, of puncturable PRF, of FHE, of SEH with proof of prefix consistency, of shiftable PRFs, and of time-lock puzzles, there exists a PViO for Turing machines.*

In particular, assuming the polynomial security of iO*, of LWE with sub-exponential modulus-to-noise ratio, and of time-lock puzzles, there exists a PViO for Turing machines.*

6 Replacing iO with EFiO

In this section, we show how our results of PViO (Section 5) from polynomially-secure iO for small circuits can be further reduced to assuming polynomially-secure EFiO for small circuits.

As opposed to iO, EFiO is a falsifiable assumption. While there's currently no known construction for EFiO without relying on iO, there are also no known barriers for constructing it directly from polynomially hard assumptions. Optimistically, this provides a potential path towards constructing PViO without relying on iO. Furthermore, as a standalone result, the construction shows how EFiO for circuits can be boosted to PViO for Turing machines supporting unbounded input length, without the need for general-purpose iO.

At a high level, we obtain a construction of PViO by simply replacing every occurrence ofiO in the previous constructions with EFiO, and instantiating all other building blocks with PV-friendly constructions. Notice that the invocation of iO security in all our previous constructions generally falls in one of the following two categories:

- **Correctness Changes:** The proof invokes iO between two circuits, where an intermediate value is computed in two different ways, and the correctness of the underlying primitive guarantees their equivalence. For instance, given an FHE ciphertext, the evaluate-then-decrypt procedure is equivalent to the decrypt-then-evaluate procedure.
- **Soundness Changes:** The proof invokes iO between two circuits, where in both circuits, some "verification" is performed, and the circuit aborts if the verification fails. Subsequently, some (intermediate) value is computed in two different ways, which are only guaranteed to be functionally equivalent if the verification passes. For instance, given an SEH hash key hk extractable on the index i, a circuit that verifies $\mathsf{Ver}(\mathsf{hk}, H, i, \mathbf{x}_i, \rho_i)$ then computes $f(\mathbf{x}_i)$ is equivalent to the circuit that runs the verification and computes $f(\bar{\mathbf{x}}_i)$, where $\bar{\mathbf{x}}_i \leftarrow \mathsf{Ext}(\mathsf{hk}, \mathsf{td}, H)$.

In both cases, the use of iO can be directly replaced with EFiO if the equivalence of the two circuits has an $\mathcal{EF}$ proof, which is the case if the underlying primitive admits a PV proof of correctness/soundness. For some of the required primitives, such as FHE or SEH, we know constructions with correctness/soundness provable in PV [Mat25, JKLM25]. These constructions all admit *perfect* correctness/soundness, allowing the properties to be formulated as a "for all" statement in PV.

However, we rely on additional primitives, including shiftable PRFs and injective PRGs. Typical constructions give such correctness/soundness guarantees only *up to some negligible probability*. In these cases, proving these guarantees for the construction relies on a *probabilistic counting argument*, to show that for an overwhelming fraction of possible randomness, two circuits are equivalent. To replace iO with EFiO, we would need a PV proof of equivalence for every pair of circuits under "valid" randomness, which does not directly follow from the mathematical proof of the counting argument.

To avoid this issue, we provide the following two solutions.

- For injective PRGs, we replace it with a *injective PRG with a trapdoor*, where given the trapdoor, one can efficiently invert the PRG. Such PRGs can be constructed from LWE through standard lattice trapdoor techniques. If (perfect) correctness of the trapdoor inversion can be proven in PV, then both injectivity and non-membership in the PRG range of any value can also be proven in PV, using the inversion algorithm. Notably, this allows us to argue that, (1) for

every y not in the range of the PRG, $\mathsf{EFiO}(\mathsf{PRG}(\cdot) = y)$ is indistinguishable from $\mathsf{EFiO}(\bot)$, and (2) for every x, y, $\mathsf{EFiO}([x \stackrel{?}{=} y])$ is indistinguishable from $\mathsf{EFiO}([G(x) \stackrel{?}{=} G(y)])$.

- For shiftable PRFs, we provide a new construction satisfying *perfect correctness*. The construction composes the algebraic sPRF construction of [PS18], satisfying approximate correctness in the sense of Definition 3.4, with a somewhere-statistical correlation intractable hash function ([PS19]). Both primitives admit PV proofs for correctness/soundness. This way, we avoid counting arguments, and argue directly that for every possible setup (pp, k, k_f) where k_f is a key shifted by f, $\mathsf{EFiO}(\mathsf{Eval}(\mathsf{pp}, k, x) + f(x))$ is indistinguishable from $\mathsf{EFiO}(\mathsf{Eval}(\mathsf{pp}, k_f, x))$.

6.1 Building Blocks

Here, we summarize the building blocks we need for the construction of PViO from EFiO, and the known constructions with PV provable correctness/soundness. We refer the reader to Appendix B for their formal construction, concrete parameters, and the sketches of the PV proofs.

Theorem 6.1. *Assuming polynomial hardness of LWE (with sub-exponential modulus-to-noise ratio), there exist the following building blocks with* PV*-friendly constructions:*

- *A leveled fully homomorphic encryption scheme, where correctness of evaluation and decryption can be proved in* PV *([GSW13]).*
- *A somewhere extractable hash function with prefix consistency proof, where somewhere extractability and somewhere statistical soundness property can be proved in* PV *([MDS25]).*
- *A shiftable PRF with output space $\mathbb{Z}_q^m$ satisfying approximate correctness with error bound $B = \mathsf{poly}(\lambda, \log q)^{\theta(s)}$ (Definition 3.4), where s is a bound on the size of supported circuits, such that correctness can be proved in* PV *([PS18]).*
- *A statistical correlation intractable hash function for efficiently searchable relations with output size $\mathsf{poly}(\lambda, d)$, where d is the depth bound of the supported circuits, such that correlation intractability can be proved in* PV *([PS19]).*
- *An injective PRG with input space $\mathbb{Z}_q^n$, where injectivity and non-membership of every element outside its domain can be proved in* PV *(Adapted from [BPR12, MP12])*[14].

6.2 Perfectly Correct Shiftable PRF

In this section, we show how to construct a perfectly correct shiftable PRF from an approximately correct shiftable PRF (in the sense of Definition 3.4) and a correlation intractable hash function, with appropriate parameters. In the construction, elements in $\mathbb{Z}_q$ are encoded as integers in $[-q/2, q/2)$, and the modular operations $a \bmod b$ are defined as values in $[-b/2, b/2)$. For elements $t \in [-2^{m'-1}, 2^{m'-1})$, we use the notation $\bar{t}$ to denote the unique binary representation of t in $\{0,1\}^{m'}$.

Construction 6.1 (Perfectly correct shiftable PRF)**.** Let $\mathsf{sPRF} = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{Shift}, \mathsf{EvalShifted})$ be a shiftable PRF with approximate correctness (Definition 3.4) with the following properties:

[14] We note that it was shown in [ABG+25] that the LWE-based injective PRG construction by [GKVW20] admits PV proof of injectivity. However, the non-membership proof is not explicitly addressed. We therefore provide a similar construction in Appendix B.7 with explicit seed extraction to support non-membership proofs in PV.

- The scheme supports the function class $\mathcal{F}_\lambda$ consists of functions $f : \{0,1\}^\ell \to \mathbb{Z}_q^m$ described by circuits with size bounded by some polynomial $s(\lambda)$.
- The output space is $\mathbb{Z}_q^m$ for some modulus q such that $q = p\Delta$, $\Delta/2 \geq 2^{m'(\lambda)}B$, where B is the error bound for sPRF.
- The evaluation circuit Eval has depth $D'(\lambda) = D(\lambda) - \theta(\log\log q)$ and size $S'(\lambda) = S(\lambda) - \theta(\log q)$.

Let $\mathsf{CIH} = (\mathsf{Gen}, \mathsf{SGen}, \mathsf{Hash})$ be a somewhere statistically correlation intractable hash function for the efficiently searchable circuit class $C_{m',D,S}$ of arbitrary input length, output length $m'(\lambda)$, depth at most $D(\lambda)$ and size at most $S(\lambda)$. We construct a new shiftable PRF supporting the function class $\mathcal{F}'_\lambda$ consisting of functions $f : \{0,1\}^\ell \to \mathbb{Z}_p^m$ of size $s'(\lambda) = s(\lambda) - O(\log\Delta)$ as follows:

- $\mathsf{Setup}'(1^\lambda)$: On input the security parameter, run $(\mathsf{pp}, k) \leftarrow \mathsf{sPRF.Setup}(1^\lambda)$. For every $i \in [m]$, run $\mathsf{hk}_i \leftarrow \mathsf{SGen}(1^\lambda, \mathsf{Bd}_{\mathsf{pp},k,i})$, where the boundary circuit $\mathsf{Bd}_{\mathsf{pp},k,i}$ is defined as follows (where we recall that $\bar{t}$ denotes the binary representation of t). Note that $\mathsf{Bd}_{\mathsf{pp},k,i}$ has output length $m'(\lambda)$, depth $D(\lambda)$ and size $S(\lambda)$. Output $\mathsf{pp}' = (\mathsf{pp}, \{\mathsf{hk}_i\}_{i\in[m]})$ and $k' = k$.

  $\underline{\mathsf{Bd}_{\mathsf{pp},k,i}(x)}$

  - Run $y \leftarrow \mathsf{sPRF.Eval}(\mathsf{pp}, k, x)$. Let y_i be the i-th coordinate of y.
  - Compute $t_i = \left\lfloor \frac{((y_i - \Delta/2) \bmod \Delta)}{2B} \right\rceil \in [-\Delta/4B, \Delta/4B)$
  - If $t_i \in [-2^{m'-1}, 2^{m'-1})$, output $\bar{t}_i$; otherwise, output $0^{m'(\lambda)}$.

- $\mathsf{Eval}'(\mathsf{pp}', k', x)$: On input $\mathsf{pp}' = (\mathsf{pp}, \{\mathsf{hk}_i\}_{i\in[m]})$, $k' = k$ and $x \in \{0,1\}^\ell$, evaluate the PRF by $\mathbf{y} \leftarrow \mathsf{sPRF.Eval}(\mathsf{pp}, k, x)$. For every $i \in [m]$, compute $\bar{t}_i = \mathsf{Hash}(\mathsf{hk}_i, x)$, and recover $t_i \in \{0,1\}^{m'}$ as an integer in $[-2^{m'-1}, 2^{m'-1})$. Compute the vector $\mathbf{t} = 2B \cdot (t_1, t_2, \ldots, t_m)^\intercal$, and output $\left\lfloor \frac{\mathbf{y}-\mathbf{t}}{\Delta} \right\rceil \in \mathbb{Z}_p^m$.
- $\mathsf{Shift}'(\mathsf{pp}', k', f)$: On input $\mathsf{pp}' = (\mathsf{pp}, \{\mathsf{hk}_i\}_{i\in[m]})$, $k' = k$ and a circuit $f : \{0,1\}^\ell \to \mathbb{Z}_p^m$ of size at most $s'(\lambda)$, define the circuit $g : x \mapsto \Delta \cdot f(x)$ which has size at most $s(\lambda)$, and run $k_g \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}, k, g)$. Output $k'_f = k_g$.
- $\mathsf{EvalShifted}'(\mathsf{pp}', k'_f, x)$: On input $\mathsf{pp}' = (\mathsf{pp}, \{\mathsf{hk}_i\}_{i\in[m]})$, $k'_f = k_g$ and $x \in \{0,1\}^\ell$, evaluate the PRF: $\mathbf{y}^* \leftarrow \mathsf{sPRF.EvalShifted}(\mathsf{pp}, k_g, x)$. For every $i \in [m]$, compute $\bar{t}_i = \mathsf{Hash}(\mathsf{hk}_i, x)$, and recover $t_i \in \{0,1\}^{m'}$ as an integer in $[-2^{m'-1}, 2^{m'-1})$. Compute the vector $\mathbf{t} = 2B \cdot (t_1, t_2, \ldots, t_m)^\intercal$, and output $\left\lfloor \frac{\mathbf{y}^*-\mathbf{t}}{\Delta} \right\rceil \in \mathbb{Z}_p^m$.

Theorem 6.2 (Correctness of Construction 6.1)**.** *Assuming the approximate correctness of* sPRF *(Definition 3.4) and the correlation intractability of* CIH*, Construction 6.1 satisfies perfect correctness. Furthermore, if the correctness of* sPRF *and the correlation intractability of* CIH *can be proven in* PV*, then the perfect correctness of the new construction can also be proven in* PV*.*

Proof. Fix any setup $\mathsf{pp}' = (\mathsf{pp}, \{\mathsf{hk}_i\}_{i\in[m]})$, any shift $f \in \mathcal{F}'_\lambda$, and any input $\mathbf{x} \in \{0,1\}^\ell$. Define $\mathbf{y} = \mathsf{Eval}'(\mathsf{pp}', k', \mathbf{x})$ and $\mathbf{z} = \mathsf{EvalShifted}'(\mathsf{pp}', k'_f, \mathbf{x})$, and let y_i, z_i be their corresponding i-th coordinates. Let $t_i^* = \left\lfloor \frac{\left((y_i - \lfloor \Delta/2 \rceil) \bmod \Delta\right)}{2B} \right\rceil$. By simple arithmetic, we have that

$$(y_i - 2Bt_i^* \bmod \Delta) \in [-\Delta/2, -\Delta/2 + B) \cup [\Delta/2 - B, \Delta/2).$$

Notice that for all $t \in [-\Delta/4B, \Delta/4B) \setminus \{t_i^*\}$, $2B \le |2Bt_i^* - 2Bt| \le \Delta - 2B$. Combining with the previous equation, we have that for all $t \in [-\Delta/4B, \Delta/4B) \setminus \{t_i^*\}$,

$$(y_i - 2Bt \bmod \Delta) \notin [-\Delta/2, -\Delta/2 + B) \cup [\Delta/2 - B, \Delta/2) \implies (y_i - 2Bt \bmod \Delta) \in [-\Delta/2 + B, \Delta/2 - B).$$

Now, by the correlation intractability of CIH, we know that the hash value $\bar{t}_i = \mathsf{Hash}(\mathsf{hk}_i, \mathbf{x})$ representing the integer $t_i \in [-2^{m'-1}, 2^{m'-1})$ must satisfy $t_i \neq t_i^*$ by the following argument.

- If $t_i^* \notin [-2^{m'-1}, 2^{m'-1})$, then $t_i \neq t_i^*$ simply by our definition of t_i.
- If $t_i^* \in [-2^{m'-1}, 2^{m'-1})$, then the correlation intractability of CIH implies that $\bar{t}_i \neq \mathsf{Bd}_{\mathsf{pp},k,i}(x) = \overline{t_i^*}$. Therefore $t_i \neq t_i^*$.

Therefore, we have $y_i - 2Bt_i \bmod \Delta \in [-\Delta/2 + B, \Delta/2 - B)$.

Finally, by the approximate correctness of sPRF, we know $z_i = y_i + g_i(x) + e_i$ for some error e_i with $|e_i| \le B$. Therefore, the i-th coordinate of the output of $\mathsf{EvalShifted}'(\mathsf{pp}', k'_f, \mathbf{x})$ can be computed by

$$\begin{aligned}\left\lfloor \frac{z_i - 2Bt_i}{\Delta} \right\rceil = \left\lfloor \frac{y_i + g_i(x) + e_i - 2Bt_i}{\Delta} \right\rceil &= \left\lfloor \frac{y_i + \Delta \cdot f_i(x) + e_i - 2Bt_i}{\Delta} \right\rceil \\ &= \left\lfloor \frac{(y_i - 2Bt_i) + e_i}{\Delta} \right\rceil + f_i(x) = \left\lfloor \frac{y_i - 2Bt_i}{\Delta} \right\rceil + f_i(x),\end{aligned}$$

where the last equality follows from the fact that $(y_i - 2Bt_i \bmod \Delta) \in [-\Delta/2 + B, \Delta/2 - B)$ and $|e_i| \le B$. Therefore, we have $\mathsf{EvalShifted}'(\mathsf{pp}', k'_f, \mathbf{x}) = \mathsf{Eval}'(\mathsf{pp}', k', \mathbf{x}) + f(\mathbf{x})$, and the perfect correctness of the new construction follows.

Note that all the above arguments consist of direct invocation of the correctness of sPRF and the correlation intractability of CIH, along with basic arithmetic operations. Therefore, the claim for the PV proof also follows. □

Theorem 6.3 (Shift Hiding of Construction 6.1)**.** *Assuming the (polynomial) shift hiding of* sPRF *and the (polynomial) relation hiding of* CIH*, Construction 6.1 satisfies (polynomial) shift hiding.*

Proof. Consider the following sequence of hybrids:

- Hybrid 0: This is the normal distribution $\left(\mathsf{pp}' = (\mathsf{pp}, \{\mathsf{hk}_i\}_{i\in[m]}), k'_f\right)$ of the shift hiding definition.
- Hybrid 1: Instead of generating each hk_i from the statistical mode generation SGen, sample $\mathsf{hk}_i \leftarrow \mathsf{Gen}(1^\lambda)$ for each $i \in [m]$.

  Claim 6.1. Assuming the relation hiding of CIH, Hybrid 0 is computationally indistinguishable from Hybrid 1.

Note that in Hybrid 1, pp' no longer depends on the master sPRF key k.

- Hybrid 2: Instead of generating the shifted key as $k'_f = k_g \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}, k, g)$, sample $k_g \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}, k, \mathbf{0})$.

  Claim 6.2. Assuming the shift hiding of sPRF, Hybrid 1 is computationally indistinguishable from Hybrid 2.

Since Hybrid 2 is now independent of the shift f, the shift hiding of Construction 6.1 follows. □

By instantiating Construction 6.1 with the approximately correct shiftable PRF for bounded-size circuits from [PS18] and the correlation intractable hash function for efficiently searchable relations from [PS19] given by Theorem 6.1, we obtain the following theorem.

Theorem 6.4. *Assuming the polynomial hardness of LWE (with sub-exponential modulus-to-noise ratio), there exists a shiftable PRF with perfect correctness and shift hiding for bounded-size circuits.*

Note that there is a circular dependency on parameters when combining the two constructions. The CIH construction requires the output length m' to grow polynomially in D, which is the depth of the shiftable PRF evaluation circuit, operating over $\mathbb{Z}_q$ with $q > 2^{m'}$. Luckily, the depth dependency of the evaluation circuit over the modulus is $\mathsf{poly}(\lambda, \log\log q)$, which allows us to choose suitable parameters by picking a sufficiently large modulus q. As discussed in Remark B.3, one could alternatively use the bootstrapped CIH construction from [PS19] to avoid the dependency.

We also remark that the use of correlation intractability in Construction 6.1 is fairly generic, and can be applied to other cases where the correctness error is the result of some rounding error. In particular, one can generically construct a perfectly correct HSS/FSS scheme from an HSS/FSS scheme with approximate correctness (in the sense of Definition 3.4, and with appropriate parameters to support rounding) following the same recipe as in Construction 6.1.

Corollary 6.1. Assuming an FSS scheme over $\mathbb{Z}_q$ satisfying approximate correctness with error bound B, where $q = p\Delta$ and $\Delta = 2^{O(\lambda)}B$, and a somewhere statistically correlation intractable hash function for the corresponding boundary circuit, there exists a FSS scheme over $\mathbb{Z}_p$ satisfying perfect correctness.

Remark 6.1 (Perfectly Correct HSS in the Literature)**.** We note that perfectly correct HSS constructions exist in the literature (e.g., [ACK23]). However, to the best of our knowledge, none of the existing constructions preserve the additive reconstruction of the output shares. Such constructions seem insufficient for our purpose, as our construction of shiftable PRF from HSS(/FSS) in Construction 4.1, and more generally our one-shot programming technique, crucially rely on the property that the HSS evaluation outputs additive shares, that is satisfying $y_0 + y_1 = f(x)$.

6.3 Deterministic Constrained Signature Scheme from EFiO

We plug the tools given by Sections 6.1 and 6.2 into the construction of constrained signature from Section 4.2, to obtain a construction assuming EFiO instead of full-fledged iO.

Theorem 6.5. *Assuming polynomial hardness of LWE (with sub-exponential modulus-to-noise ratio) and a polynomially-secure* EFiO *for circuits, there exists a polynomially-secure deterministic constrained signature scheme for bounded-size circuits.*

Proof of Theorem 6.5. We claim that Construction 4.2 remains a deterministic constrained signature scheme when instantiated as follows:

- The iO is instantiated with an EFiO.
- The shiftable PRF is instantiated with the perfectly correct one from Construction 6.1.
- The injective PRG is instantiated with the injective PRG with trapdoor from Theorem 6.1.

It suffices to verify that, for every hybrid in the security proof of Theorem 4.2 where iO security is invoked, the two circuits can be proven equivalent in PV using the properties of the building blocks. We verify all the invocations within the proof of Theorem 4.2 as follows:

- Between Hybrid 0 and 1 (Claim 4.4), Hybrid 2 and 3 (Claim 4.6), and Hybrid 3 and 4 (Claim 4.7), the equivalence of the two circuits follows from the *perfect correctness* of sPRF.
- Between Hybrid 4 and 5 (Claim 4.8), the equivalence of the two circuits follows from the injectivity of PRG.
- Between Hybrid 6 and 7 (Claim 4.10), the check within the circuit is replaced as follows

$$f(x) = 1 \wedge \mathsf{PRG}(\ldots) = T_f(x) \implies \mathsf{PRG}(\ldots) = T_f(x)$$

  where the right hand side can be written as $(f(x) = 1 \wedge \mathsf{PRG}(\ldots) = T_1) \vee (f(x) = 0 \wedge \mathsf{PRG}(\ldots) = T_0)$. From the PV proof of non-membership of PRG, for every $T_0 \notin \mathsf{Im}(\mathsf{PRG})$, there exists a PV proof that the check $\mathsf{PRG}(\ldots) = T_0$ is equivalent to an all-rejection check. Therefore, with overwhelming probability over the choice of T_0, we have $\mathsf{EFiO}(V'_{\mathsf{pp},k,1,T_0,T_1}) \approx \mathsf{EFiO}(V''_{\mathsf{pp},k,1,T_0,T_1})$, which immediately implies the indistinguishability between Hybrid 6 and 7.

□

6.4 PViO from EFiO

We now show how to reduce the EFiO and PViO construction from Section 5 to only rely on EFiO for small circuits instead of general-purpose iO. Similar to the constrained signature case, we can simply replace every iO invocation with EFiO, and instantiate the building blocks (FHE, SEH, shiftable PRF and constrained signature) with those from Theorem 6.1.

One subtle issue is that the construction of constrained signature in Theorem 6.5 already relies on EFiO. In other words, there are two layers of EFiO: an "inner" layer to build the constrained signature, and an "outer" layer from the construction of Section 5. As is, the correctness and soundness of the constrained signature scheme also rely on the correctness of the inner EFiO. To avoid the need of assuming EFiO with PV-provable correctness, we rely on the observation in Remark 4.5 that we can "strip off" the inner EFiO from the constrained, whenever signing/verification keys are hardcoded inside another circuit obfuscated with EFiO – here the "outer" layer. Correctness of the resulting "constrained signature" then only relies on shiftable PRF correctness, without relying on the correctness of any "inner" EFiO. Summing up, in the resulting construction we now use the FSS/shiftable PRF directly, without the "inner" EFiO from Definition 4.1. The security guarantee of the constrained signature still holds, with the security proof relying instead on the "outer" layer of EFiO obfuscation.

Theorem 6.6. *There exists a fixed polynomial $p(\lambda)$, such that assuming polynomial hardness of LWE (with sub-exponential modulus-to-noise ratio) and a polynomially-secure* EFiO *for circuits of size at most $p(\lambda)$, there exists a polynomially-secure* EFiO *for all polynomial-size circuits.*

Proof. The construction is obtained by instantiating the EFiO scheme in Construction 5.1 with the following building blocks:

- The iO is instantiated with an EFiO.
- The FHE and SEH are instantiated using Theorem 6.1.
- The perfectly correct shiftable PRF is instantiated using Construction 6.1.
- The deterministic constrained signature scheme is defined as in Construction 4.2, with building blocks instantiated using Theorem 6.5. However, whenever the keys $\mathsf{sk}_f = \mathsf{EFiO}(E_{\mathsf{pp},k,f})$ or $\mathsf{vk}_f = \mathsf{EFiO}(V_{\mathsf{pp},k,f})$ are hardcoded inside the circuit Gate, hardcode the plain circuit $E_{\mathsf{pp},k,f}$ or $V_{\mathsf{pp},k,f}$ instead.

Similar to the proof of Theorem 6.5, every hybrid in the security proof of Theorem 5.1 that invokes iO security, along with the correctness/soundness of FHE and SEH, can be argued with EFiO security using the PV proofs of the instantiations. Therefore, it remains to consider the hybrids involving the constrained signature scheme. The core observation enabling the argument is that, in the proof of Theorem 5.1, every hybrid that involves changing the constrained signature keys is changing keys hardcoded inside obfuscated circuits.

We verify that all the hybrids involving the constrained signature scheme are still indistinguishable after the removal of the "inner" EFiO layer as follows:

- Between $\mathcal{H}^b_{3,t,0}$ and $\mathcal{H}^b_{3,t,1}$ (Claim 5.4), the equivalence of the two circuits follows from the deterministic signing property of the constrained signature scheme. This still holds without the "inner" EFiO layer, and can be trivially proven in PV since, by construction, the signature output by circuit $E_{\mathsf{pp},k,f}$ is a PRF evaluation independent of f.
- Between $\mathcal{H}^b_{3,t,1}$ and $\mathcal{H}^b_{3,t,2}$ (Claim 5.5), we invoke the constraint hiding property of the constrained signature scheme. Note that it is invoked on an *internal wire* of S, i.e., both the signing key sk_o and the corresponding verification key vk_o are hardcoded inside the obfuscated Gate circuit. Therefore, the indistinguishability follows by expanding the constraint hiding proof of the constrained signature scheme, where we use the EFiO over Gate (the "outer" layer) to switch between hybrids relying on EFiO security.
- The input-soundness property of the internal wires (Lemma 5.1) follows from the (perfect) soundness of the constrained signature scheme, which still holds without the "inner" EFiO layer, as the verifying circuit $V_{\mathsf{pp},k,f}$ by definition rejects all x such that $f(x) = 0$.

□

The same argument also applies to the PViO construction in Construction 5.2. While the construction additionally requires a (relaxed) time-lock puzzle and a puncturable PRF, it is clear from inspection that:

- The time-lock puzzle generation/evaluation is never obfuscated, and as a result, we do not rely on time-lock puzzle correctness when invoking iO security.

- Puncturable PRFs can be trivially constructed from shiftable PRFs. In particular, puncturing the key at points $S = \{x_i\}$ can be mimicked by shifting the key with a function f_S, where $f_S(x) = 0$ when $x \notin S$, and $f_S(x) = y_i$ if $x = x_i$ for some random, hardcoded string y_i. As the perfect correctness of the shiftable PRF is provable in PV, so is the perfect correctness of the above puncturable PRF.

This leads to the final main result of this section.

Theorem 6.7. *Assuming polynomial hardness of LWE (with sub-exponential modulus-to-noise ratio), a polynomially hard relaxed time-lock puzzle, and a polynomially-secure* EFiO *for circuits, there exists a polynomially-secure* PViO *for efficient Turing machines.*

Proof. The construction is obtained by instantiating the PViO scheme in Construction 5.2 with the following building blocks:

- The iO for circuits is instantiated with an EFiO.
- The FHE and SEH are instantiated using Theorem 6.1.
- The perfectly correct shiftable PRF is instantiated using Construction 6.1, and so is the puncturable PRF, derived from the shiftable PRF as described above.
- The deterministic constrained signature scheme is defined as in Construction 4.2, with building blocks instantiated using Theorem 6.5. However, whenever the key $\mathsf{sk}_f = \mathsf{EFiO}(E_{\mathsf{pp},k,f})$ or $\mathsf{vk}_f = \mathsf{EFiO}(V_{\mathsf{pp},k,f})$ is hardcoded inside the circuit Gate, hardcode the plain circuit $E_{\mathsf{pp},k,f}$ or $V_{\mathsf{pp},k,f}$ instead.

The security proof follows the exact same argument as in the proof of Theorem 6.6. □

7 Other Applications

7.1 Fully Homomorphic Encryption

In this section, we prove Theorem 1.3, namely, how to build fully homomorphic encryption based on the polynomial security of iO, of polynomially-secure shiftable PRFs, of (relaxed) time-lock puzzles, and of a rerandomizable encryption with additional algebraic properties, which we call algebraically rerandomizable encryption (Definition 7.1).

Very similarly to [CLTV15], we proceed in two steps. In Theorem 7.1, we first build leveled homomorphic encryption, crucially using shiftable PRFs and a structured form of rerandomizable encryption. We then extend our construction to an unleveled FHE in Theorem 7.4.

Definition 7.1 (Algebraically-Rerandomizable Encryption)**.** *Let* $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *be a public key encryption scheme, where* Enc *takes randomness from a ring* R.[15]

We say that $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *is algebraically rerandomizable if there exists a PPT algorithm:*

- $\mathsf{Rerand}(\mathsf{pk}, \mathsf{ct}; r) \to \mathsf{ct}'$

with the following property: for all messages m *and encryption randomness* $\rho \in R$*, letting* $\mathsf{ct} = \mathsf{Enc}_{\mathsf{pk}}(m; \rho)$*:*

$$\mathsf{Rerand}(\mathsf{pk}, \mathsf{ct}; r) = \mathsf{Enc}_{\mathsf{pk}}(m; r + \rho)$$

where $+$ *refers to the addition operation over* R*, the randomness space for* Enc*.*

Remark 7.1 (Instantiations of Algebraically-Rerandomizable Encryption)**.** We observe that (polynomially-secure) algebraically-rerandomizable encryption exists assuming (polynomial hardness of) DDH: the ElGamal encryption scheme satisfies the property above. Indeed, denoting the group generator by $g \in \mathbb{G}$, the public key by $\mathsf{pk} = h \in \mathbb{G}$ and encryption

$$\mathsf{Enc}(h, m; \rho) = (g^\rho, h^\rho \cdot m),$$

where $m \in \{0, 1\}$, $\rho \in \mathbb{Z}_q$ where q is the (not necessarily prime) order of the group, one can define:

- $\mathsf{Rerand}(\mathsf{pk}, \mathsf{ct}; r)$: Parse $\mathsf{ct} = (g_1, h_1)$. Output:

$$\mathsf{ct}' = (g_1 \cdot g^r,\ h_1 \cdot h^r).$$

We summarize this in the following claim.

Claim 7.1. Assume the polynomial hardness of DDH over a group of order q. Then there exists a (polynomially-secure) algebraically-rerandomizable encryption with randomness space $\mathbb{Z}_q$.

More generally, homomorphic encryption schemes satisfying an appropriate notion of randomness homomorphism, where the randomness of homomorphically summed ciphertexts also sums up, directly give an algebraically-rerandomizable encryption. ElGamal is simply a special case of this phenomenon.

[15] The assumption that the randomness space is a ring R is mostly for notational convenience, in order to match the definition of shiftable PRFs Definition 3.3. Throughout this section, it would be sufficient to have the randomness space be a group.

Leveled Fully Homomorphic Encryption. We start by focusing on building *leveled* FHE, which supports homomorphism up to some depth upper bound d.

Theorem 7.1. *Assuming polynomially-secure* iO*, polynomially-secure shiftable PRF with range R, and polynomially-secure algebraically-rerandomizable encryption with randomness space R, there exists a leveled homomorphic encryption scheme.*

Construction 7.1. Let iO be an iO scheme, $\mathsf{sPRF} = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{Shift}, \mathsf{EvalShifted})$ be a shiftable PRF with output space R for a function class $\mathcal{F}$ to be determined later, and $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Rerand})$ be an algebraically-rerandomizable encryption scheme with randomness space R. We construct our leveled FHE scheme $(\mathsf{FHE.KeyGen}, \mathsf{FHE.Enc}, \mathsf{FHE.Dec}, \mathsf{FHE.Eval})$ as follows:

- $\mathsf{FHE.KeyGen}(1^\lambda, 1^d)$: sample, for every $0 \le i \le d$:

  $$(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda), \quad \mathsf{ct}_{i,0} \leftarrow \mathsf{Enc}(\mathsf{pk}_i, 0), \quad \mathsf{ct}_{i,1} \leftarrow \mathsf{Enc}(\mathsf{pk}_i, 1)$$

  and

  $$(\mathsf{pp}_i, K_i) \leftarrow \mathsf{sPRF.Setup}(1^\lambda).^{16}$$

  For every $0 \le i \le d-1$, let C_i be the following circuit:

  > Circuit C_i
  >
  > Hard-coded: $\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,0}, \mathsf{ct}_{i+1,1}, \mathsf{sk}_i, \mathsf{pp}_i, K_i$
  > On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, compute $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_i}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_i}(\mathsf{ct}^1))$ and output:
  >
  > $$\mathsf{ct} = \mathsf{Rerand}(\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,b_x}\,;\, \mathsf{sPRF.Eval}(K_i, x)).$$
  >
  > Otherwise, if $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_i}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_i}(\mathsf{ct}^1))$ is not well-defined (e.g. if any call to $\mathsf{Dec}_{\mathsf{sk}_i}$ outputs $\perp$), output:
  >
  > $$\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}_{i+1}, 0\,;\, \mathsf{sPRF.Eval}(K_i, x)).$$

  Output:

  $$\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_i)\}_{0 \le i \le d-1}), \quad \mathsf{FHE.sk} = \mathsf{sk}_d.$$

- $\mathsf{FHE.Enc}_{\mathsf{pk}_0}(b)$: Output $\mathsf{ct} \leftarrow \mathsf{Enc}_{\mathsf{pk}_0}(b)$.

- $\mathsf{FHE.Dec}_{\mathsf{sk}_d}(\mathsf{ct})$: Output $\mathsf{Dec}_{\mathsf{sk}_d}(\mathsf{ct})$.

- $\mathsf{FHE.Eval}(C, \{\mathsf{ct}_j\}_{j \in [\ell]})$: Without loss of generality, assume that C is a layered circuit of NAND gates with ℓ-bit input, where the set of all wires in C can be partitioned in sets $L_0, \ldots L_{d-1}$ such that every gate of C has its input wires in L_i and output wires in L_{i+1}, for some $0 \le i \le d-1$.[17]

  To evaluate, view $\{\mathsf{ct}_j\}$ as input wires, and inductively evaluate C layer-by-layer using $\mathsf{iO}(C_i)$ for the NAND gates connecting the ith layer to the $i+1$st layer, which defines the wire values for layer $i+1$.

[16] When clear from the context, we will omit, for notational clarity, the public parameters pp_i as input from the shiftable PRF algorithms.

[17] If not, pad the circuit by adding "identity gates" as needed, which induces at most a quadratic overhead over the size of C.

Theorem 7.2. *Assuming that* (KeyGen, Enc, Dec, Rerand) *and* iO *satisfy (statistical) correctness, and that* (KeyGen, Enc, Dec, Rerand) *is algebraically rerandomizable (Definition 7.1), then Construction 7.1 is correct.*

Proof. First, observe that the claim would follow immediately if (KeyGen, Enc, Dec, Rerand) satisfied *perfect correctness* instead. We extend the claim to statistically correct (KeyGen, Enc, Dec, Rerand) by observing that, by algebraic rerandomization, the marginal distribution of ciphertexts produced by C_i when b_x is well-defined is identically distributed as fresh ciphertexts, *over the random choice of* $\mathsf{ct}_{i,b_x} \leftarrow \mathsf{Enc}(\mathsf{pk}_i, b_x)$ *alone*. The claim then follows by the statistical correctness of the encryption. □

Theorem 7.3. *Assuming that* iO *is polynomially-secure (Definition 3.7),* sPRF = (Setup, Eval, Shift, EvalShifted) *satisfies (polynomially-secure) shift-hiding (Definition 3.3), and that* (KeyGen, Enc, Dec, Rerand) *is a polynomially-secure algebraically-rerandomizable encryption scheme (Definition 7.1), then Construction 7.1 is secure.*

Proof. We prove the claim above using a sequence of hybrids.

- $\mathcal{H}_0^b$: This is the view of the adversary in the security experiment for the FHE, which includes the public key $\mathsf{pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_i)\}_{0 \le i \le d-1})$ and a ciphertext $\mathsf{ct}_b \leftarrow \mathsf{Enc}_{\mathsf{pk}_0}(b)$ of the bit b.
- $\mathcal{H}_i^b$, where $1 \le i \le d$: Let $j = d - i$. We change the way pk is generated, and more specifically the way the circuits C_k, $k \ge j+1$ are defined (intuitively, we switch them from the last level to the first).
  - For all $j \le k \le d$, sample:

    $$(\mathsf{pk}_k, \mathsf{sk}_k) \leftarrow \mathsf{KeyGen}(1^\lambda), \quad \mathsf{ct}_{k,0} \leftarrow \mathsf{Enc}(\mathsf{pk}_k, 0), \quad \mathsf{ct}_{k,1} \leftarrow \mathsf{Enc}(\mathsf{pk}_k, 0)$$

    and

    $$K_k \leftarrow \mathsf{sPRF.Setup}(1^\lambda).$$

    If $k \le d-1$, consider the following circuit:

    Circuit $\widetilde{C}_k$

    Hard-coded: pk_{k+1}, K_k
    On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, output:

    $$\mathsf{ct} = \mathsf{Enc}_{\mathsf{pk}_{k+1}}(0\,;\, \mathsf{sPRF.Eval}(K_k, x)).$$

  - For $k < j$, compute C_k as in the construction.

  The public key of the FHE is now defined as:

  $$\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_0), \ldots, \mathsf{iO}(C_{j-1}), \mathsf{iO}(\widetilde{C}_j), \cdots, \mathsf{iO}(\widetilde{C}_{d-1})\}).$$

  In other words, whenever $k \ge j$, instead of computing b_x and rerandomizing ciphertext ct_{k+1,b_x}, circuit $\widetilde{C}_k$ always outputs an encryption of 0 under pk_{k+1}.

  To argue that two consecutive $\mathcal{H}_{i-1}$ and $\mathcal{H}_i$ are computationally indistinguishable for $1 \le i \le d-1$, we consider the following sub-hybrids:

- $\mathcal{H}^b_{i,1}$: We change the way $\mathsf{FHE.pk}$ is computed from $\mathcal{H}^b_{i-1}$. We now sample:

$$\mathsf{ct}_{j+1,0} \leftarrow \mathsf{Enc}(\mathsf{pk}_{j+1}, 0), \quad \widetilde{\mathsf{ct}}_{j+1,1} \leftarrow \mathsf{Enc}(\mathsf{pk}_{j+1}, 0),$$

where we recall that $j = d - i \leq d - 1$, and set accordingly

> Circuit $C_{j,1}$
>
> Hard-coded: $\mathsf{pk}_{j+1}, \mathsf{ct}_{j+1,0}, \widetilde{\mathsf{ct}}_{j+1,1}, \mathsf{sk}_j, K_j$
> On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, compute $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^1))$ and output:
>
> $$\mathsf{ct} = \mathsf{Rerand}(\mathsf{pk}_{j+1}, \mathsf{ct}_{j+1,b_x}\,;\, \mathsf{sPRF.Eval}(K_j, x)).$$
>
> Otherwise, ...

where we abuse notation and define $\mathsf{ct}_{j+1,b_x} = \mathsf{ct}_{j+1,0}$ if $b_x = 0$ and $\mathsf{ct}_{j+1,b_x} = \widetilde{\mathsf{ct}}_{j+1,1}$ if $b_x = 1$. Set:

$$\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_0), \ldots, \mathsf{iO}(C_{j-1}), \mathsf{iO}(C_{j,1}), \cdots, \mathsf{iO}(\widetilde{C}_{d-1})\}).$$

Claim 7.2. Assuming $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Rerand})$ is polynomially secure, then for all $1 \leq i \leq d$, $\mathcal{H}^b_{i-1}$ and $\mathcal{H}^b_{i,1}$ are computationally indistinguishable.

Observe that in both hybrids, the secret key sk_{j+1} is not needed to compute $\mathsf{FHE.pk}$ (nor anything else in the views of the adversary). Thus, indistinguishability of $\mathcal{H}^b_i$ and $\mathcal{H}^b_{i,1}$ follows by semantic security of the algebraically-rerandomizable encryption $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Rerand})$ via a direct reduction.

- $\mathcal{H}^b_{i,2}$: We change the way C_j is computed. We now sample a shifted-by-zero PRF key:

$$K_{j,2} \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}, K_j, \mathbf{0})$$

where $\mathbf{0}$ denotes the all 0 circuit. We now define:

> Circuit $C_{j,2}$
>
> Hard-coded: $\mathsf{pk}_{j+1}, \mathsf{ct}_{j+1,0}, \mathsf{ct}_{j+1,1}, \mathsf{sk}_j, K_{j,2}$
> On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, compute $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^1))$ and output:
>
> $$\mathsf{ct} = \mathsf{Rerand}(\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,b_x}\,;\, \mathsf{sPRF.EvalShifted}(K_{j,2}, x)).$$
>
> Otherwise, ...

and then set

$$\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_0), \ldots, \mathsf{iO}(C_{j-1}), \mathsf{iO}(C_{j,2}), \cdots, \mathsf{iO}(\widetilde{C}_{d-1})\}).$$

Claim 7.3. Assuming $\mathsf{sPRF} = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{Shift}, \mathsf{EvalShifted})$ is correct (Definition 3.3), and iO is polynomially secure (Definition 3.7), then for all $1 \leq i \leq d$, the views of the adversary in $\mathcal{H}^b_{i,1}$ and $\mathcal{H}^b_{i,2}$ are computationally indistinguishable.

By correctness of the shiftable PRF, the circuits $C_{j,1}$ and $C_{j,2}$ are functionally equivalent, and therefore distributions induced in the two hybrids are computationally indistinguishable, by a direct reduction to the (polynomial) security of iO.

- $\mathcal{H}^b_{i,3}$: We change again how $\mathsf{FHE.pk}$ is defined. We sample as before:

$$\mathsf{ct}_{j+1,0} \leftarrow \mathsf{Enc}(\mathsf{pk}_{j+1}, 0; \rho_{j+1,0}), \quad \mathsf{ct}_{j+1,1} \leftarrow \mathsf{Enc}(\mathsf{pk}_{j+1}, 0, \rho_{j+1,1}),$$

where $\rho_{j+1,0}, \rho_{j+1,1} \leftarrow R$ denote the randomness used to produce these ciphertexts.
Consider the following circuit:

> Circuit S_j
>
> Hard-coded: $\mathsf{sk}_j, \rho_{j+1,0}, \rho_{j+1,1}$
> On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, compute $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^1))$, and output $-\rho_{j+1,b_x}$, where $-$ denotes subtraction over R.
> Otherwise, if $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^1))$ is not well-defined (e.g. if any call to $\mathsf{Dec}_{\mathsf{sk}_j}$ outputs $\perp$), output 0.

We now compute instead:

$$K_{j,3} \leftarrow \mathsf{sPRF.Shift}(\mathsf{pp}, K_j, S_j),$$

and define

> Circuit $C_{j,3}$
>
> Hard-coded: $\mathsf{pk}_{j+1}, \mathsf{ct}_{j,0}, \mathsf{ct}_{j,1}, \mathsf{sk}_j, K_{j,3}$
> On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, compute $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^1))$ and output:
>
> $$\mathsf{ct} = \mathsf{Rerand}(\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,b}\,;\, \mathsf{sPRF.EvalShifted}(K_{j,3}, x)).$$
>
> Otherwise, ...

and then set

$$\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_0), \ldots, \mathsf{iO}(C_{j-1}), \mathsf{iO}(C_{j,3}), \cdots, \mathsf{iO}(\widetilde{C}_{d-1})\}).$$

Claim 7.4. Assuming $\mathsf{sPRF} = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{Shift}, \mathsf{EvalShifted})$ satisfies shift-hiding for a function class $\mathcal{F}$ that includes S_j (Definition 3.3), then for all $1 \le i \le d$, the views of the adversary in $\mathcal{H}^b_{i,2}$ and $\mathcal{H}^b_{i,3}$ are computationally indistinguishable.

This follows by a direct reduction to shift-hiding security (noting that the master key K_j is used in neither $\mathcal{H}_{i,2}$ nor $\mathcal{H}_{i,3}$, beyond computing the shifted keys $K_{j,2}, K_{j,3}$).

- $\mathcal{H}^b_i$: For redundancy, this corresponds to changing how C_j is computed. We now define

$$K_j \leftarrow \mathsf{Setup}(1^\lambda),$$

and set:

> Circuit $\widetilde{C}_j$
>
> Hard-coded: pk_{j+1}, K_j
> On input $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$, output:
>
> $$\mathsf{ct} = \mathsf{Enc}_{\mathsf{pk}_{j+1}}(0\,;\, \mathsf{sPRF.Eval}(K_j, x)).$$

and set

$$\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{iO}(C_0), \ldots, \mathsf{iO}(C_{j-1}), \mathsf{iO}(\widetilde{C}_j), \cdots, \mathsf{iO}(\widetilde{C}_{d-1})\}).$$

Claim 7.5. Assuming sPRF = (Setup, Eval, Shift, EvalShifted) is correct (Definition 3.3), (KeyGen, Enc, Dec, Rerand) is algebraically-rerandomizable (Definition 7.1) and iO is polynomially secure (Definition 3.7), then for all $1 \le i \le d$, $\mathcal{H}^b_{i,3}$ and $\mathcal{H}^b_i$ are computationally indistinguishable.

Recall that, in these hybrids:

$$\mathsf{ct}_{j+1,0} \leftarrow \mathsf{Enc}(\mathsf{pk}_{j+1}, 0; \rho_{j+1,0}), \quad \mathsf{ct}_{j+1,1} \leftarrow \mathsf{Enc}(\mathsf{pk}_{j+1}, 0; \rho_{j+1,1}).$$

By algebraic rerandomizability of (KeyGen, Enc, Dec, Rerand), we have for all $r \in R$, and $b \in \{0,1\}$:

$$\mathsf{Rerand}(\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,b}\,;\, r) = \mathsf{Enc}(\mathsf{pk}_{j+1}, 0\,;\, \rho_{j+1,b} + r). \tag{3}$$

Fix now an arbitrary $x = (\mathsf{ct}^0 \| \mathsf{ct}^1)$. We show that $\widetilde{C}_j(x) = C_{j,3}(x)$.

Let $b_x = \mathrm{NAND}(\mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^0), \mathsf{Dec}_{\mathsf{sk}_j}(\mathsf{ct}^1))$. If b_x is not well-defined, then by definition $\widetilde{C}_j(x) = C_{j,3}(x)$. So assume that b_x is well-defined.

Taking Equation (3) with $r = \mathsf{sPRF.EvalShifted}(K_{j,3}, x)$ gives:

$$\begin{aligned}
&\mathsf{Rerand}(\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,b}\,;\mathsf{sPRF.EvalShifted}(K_{j,3}, x)) \\
&\qquad = \mathsf{Enc}(\mathsf{pk}_{j+1}, 0\,;\, \rho_{j+1,b_x} + \mathsf{sPRF.EvalShifted}(K_{j,3}, x)) \\
&\qquad = \mathsf{Enc}(\mathsf{pk}_{j+1}, 0\,;\, \rho_{j+1,b_x} + \mathsf{sPRF.Eval}(K_j, x) + S_j(x)) \\
&\qquad = \mathsf{Enc}(\mathsf{pk}_{j+1}, 0\,;\, \mathsf{sPRF.Eval}(K_j, x))
\end{aligned}$$

where the second equality follows by shifted correctness, and the third equality using that $S_j(x) = -\rho_{j+1,b_x}$.

Overall, for all x, we have $\widetilde{C}_j(x) = C_{j,3}(x)$, and indistinguishability of the views of $\mathcal{H}_{i,3}$ and $\mathcal{H}_i$ then follows by security of iO.

Combining the above, we concluded that $\mathcal{H}^b_0$ is indistinguishable from $\mathcal{H}^b_d$. It remains to argue that $\mathcal{H}^b_d$ hides the bit b.

Claim 7.6. Assuming (KeyGen, Enc, Dec, Rerand) is semantically secure, $\mathcal{H}^0_d$ and $\mathcal{H}^1_d$ are computationally indistinguishable.

This follows by a direct reduction, by observing that in $\mathcal{H}^b_d$ no part of the public key FHE.pk depends on sk_0.

□

We proceed with a couple of comments.

Remark 7.2 (On the Complexity of the Shifts Required)**.** In our proof of security, we require our shiftable PRF to support shifts defined by the circuit S_j, whose complexity is dominated by the decryption of the algebraically rerandomizable encryption. In particular, the more efficient decryption for the algebraically rerandomizable encryption is, the weaker class of shift is needed in our proof. For concreteness, in the example of ElGamal over an order q subgroup of $\mathbb{Z}^*_p$ or suitable elliptic curves, this decryption can be computed in NC^1, and thus shiftable PRFs for NC^1 would then be sufficient. We then instantiate the FSS with prime order message space $\mathbb{Z}_q$ (e.g. [ADOS22, CL15] from class groups) where q is the order of the ElGamal group. Alternatively, if q is a strong RSA number we can use a DCR-based FSS [OSY21, RS21]. We summarize this in the following corollary.

Corollary 7.1. Assuming the polynomial security of iO, of DDH over a group $\mathbb{G}$ of order q that is either a subgroup of $\mathbb{Z}_p^*$ or a suitable elliptic curve, and of a strongly correct FSS for NC^1 shifts with range $\mathbb{Z}_q$, there exists a leveled FHE scheme.

In particular, one can replace the FSS assumption above with the polynomial hardness of either DDH over appropriate class groups [ADOS22, CL15] for prime-order q, DCR for q that is a strong RSA number, or of LWE (with sub-exponential modulus-to-noise ratio) [BKS19].

Remark 7.3 (On the Elementary Gates Supported by the Construction)**.** As written, our construction only supports homomorphic NAND gates, which suffices for feasibility because of their universality. We note that we only did so out of notational convenience: the proof of security directly extends to support many different gates at each level. The only repercussions of considering potentially powerful sets of gates would be both the size of the resulting public key (where the obfuscation would grow with a description of the gate), and the shiftable PRF which would need to support such a gate evaluation (composed with decryption).

Unleveled Fully Homomorphic Encryption. Next, we turn to building fully homomorphic encryption. Our compiler is very similar to the one in [CLTV15]: we define *implicitly* an exponential number of circuits C_i for pseudorandomly generated keys $\mathsf{pk}_i, \mathsf{sk}_i$. This implicit representation is given out by obfuscating programs, which, on input i, outputs C_i. We do so by generating all the necessary hard-coded information to C_i pseudorandomly, using a (punctured) PRF, and also obfuscating C_i using more pseudorandom coins.

The main difference is that we encrypt these obfuscated programs under a time-lock puzzle, so that, for any efficient adversary running in time T, we only need to argue security of at most $\log T$ different programs. Details follow.

Theorem 7.4. *Assuming the existence of polynomially-secure* iO*, polynomially-secure shiftable PRF, polynomially-secure algebraically-rerandomizable encryption, and polynomially-secure (relaxed) time-lock puzzles, there exists a unleveled, fully homomorphic encryption scheme.*

We only make explicit the main differences with Construction 7.1, and only provide proof sketches: the arguments are very similar to [CLTV15] and Section 5.2.

The main difference is how the evaluation key is generated.

- $\mathsf{FHE.KeyGen}(1^\lambda)$: Sample $(\mathsf{pk}_0, \mathsf{sk}_0) \leftarrow \mathsf{KeyGen}(1^\lambda)$, and punctured PRF keys K, K'. Consider the following circuit:

Circuit MProg_m

Hard-coded: sk_0, K, K', m
On input an integer i:

- If $i \geq 2^{m+1}$, output $\perp$.
- Otherwise, compute $r_{i-1} = \mathsf{PRF.Eval}(K, i-1)$, $r_i = \mathsf{PRF.Eval}(K, i)$, and $r'_i = \mathsf{PRF.Eval}(K', i)$.

  Parse $r_{i-1} = (r^0_{i-1} \| r^1_{i-1} \| r^2_{i-1} \| r^3_{i-1} \| r^4_{i-1})$ and $r_i = (r^0_i \| r^1_i \| r^2_i \| r^3_i \| r^4_i)$.

  Compute $(\mathsf{pk}_{i-1}, \mathsf{sk}_{i-1}) = \mathsf{KeyGen}(1^\lambda; r^0_{i-1})$, and compute

$$(\mathsf{pk}_i, \mathsf{sk}_i) = \mathsf{KeyGen}(1^\lambda; r^0_i), \quad \mathsf{ct}_{i,0} = \mathsf{Enc}(\mathsf{pk}_i, 0; r^1_i), \quad \mathsf{ct}_{i,1} = \mathsf{Enc}(\mathsf{pk}_i, 1; r^2_i)$$

  and

$$(\mathsf{pp}_i, K_i) = \mathsf{PRF.Setup}(1^\lambda; r^3_i).$$

  Output $\mathsf{iO}(C_i[\mathsf{pk}_{i+1}, \mathsf{ct}_{i+1,0}, \mathsf{ct}_{i+1,1}, \mathsf{sk}_i, \mathsf{pp}_i, K_i]; r'_i)$, where C_i is the circuit defined in Construction 7.1.

Compute the obfuscations

$$\widetilde{\mathsf{MProg}}_m = \mathsf{iO}(1^\lambda, \mathsf{MProg}_m),$$

and encrypt them under a time-lock puzzle with time-parameter 2^m:

$$\widetilde{\mathsf{ct}}_m = \mathsf{TLP.Gen}(1^\lambda, 2^m, \widetilde{\mathsf{MProg}}_m).$$

Output: $\mathsf{FHE.pk} = (\mathsf{pk}_0, \{\mathsf{ct}_m\}_{m \leq \lambda})$, $\mathsf{FHE.sk} = (\mathsf{sk}_0, K)$.

Decryption proceeds by recovering the secret key at some level i using $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda; r^0_i)$.

Evaluation for a computation running in time T proceeds by solving the time-lock puzzle $\mathsf{ct}_{\lceil \log T \rceil}$, and retrieving the appropriate circuits C_i from $\mathsf{MProg}_{\lceil \log T \rceil}$.

We sketch the security proof. Suppose an adversary runs in time T.

- We first rely on time-lock-puzzle security to argue that all the $\{\widetilde{\mathsf{ct}}_m\}$, $\lfloor \log T + 1 \rfloor \leq m \leq \lambda$ are hidden. This relies on at most λ applications of TLP security.
- We change every output of all the MProg_m, $m \leq \lfloor \log T + 1 \rfloor$ to output $\widetilde{C}_i$ instead, where $\widetilde{C}_i$ is defined in the proof of Theorem 7.3. We do so input by input (every circuit MProg_m has an input space of size at most T), by puncturing the PRFs K, K', and applying the proof of Theorem 7.3 to switch from outputting $\mathsf{iO}(C_i)$ to outputting $\mathsf{iO}(\widetilde{C}_i)$. This relies on $O(T\lambda)$ invocations of punctured PRF security, iO, and the proof of Theorem 7.3.
- After having switched all these outputs of MProg_m, the public key $\mathsf{FHE.pk}$ does not depend on sk_0 anymore, and security follows by security of KeyGen.

Remark 7.4. As in the proof of Theorem 5.4, time-lock puzzle security is only used in the first step above, to hide the puzzles $\widetilde{\mathsf{ct}}_m$ with superpolynomial time parameter 2^m from a size-bounded adversary. Therefore, the relaxed time-lock puzzles as described in Definition 3.14 suffices.

Again, one can instantiate the rerandomizable encryption from DDH via ElGamal over a suitable group (see Remark 7.2). This yields the following analogue to Corollary 7.1.

Corollary 7.2. Assume the polynomial security of iO, of DDH over a group $\mathbb{G}$ of order p that is either a subgroup of $\mathbb{Z}_q^*$ or a suitable elliptic curve, of a strongly correct FSS for NC^1 shifts with range $\mathbb{Z}_p$, and of (relaxed) time-lock puzzles. Then there exists an unleveled FHE scheme.

In particular, one can replace the FSS assumption above with the polynomial hardness of either DDH over appropriate class groups [ADOS22, CL15] for prime-order q, DCR for q that is a strong RSA number, or of LWE (with sub-exponential modulus-to-noise ratio)[BKS19].

Remark 7.5 (Comparison with [CLTV15])**.** The work of [CLTV15] previously obtained an analogue of Theorem 7.4, assuming instead sub-exponentially-secure iO, sub-exponentially-secure puncturable PRFs (which can be built from sub-exponentially-secure one-way functions), and a quantitatively strong form of rerandomizable encryption, where the distribution of rerandomized ciphertexts is within exponentially small statistical distance to a fresh encryption.

We note that our notion of algebraically-rerandomizable encryption does imply perfect rerandomizability of the encryption (as a rerandomized ciphertext will be distributed identically to a ciphertext with uniform randomness over the group).

Thus, our construction improves on [CLTV15] in terms of quantitative hardness — all primitives are only required to be polynomially-secure, at the cost of functionally stronger primitives (shiftable PRFs instead of one-way functions, algebraically-rerandomizable encryption instead of plain rerandomizable encryption, and additionally relying on time-lock puzzles).

A similar comparison holds for [ACH20], who built unleveled FHE assuming polynomially secure iO, one-way functions, and extremely lossy functions, which can be built assuming the *exponential security of DDH*. Here again, we improve on quantitative assumptions, relying exclusively on polynomial security, at the cost of additionally assuming shiftable PRFs and time-lock puzzles: this is because algebraically rerandomizable encryption can be built from (the polynomial hardness of) DDH.

Relying on EFiO instead of iO. We observe that we use iO in three types of steps. In the first one, functional equivalence relies on correctness of the shiftable PRF. In a second step, it relies on the algebraically rerandomizability property of our encryption scheme. The third step relies on punctured PRF security in the proof of Theorem 7.4. Overall, if the shiftable PRF and the algebraic rerandomization procedure have efficient $\mathcal{EF}$ proofs of correctness, then we only need to rely on EFiO in our construction instead of iO. Combined with Theorem 6.1, and the fact that the algebraically rerandomizability of El-Gamal over a subgroup of $\mathbb{Z}_q^*$ can be argued in PV and thus $\mathcal{EF}$ (as it only relies on basic properties of modular arithmetic), we obtain the following theorem:

Theorem 7.5. *Assuming the polynomial security of* EFiO*, the polynomial hardness of LWE, of DDH over a subgroup of $\mathbb{Z}_p^*$, and the polynomial security of (relaxed) time-lock puzzles, there exists an unleveled FHE scheme.*

To our knowledge, Theorem 7.5 gives the first non-trivial construction of unleveled FHE, which (1) does not rely on circular security assumptions, and (2) only relies on falsifiable assumptions.

7.2 Adaptively-Sound SNARGs for Trapdoor Languages

Definition 7.2 (Trapdoor Language)**.** *We call $\mathcal{L}$ a t-trapdoor language if there exists a size circuit* Trap *of size t with the property*

$$\mathsf{Trap}(x) = 1 \iff x \in \mathcal{L}.$$

For an NP language $\mathcal{L}$, we define the underlying efficiently computable relation R such that $R(x,w) = 1 \iff (x,w) \in \mathcal{L}$, represented by a circuit.

Remark 7.6. Notice that a t-trapdoor language $\mathcal{L}$ cannot be non-uniformly hard to decide for polynomial t: this is because Trap efficiently decides $\mathcal{L}$. On the other hand, a *family* of trapdoor languages can potentially remain non-uniformly hard to decide on the average, over the random choice of a language in the family.

Definition 7.3 (SNARG for Trapdoor Languages)**.** *A succinct non-interactive argument (SNARG) for trapdoor languages is a tuple of PPT algorithms* $\mathsf{SNARG} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Ver})$ *defined as follows:*

- $\mathsf{Setup}(1^\lambda, 1^t, R) \to \mathsf{crs}$*: Takes as input the security parameter* λ*, trapdoor circuit size* t*, an* NP *relation* R*, and outputs a common reference string* crs*.*
- $\mathsf{Prove}(\mathsf{crs}, x, w) \to \pi$*: Takes as input a common reference string* crs*, an instance* x*, and a witness* w*. It outputs a proof* π*.*
- $\mathsf{Ver}(\mathsf{crs}, x, \pi) \to \{0,1\}$*: Takes as input a common reference string* crs*, an instance* x*, and a proof* π*. It outputs a decision bit.*

We require the following properties

- **Completeness***: For any security parameter* $\lambda \in \mathbb{N}$*, trapdoor circuit size* t*, any circuit* $R : \{0,1\}^n \times \{0,1\}^h \to \{0,1\}$*, and any instance witness pair* x*,* w *s.t.* $R(x,w) = 1$ *it holds that*

$$\Pr\left[\mathsf{Ver}(\mathsf{crs}, x, \mathsf{Prove}(\mathsf{crs}, x, w)) = 1 | \mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^t, R)\right] = 1.$$

- **Adaptive Soundness***: There exists a negligible function* negl *s.t. for all PPT adversary* $\mathcal{A}$ *and all* t*-trapdoor language with verification circuit* $R : \{0,1\}^n \times \{0,1\}^h \to \{0,1\}$*, it holds that*

$$\Pr\left[x^* \notin \mathcal{L} \wedge \mathsf{Ver}(\mathsf{crs}, x^*, \pi) = 1 | \mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^t, R), (x^*, \pi) \leftarrow \mathcal{A}(\mathsf{crs})\right] \leq \mathsf{negl}(\lambda).$$

- **Succinctness***: There exists a polynomial* p *such that for all Boolean circuits* $R : \{0,1\}^n \times \{0,1\}^h \to \{0,1\}$*, and all* crs *in the support of* $\mathsf{Setup}(1^\lambda, 1^t, R)$*, all statements* $x \in \{0,1\}^n$*, and all witnesses* $w \in \{0,1\}^h$*, the size of the proof* π *output by* $\mathsf{Prove}(\mathsf{crs}, x, w)$ *satisfies* $|\pi| \leq p(\lambda + \log(n+h))$*.*

Construction 7.2. Let iO be a polynomially-secure indistinguishability obfuscation scheme, and let Sig be a deterministic constrained signature scheme for poly-size circuits (Definition 4.1). We construct a SNARG for trapdoor languages as follows:

- $\mathsf{Setup}(1^\lambda, 1^t, R)$:
  Let $\mathsf{msk} \leftarrow \mathsf{Sig.Setup}(1^\lambda)$. Let $(\mathsf{vk_1}, \mathsf{sk_1}) \leftarrow \mathsf{Constrain}(\mathsf{msk}, \mathbf{1})$, where $\mathbf{1}$ is the constant 1 function. Define the circuit:

  > $P_{\mathsf{sk_1}}(x,w)$: If $R(x,w) = 1$, output $\mathsf{Sig.Sign}(\mathsf{sk_1}, x)$. Else, output $\perp$.

  Output $\mathsf{crs} = (\widetilde{P} = \mathsf{iO}(P_{\mathsf{sk_1}}), \mathsf{vk} = \mathsf{vk_1})$[18].
- $\mathsf{Prove}(\mathsf{crs}, x, w)$: Parse $\mathsf{crs} = (\widetilde{P}, \mathsf{vk})$. Output $\widetilde{P}(x,w)$.

[18]Technically, we pad $P_{\mathsf{sk_1}}$ have size $\mathsf{poly}(t, \lambda)$, where t is an upper bound on the size of the trapdoor for the language. More precisely, this size corresponds to the circuit size of $P_{\mathsf{sk_{Trap}}}$, defined below.

- $\mathsf{Ver}(\mathsf{crs}, x, \pi)$: Parse $\mathsf{crs} = (\widetilde{P}, \mathsf{vk})$. Output $\mathsf{Sig.Ver}(\mathsf{vk}, x, \pi)$.

Theorem 7.6. *If* iO *is a polynomially secure indistinguishability obfuscations,* $\{\mathcal{L}_{\mathsf{pp}}\}$ *is a family of trapdoor languages, and* Sig *is a deterministic constrained signature scheme for poly-size circuits, then Construction 7.2 is an adaptively-sound SNARG for* $\{\mathcal{L}_{\mathsf{pp}}\}$.

Proof. Completeness of Construction 7.2 directly follows from the correctness of iO and Sig.

We prove adaptive soundness in a sequence of hybrids:

- Hybrid 0: This hybrid distribution is the one from in the real experiment.
  Let $\mathsf{msk} \leftarrow \mathsf{Sig.Setup}(1^\lambda)$. Let $(\mathsf{vk_1}, \mathsf{sk_1}) \leftarrow \mathsf{Constrain}(\mathsf{msk}, \mathbf{1})$. Define the two circuits:

  > $P_{\mathsf{sk}_1}(x, w)$: If $R(x, w) = 1$, output $\mathsf{Sig.Sign}(\mathsf{sk_1}, x)$. Else, output $\perp$.

  The adversary receives $\mathsf{crs} = (\widetilde{P} = \mathsf{iO}(P_{\mathsf{sk}_1}), \mathsf{vk}_1)$.

- Hybrid 1: In this hybrid we change the proving circuit. Instead of a signing key that works for every input, we give out one that only verifies proofs on statements x such that $\mathsf{Trap}(x) = 1$. Specifically, let $\mathsf{msk} \leftarrow \mathsf{Sig.Setup}(1^\lambda)$. Let $(\mathsf{vk_1}, \mathsf{sk_1}) \leftarrow \mathsf{Constrain}(\mathsf{msk}, \mathbf{1})$. Let $(\mathsf{vk}_{\mathsf{Trap}}, \mathsf{sk}_{\mathsf{Trap}}) \leftarrow \mathsf{Constrain}(\mathsf{msk}, \mathsf{Trap})$. Define the new circuit:

  > $P_{\mathsf{sk}_{\mathsf{Trap}}}(x, w)$: If $R(x, w) = 1$, output $\mathsf{Sig.Sign}(\mathsf{sk}_{\mathsf{Trap}}, x)$. Else, output $\perp$.

  The adversary receives $\mathsf{crs} = (\widetilde{P} = \mathsf{iO}(P_{\mathsf{sk}_{\mathsf{Trap}}}), \mathsf{vk}_1)$.

  Claim 7.7. Assuming the security of iO and that $\mathcal{L}$ is a t-trapdoor language Hybrid 0 and Hybrid 1 are computationally indistinguishable.

  Proof. We argue that P_{sk_1} and $P_{\mathsf{sk}_{\mathsf{Trap}}}$ are functionally equivalent. This follows as P_{sk_1} only outputs a non-$\perp$ value on input $x \in \mathcal{L}$, and Trap perfectly decides $\mathcal{L}$. The determinism of Sig (definition 4.1) concludes the equivalence.

  Because the two circuits are equivalent the security of iO guarantees that Hybrid 0 and Hybrid 1 are computationally indistinguishable. □

- Hybrid 2: In this hybrid we change how the vk is computed. We now compute it as $\mathsf{vk}_{\mathsf{Trap}}$, instead of $\mathsf{vk_1}$.

  Claim 7.8. Assuming Sig is constraint-hiding Hybrid 1 and Hybrid 2 are computationally indistinguishable.

Claim 7.9. Assuming the constrained signature is (statistically) sound (definition 4.1), no (even unbounded) adversary can win the adaptive soundness experiment in Hybrid 2.

Proof. An adversary winning the adaptive soundness experiment in Hybrid 2 produces an input $x^* \notin \mathcal{L}$ accepted by $\mathsf{vk}_{\mathsf{Trap}}$, for some $x^* \notin \mathcal{L}$ which is equivalent to $\mathsf{Trap}(x^*) = 0$ by definition of Trap. Thus any such adversary would break the (statistical) soundness of the constrained signature. □

□

Remark 7.7 (Adaptive Soundness of Sahai-Waters for Trapdoor Languages)**.** In Section 4.2, we construct a deterministic constrained signature from a shiftable PRF and iO; we will call this an "inner" iO. In Construction 7.2, we obfuscate a circuit that has the constrained signing key hard-coded; we call this the "outer" iO. Technically, this double-use of iO is not necessary; the outer layer of obfuscation is enough to prove security. We keep the two layers in Construction 7.2, for clarity of notation, and the sake of conceptual abstraction.

Alternatively, one can remove the "inner" layer of iO from the deterministic constrained signatures. The resulting construction is *nearly identical* to the selectively secure SNARG of [SW14] with the differences that (1) we use a shiftable PRF instead of a puncturable PRF, and (2) Sahai-Waters check instead that *one-way functions* of the signatures are equal, which is no less secure. In other words, the proof above directly extends to show that Sahai-Waters, when instantiated with a shiftable PRF, is adaptively sound for trapdoor languages.

Going one step further, the transformation from FSS to shiftable PRF Theorem 4.1 also uses a layer of "inner" obfuscation. For the same reason as above, we can remove this inner layer, thus directly instantiating the construction of Construction 7.2 where the punctured PRF is replaced by an FSS evaluation.

Acknowledgements

We thank George Lu for discussions on injective PRGs. We thank anonymous reviewers of Crypto 2026 for suggestions and improvements over the time-lock puzzle construction.

A. Jain was supported in part by NSF CAREER 1942789, JP Morgan Faculty Award and research gifts from Ethereum Foundation, Stellar Development Foundation, and Cisco.

Part of this work was done while the authors were visiting the Simons Institute for the Theory of Computing, UC Berkeley.

References

[ABG+25] Joël Alwen, Chris Brzuska, Jérôme Govinden, Patrick Harasser, and Stefano Tessaro. Succinct PPRFs via memory-tight reductions. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part V*, volume 16004 of *LNCS*, pages 628–662. Springer, Cham, August 2025. 4, 55

[ACH20] Thomas Agrikola, Geoffroy Couteau, and Dennis Hofheinz. The usefulness of sparsifiable inputs: How to avoid subexponential iO. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part I*, volume 12110 of *LNCS*, pages 187–219. Springer, Cham, May 2020. 6, 21, 70

[ACK23] Thomas Attema, Pedro Capitão, and Lisa Kohl. On homomorphic secret sharing from polynomial-modulus LWE. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 3–32. Springer, Cham, May 2023. 6, 58

[ADOS22] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 421–452. Springer, Cham, August 2022. 19, 22, 67, 68, 70

[AJ15] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 308–326. Springer, Berlin, Heidelberg, August 2015. 20

[Ajt99] Miklós Ajtai. Generating hard instances of the short basis problem. In Jirí Wiedermann, Peter van Emde Boas, and Mogens Nielsen, editors, *ICALP 99*, volume 1644 of *LNCS*, pages 1–9. Springer, Berlin, Heidelberg, July 1999. 92

[AMR25] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for RAM: Fully succinct randomised encodings and more. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 236–268. Springer, Cham, August 2025. 5

[AS15] Gilad Asharov and Gil Segev. Limits on the power of indistinguishability obfuscation and functional encryption. In Venkatesan Guruswami, editor, *56th FOCS*, pages 191–209. IEEE Computer Society Press, October 2015. 20

[BDV17] Nir Bitansky, Akshay Degwekar, and Vinod Vaikuntanathan. Structure vs. hardness through the obfuscation lens. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 696–723. Springer, Cham, August 2017. 20

[BGG+14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014. 3, 85

[BGI+01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2001. 4

[BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In Hugo Krawczyk, editor, *PKC 2014*, volume 8383 of *LNCS*, pages 501–519. Springer, Berlin, Heidelberg, March 2014. 20, 23

[BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367. Springer, Berlin, Heidelberg, April 2015. 5, 18, 22

[BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303. ACM Press, October 2016. 22

[BGI19] Elette Boyle, Niv Gilboa, and Yuval Ishai. Secure computation with preprocessing via function secret sharing. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part I*, volume 11891 of *LNCS*, pages 341–371. Springer, Cham, December 2019. 22

[BGJ+16] Nir Bitansky, Shafi Goldwasser, Abhishek Jain, Omer Paneth, Vinod Vaikuntanathan, and Brent Waters. Time-lock puzzles from randomized encodings. In Madhu Sudan, editor, *ITCS 2016*, pages 345–356. ACM, January 2016. 5, 30

[BKS19] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 3–33. Springer, Cham, May 2019. 68, 70

[BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 719–737. Springer, Berlin, Heidelberg, April 2012. 3, 55, 92, 93

[BPR15] Nir Bitansky, Omer Paneth, and Alon Rosen. On the cryptographic hardness of finding a Nash equilibrium. In Venkatesan Guruswami, editor, *56th FOCS*, pages 1480–1498. IEEE Computer Society Press, October 2015. 20

[BPW16] Nir Bitansky, Omer Paneth, and Daniel Wichs. Perfect structure on the edge of chaos - trapdoor permutations from indistinguishability obfuscation. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part I*, volume 9562 of *LNCS*, pages 474–502. Springer, Berlin, Heidelberg, January 2016. 20, 81, 84

[BS23] Nir Bitansky and Tomer Solomon. Bootstrapping homomorphic encryption via functional encryption. In Yael Tauman Kalai, editor, *ITCS 2023*, volume 251, pages 17:1–17:23. LIPIcs, January 2023. 5, 14, 18, 30

[BTVW17] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 264–302. Springer, Cham, November 2017. 3, 85, 86

[Bus85] Samuel R Buss. *Bounded arithmetic*. Princeton University, 1985. 85

[BV15] Nir Bitansky and Vinod Vaikuntanathan. Indistinguishability obfuscation from functional encryption. In Venkatesan Guruswami, editor, *56th FOCS*, pages 171–190. IEEE Computer Society Press, October 2015. 20

[BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 280–300. Springer, Berlin, Heidelberg, December 2013. 20, 23

[BZ14] Dan Boneh and Mark Zhandry. Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 480–499. Springer, Berlin, Heidelberg, August 2014. 20, 34, 35

[CCH+19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-Shamir: from practice to theory. In Moses Charikar and Edith Cohen, editors, *51st ACM STOC*, pages 1082–1090. ACM Press, June 2019. 6, 15, 30

[CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th ACM STOC*, pages 209–218. ACM Press, May 1998. 6, 15

[CGKS23] Matteo Campanelli, Chaya Ganesh, Hamidreza Khoshakhlagh, and Janno Siim. Impossibilities in succinct arguments: Black-box extraction and more. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 465–489. Springer, Cham, July 2023. 7

[CL15] Guilhem Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from DDH. In Kaisa Nyberg, editor, *CT-RSA 2015*, volume 9048 of *LNCS*, pages 487–505. Springer, Cham, April 2015. 67, 68, 70

[CLTV15] Ran Canetti, Huijia Lin, Stefano Tessaro, and Vinod Vaikuntanathan. Obfuscation of probabilistic circuits and applications. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 468–497. Springer, Berlin, Heidelberg, March 2015. 6, 7, 15, 16, 17, 18, 20, 62, 68, 70

[CMPR23] Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudorandom functions from homomorphic secret sharing. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 194–224. Springer, Cham, April 2023. 20, 32

[Coo75] Stephen A. Cook. Feasibly constructive proofs and the propositional calculus (preliminary version). In *Proceedings of the Seventh Annual ACM Symposium on Theory of Computing*, STOC '75, page 83–97, New York, NY, USA, 1975. Association for Computing Machinery. 4, 6

[CR79] Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44(1):36—50, 1979. 5, 6

[DHRW16] Yevgeniy Dodis, Shai Halevi, Ron D. Rothblum, and Daniel Wichs. Spooky encryption and its applications. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 93–122. Springer, Berlin, Heidelberg, August 2016. 6, 20, 22

[DJJ+25] Pratish Datta, Abhishek Jain, Zhengzhong Jin, Alexis Korb, Surya Mathialagan, and Amit Sahai. Incrementally verifiable computation for NP from standard assumptions. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 611–642. Springer, Cham, August 2025. 7

[GGH+13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th FOCS*, pages 40–49. IEEE Computer Society Press, October 2013. 4

[GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479. IEEE Computer Society Press, October 1984. 23

[GGSW13] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 467–476. ACM Press, June 2013. 4

[GKVW20] Rishab Goyal, Venkata Koppula, Satyanarayana Vusirikala, and Brent Waters. On perfect correctness in (lockable) obfuscation. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 229–259. Springer, Cham, November 2020. 55

[GPS16] Sanjam Garg, Omkant Pandey, and Akshayaram Srinivasan. Revisiting the cryptographic hardness of finding a nash equilibrium. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 579–604. Springer, Berlin, Heidelberg, August 2016. 20

[GPSZ17] Sanjam Garg, Omkant Pandey, Akshayaram Srinivasan, and Mark Zhandry. Breaking the sub-exponential barrier in obfustopia. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 156–181. Springer, Cham, April / May 2017. 20

[GS16] Sanjam Garg and Akshayaram Srinivasan. Single-key to multi-key functional encryption with polynomial loss. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part II*, volume 9986 of *LNCS*, pages 419–442. Springer, Berlin, Heidelberg, October / November 2016. 20

[GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 75–92. Springer, Berlin, Heidelberg, August 2013. 3, 55, 86

[GVW15] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Predicate encryption for circuits from LWE. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 503–523. Springer, Berlin, Heidelberg, August 2015. 3, 85, 86

[HW15] Pavel Hubacek and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In Tim Roughgarden, editor, *ITCS 2015*, pages 163–172. ACM, January 2015. 3, 24, 88

[JJ22] Abhishek Jain and Zhengzhong Jin. Indistinguishability obfuscation via mathematical proofs of equivalence. In *63rd FOCS*, pages 1023–1034. IEEE Computer Society Press, October / November 2022. 4, 5, 9, 10, 12, 13, 14, 16, 18, 27, 28, 29, 39, 42, 47, 53

[JJMP25] Abhishek Jain, Zhengzhong Jin, Surya Mathialagan, and Omer Paneth. On succinct obfuscation via propositional proofs. In *66th FOCS*, pages 1703–1740. IEEE Computer Society Press, December 2025. 4

[JKLM25] Zhengzhong Jin, Yael Tauman Kalai, Alex Lombardi, and Surya Mathialagan. Universal SNARGs for NP from proofs of correctness. In Michal Koucký and Nikhil Bansal, editors, *57th ACM STOC*, pages 933–943. ACM Press, June 2025. 14, 15, 54, 85, 86, 89

[JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd ACM STOC*, pages 60–73. ACM Press, June 2021. 4, 7, 20

[JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0. In Orr Dunkelman and Stefan Dziembowski, editors,

EUROCRYPT 2022, Part I, volume 13275 of *LNCS*, pages 670–699. Springer, Cham, May / June 2022. 4, 7, 20

[KLMR18] Yuan Kang, Chengyu Lin, Tal Malkin, and Mariana Raykova. Obfuscation from polynomial hardness: Beyond decomposable obfuscation. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 407–424. Springer, Cham, September 2018. 20

[KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 669–684. ACM Press, November 2013. 20, 23

[KWZ22] Venkata Koppula, Brent Waters, and Mark Zhandry. Adaptive multiparty NIKE. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 244–273. Springer, Cham, November 2022. 21

[LNW25] George Lu, Shafik Nassar, and Brent Waters. Succinct computational secret sharing for monotone circuits. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part VIII*, volume 16252 of *LNCS*, pages 99–129. Springer, Singapore, December 2025. 81

[LV22] Alex Lombardi and Vinod Vaikuntanathan. Correlation-intractable hash functions via shift-hiding. In Mark Braverman, editor, *ITCS 2022*, volume 215, pages 102:1–102:16. LIPIcs, January / February 2022. 9, 11, 20

[LZ21] Qipeng Liu and Mark Zhandry. Decomposable obfuscation: A framework for building applications of obfuscation from polynomial hardness. *Journal of Cryptology*, 34(3):35, July 2021. 4, 20, 30

[Mat25] Surya Mathialagan. *Succinct Cryptography via Propositional Proofs*. Thesis, Massachusetts Institute of Technology, May 2025. Accepted: 2025-11-25T19:38:41Z. 14, 15, 54, 85, 86

[MDS25] Yaohua Ma, Chenxin Dai, and Elaine Shi. Quasi-linear indistinguishability obfuscation via mathematical proofs of equivalence and applications. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 157–186. Springer, Cham, May 2025. 3, 4, 25, 26, 39, 47, 55, 88, 89

[MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, April 2012. 3, 55, 92

[MT19] Giulio Malavolta and Sri Aravinda Krishnan Thyagarajan. Homomorphic time-lock puzzles and applications. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 620–649. Springer, Cham, August 2019. 20, 30

[Nao91] Moni Naor. Bit commitment using pseudorandomness. *Journal of Cryptology*, 4(2):151–158, January 1991. 81

[Nao03] Moni Naor. On cryptographic assumptions and challenges (invited talk). In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 96–109. Springer, Berlin, Heidelberg, August 2003. 5

[NW26] Shafik Nassar and Brent Waters. Adaptive nike for unbounded parties. *Crypto*, 2026. 21

[OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 678–708. Springer, Cham, October 2021. 7, 19, 22, 67

[PS18] Chris Peikert and Sina Shiehian. Privately constraining and programming PRFs, the LWE way. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 675–701. Springer, Cham, March 2018. 3, 6, 9, 11, 15, 23, 24, 55, 58, 89, 90, 91

[PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 89–114. Springer, Cham, August 2019. 3, 6, 15, 30, 55, 58, 91, 92

[RS21] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 687–717, Virtual Event, August 2021. Springer, Cham. 7, 19, 22, 67

[RSW96] Ronald L Rivest, Adi Shamir, and David A Wagner. Time-lock puzzles and timed-release crypto. 1996. 4, 5, 30

[RVV24] Seyoon Ragavan, Neekon Vafa, and Vinod Vaikuntanathan. Indistinguishability obfuscation from bilinear maps and LPN variants. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 3–36. Springer, Cham, December 2024. 4, 7, 20

[SC04] Michael Soltys and Stephen A. Cook. The proof complexity of linear algebra. *Ann. Pure Appl. Log.*, 130(1-3):277–323, 2004. 85, 86

[SLM+23] Shravan Srinivasan, Julian Loss, Giulio Malavolta, Kartik Nayak, Charalampos Papamanthou, and Sri Aravinda Krishnan Thyagarajan. Transparent batchable time-lock puzzles and applications to byzantine consensus. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 554–584. Springer, Cham, May 2023. 20

[SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th ACM STOC*, pages 475–484. ACM Press, May / June 2014. 4, 5, 7, 10, 11, 18, 19, 73

[WW24] Brent Waters and David J. Wu. Adaptively-sound succinct arguments for NP from indistinguishability obfuscation. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *56th ACM STOC*, pages 387–398. ACM Press, June 2024. 7

[WW25] Brent Waters and David J. Wu. A pure indistinguishability obfuscation approach to adaptively-sound SNARGs for NP. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 292–326. Springer, Cham, August 2025. 7, 20

[WZ24] Brent Waters and Mark Zhandry. Adaptive security in SNARGs via iO and lossy functions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 72–104. Springer, Cham, August 2024. 7

[Zha16] Mark Zhandry. The magic of ELFs. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 479–508. Springer, Berlin, Heidelberg, August 2016. 21

A Injective PRGs

This section shows how to construct injective PRGs from injective OWFs and iO (Theorem A.2). Our construction matches the SIOWF to weak OWF transform of [BPW16], but our proof uses the hardcore-bit puncturing trick from [LNW25].

A.1 Preliminaries

Definition A.1 (Two-message commitments)**.** *A two-message pseudorandom commitment scheme is a pair of PPT algorithms* $(\mathsf{Com}_1, \mathsf{Com}_2)$ *defined as follows:*

- $\mathsf{Com}_1(1^\lambda) \to \mathsf{ck}$ *outputs the commitment key.*
- $\mathsf{Com}_2(x, \mathsf{ck}; r) \to \mathsf{com}$ *takes a message* $x \in \{0,1\}^\lambda$*, commitment key* ck*, and randomness* $r \in \{0,1\}^\lambda$*, and outputs the commitment* $\mathsf{com} \in \{0,1\}^{n(\lambda)}$*.*

We require the following properties:

Statistical Binding: *for all* $\lambda \in \mathbb{N}$*, and for all distinct* $x, x' \in \{0,1\}^\lambda$*,*

$$\Pr\left[\exists x, x', r, r' \text{ s.t. } \mathsf{Com}_2(x, \mathsf{ck}; r) = \mathsf{Com}_2(x', \mathsf{ck}; r') \,\middle|\, \mathsf{ck} \leftarrow \mathsf{Com}_1(1^\lambda)\right] \leq \mathsf{negl}(\lambda).$$

Pseudorandomness: *for all PPT adversaries* $\mathcal{A}$*,*

$$|\Pr[\mathcal{A}(\mathsf{ck}, \mathsf{Com}_2(x, \mathsf{ck}; r)) = 1] - \Pr[\mathcal{A}(\mathsf{ck}, u) = 1]| \leq \mathsf{negl}(\lambda),$$

where $\mathsf{ck} \leftarrow \mathsf{Com}_1(1^\lambda)$*,* $r \leftarrow \{0,1\}^\lambda$*,* $u \leftarrow \{0,1\}^{n(\lambda)}$*, and* x *is chosen by* $\mathcal{A}$*.*

Remark A.1. The commitment scheme from one-way functions by [Nao91] meets these requirements for any polynomial stretch $n(\lambda)$. In particular, such commitments exist assuming (polynomial secure) one-way functions.

A.2 Injective PRGs from Injective OWF and iO

Construction A.1. Let iO be an indistinguishability obfuscator for polynomial size circuits, and let PRF be a puncturable PRF. Let $(\mathsf{Com}_1, \mathsf{Com}_2)$ be a two-message statistically binding commitment scheme with a second message size of $n(\lambda) > \lambda$.

$\mathsf{KeyGen}(1^\lambda)$**:** Sample $\mathsf{ck} \leftarrow \mathsf{Com}_1(1^\lambda)$ and $k \leftarrow \{0,1\}^\lambda$. Define the circuit:

> $C_{\mathsf{ck},k}(x \in \{0,1\}^\lambda)$: Output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$.

Output $\widetilde{C} \leftarrow \mathsf{iO}(C_{\mathsf{ck},k})$.

$\mathsf{PRG}_{\widetilde{C}}(x \in \{0,1\}^\lambda)$**:** Output $\widetilde{C}(x)$.

Theorem A.1. *If* OWF *is a polynomially secure injective one-way function,* iO *is a polynomially secure indistinguishability obfuscation, and* Com *is a statistically binding pseudorandom commitment, then Construction A.1 is a family of injective pseudorandom generators.*

Proof.

Claim A.1 (Injectivity)**.** $\mathsf{PRG}_{\widetilde{C}}$ is injective with overwhelming probability over the choice of $\widetilde{C}$.

Proof. For a random ck and any k, with overwhelming probability over ck, the function $x \mapsto \mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$ is injective by statistical binding of the commitment scheme. Since $\widetilde{C}$ computes the same function as $C_{\mathsf{ck},k}$ (by correctness of iO), the claim follows. □

We prove pseudorandomness in the following sequence of hybrids:

- Hybrid 0: This is the real experiment. The adversary $\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{\mathsf{ck},S}(x) := \mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_S(x))), \widetilde{C}(x^*))$ for random $x^* \leftarrow \{0,1\}^\lambda$.
- Hybrid 1: Let $k_{x^*} \leftarrow \mathsf{Punc}(1^\lambda, k, \{x^*\})$ We define a new circuit:

  Circuit C_1

  Hard-coded: ck, k_{x^*}, $\mathsf{PRF}_k(x^*)$, x^*.
  If $x \neq x^*$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_{k_{x^*}}(x))$.
  Otherwise output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x^*))$.

  $\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_1), \widetilde{C}(x^*))$ for a random $x^* \leftarrow \{0,1\}^\lambda$ hard-coded in C_1.

  Claim A.2. Assuming iO is (polynomially) secure, and PRF is a correct puncturable PRF, Hybrid 0 and Hybrid 1 are computationally indistinguishable.

  This follows by functional equivalence of $C_{\mathsf{ck},k}$ and $C_{1,\mathsf{ck},k_{x^*},\mathsf{PRF}_k(x^*),x^*}$, which follows from puncturable correctness.

- Hybrid 2: We define a new circuit:

  Circuit C_2

  Hard-coded: ck, k_{x^*}, r, x^*.
  If $x \neq x^*$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_{k_{x^*}}(x))$.
  Otherwise, output $\mathsf{Com}_2(x, \mathsf{ck}; r)$.

  $\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_2), \widetilde{C}(x^*))$ for random $x^* \leftarrow \{0,1\}^\lambda$ and random $r \leftarrow \{0,1\}^\lambda$ hard-coded in C_2.

  Claim A.3. Assuming PRF is a (polynomially) secure puncturable PRF, Hybrid 1 and Hybrid 2 are computationally indistinguishable.

- Hybrid 3: We define a new circuit:

  Circuit C_3

  Hard-coded: ck, k_{x^*}, R, x^*.
  If $x \neq x^*$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_{k_{x^*}}(x))$.
  Otherwise, output R.

  $\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{3,\mathsf{ck},k_{x^*},R,x^*}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$ and random $R \leftarrow \{0,1\}^{n(\lambda)}$.

Claim A.4. Assuming $(\mathsf{Com}_1, \mathsf{Com}_2)$ is pseudorandom, Hybrid 2 and Hybrid 3 are computationally indistinguishable.

- Hybrid 4: We define a new circuit:

Circuit C_4
Hard-coded: ck, k, R, x^* If $x \neq x^*$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$. Otherwise, output R.

$\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{4,\mathsf{ck},k_{x^*},R,x^*}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$ and random $R \leftarrow \{0,1\}^{n(\lambda)}$.

Claim A.5. Assuming iO is (polynomially) secure and PRF is a correct puncturable PRF, Hybrid 3 and Hybrid 4 are computationally indistinguishable because $C_{3,\mathsf{ck},k_{x^*},R,x^*}$ and $C_{4,\mathsf{ck},k_{x^*},R,x^*}$ are functionally equivalent.

- Hybrid 5: We define a new circuit:

Circuit C_5
Hard-coded: ck, k, R, x^* If $(\mathsf{OWF}(x), \mathsf{HC}(x)) \neq (\mathsf{OWF}(x^*), \mathsf{HC}(x^*))$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$. Otherwise, output R.

$\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{5,\mathsf{ck},k,R,x^*}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$ and random $R \leftarrow \{0,1\}^{n(\lambda)}$.

Here, OWF denotes an injective one-way function (from our assumptions), and HC an associated hard-core bit.

Claim A.6. Assuming iO is polynomially secure, and OWF is injective, Hybrid 4 and Hybrid 5 are computationally indistinguishable.

This follows as $C_{4,\mathsf{ck},k_{x^*},R,x^*}$ and $C_{5,\mathsf{ck},k,R,x^*}$ are functionally equivalent since OWF is injective.

- Hybrid 6: We define a new circuit:

Circuit C_6
Hard-coded: $\mathsf{ck}, k, R, b, x^*$ If $(\mathsf{OWF}(x), \mathsf{HC}(x)) \neq (\mathsf{OWF}(x^*), b)$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$. Otherwise, output R.

$\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{6,\mathsf{ck},k,R,b,x^*}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$, $b \leftarrow \{0,1\}$, and $R \leftarrow \{0,1\}^{n(\lambda)}$.

Claim A.7. Assuming HC is a hardcore bit for OWF, then Hybrid 5 and Hybrid 6 are computationally indistinguishable.

- Hybrid 7: We define a new circuit:

Circuit C_7
Hard-coded: $\mathsf{ck}, k, R, b, x^*$ If $(\mathsf{OWF}(x), \mathsf{HC}(x)) \neq (\mathsf{OWF}(x^*), b \oplus 1)$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$. Output R, otherwise.

$\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{7,\mathsf{ck},k,R,b,x^*}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$, $b \leftarrow \{0,1\}$, and $R \leftarrow \{0,1\}^{n(\lambda)}$.

Claim A.8. Hybrid 6 is identically distributed to Hybrid 7.

This follows as b is sampled uniformly at random.

- Hybrid 8: We define a new circuit:

Circuit $C_{8,\mathsf{ck},k,R,x^*}(x)$
Hard-coded: ck, k, R, x^* If $(\mathsf{OWF}(x), \mathsf{HC}(x)) \neq (\mathsf{OWF}(x^*), \mathsf{HC}(x^*) \oplus 1)$, output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$. Otherwise, output R.

$\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{8,\mathsf{ck},k,R,x^*}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$ and random $R \leftarrow \{0,1\}^{n(\lambda)}$.

Claim A.9. Assuming HC is a hardcore bit for OWF then Hybrid 7 and Hybrid 8 are computationally indistinguishable.

- Hybrid 9: We define a new circuit:

Circuit $C_9(x)$
Hard-coded: ck, k Output $\mathsf{Com}_2(x, \mathsf{ck}; \mathsf{PRF}_k(x))$.

$\mathcal{A}$ is given $(\widetilde{C} := \mathsf{iO}(C_{9,\mathsf{ck},k}), R)$ for random $x^* \leftarrow \{0,1\}^\lambda$ and random $R \leftarrow \{0,1\}^{n(\lambda)}$.

Claim A.10. Assuming iO is secure, Hybrid 8 and Hybrid 9 are computationally indistinguishable.

Proof. C_8 and C_9 are functionally equivalent on inputs $x \neq x^*$ because OWF is injective, and on input $x = x^*$ because $\mathsf{HC}(x) \neq \mathsf{HC}(x^*) \oplus 1$. □

Hybrid 0 exactly matches the real distribution and Hybrid 9 the uniform one. Therefore, the adversary can not distinguish them except with negligible probability. □

Combined with the construction of injective OWFs from polynomially-secure iO and polynomially-secure OWF [BPW16], we obtain the following.

Theorem A.2. *Assuming the polynomial security of iO and of one-way functions, there exists an injective PRG.*

B PV-friendly building blocks

In this section, we present the concrete constructions of the PV-friendly building blocks stated in Theorem 6.1. We also provide sketches on how to prove their correctness/statistical soundness properties in PV.

All following constructions are lattice-based. When instantiating with truncated Gaussian error distributions, proving their correctness generally boils down to providing norm bounds to sums or products of low-norm matrices. These statements follow basic arithmetic and matrix properties, and are known to be provable in PV [Bus85, SC04, JKLM25]. In particular, as pointed out in [Mat25], the following statement is provable in PV:

$$\forall \mathbf{A} \in \mathbb{Z}_p^{n\times m}, \mathbf{B} \in \mathbb{Z}_p^{m\times k}, \|\mathbf{AB}\|_\infty \le m \cdot \|\mathbf{A}\|_\infty \cdot \|\mathbf{B}\|_\infty. \tag{4}$$

B.1 Lattice notations

In this section, we use $\mathbb{Z}_q$ to denote the integer modulo q ring. We use bold face upper case letters (e.g., $\mathbf{A}$) to denote matrices, and bold face lowercase letters (e.g., $\mathbf{a}$) to denote vectors. All vectors are by default column vectors. For a matrix $\mathbf{A}$, we use $\|\mathbf{A}\|_\infty$ to denote the maximum absolute value of its entries. We use $\mathbf{A} \otimes \mathbf{B}$ to denote the Kronecker product between matrices $\mathbf{A}$ and $\mathbf{B}$. For matrices with suitable dimensions, it holds that $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$.

We denote the gadget vector $\mathbf{g}_q^\intercal = (1, 2, 4, \ldots, 2^{k-1}) \in \mathbb{Z}_q^k$ where $k = \lceil \log q \rceil$, and denote the gadget matrix $\mathbf{G}_{n,q} = \mathbf{I}_n \otimes \mathbf{g}_q^\intercal \in \mathbb{Z}_q^{n\times nk}$. When clear from context, we omit the subscript dimension n and modulus q.

For $\mathbb{Z}_q$ vectors $\mathbf{a} \in \mathbb{Z}_q^n$, we denote $\mathbf{G}^{-1}(\mathbf{a})$ to be the binary decomposition of $\mathbf{a}$ such that $\mathbf{G}_{n,q} \cdot \mathbf{G}^{-1}(\mathbf{a}) = \mathbf{a}$ and $\|\mathbf{G}^{-1}(\mathbf{a})\|_\infty \le 1$. We extend the notation to matrices column-wise.

We use the notation $\mathsf{Bit}(\cdot)$ to denote the bit-wise representations of an object, which is treated as a binary vector. There exists a packing matrix $\mathbf{P}_{n,m,q} \in \{0,1\}^{n^2mk^2\times m}$ where $k = \lceil \log q \rceil$, such that for any $\mathbf{M} \in \mathbb{Z}_q^{n\times m}$, it holds that $(\mathsf{Bit}(\mathbf{M})^\intercal \otimes \mathbf{G}_n) \cdot \mathbf{P}_{n,m,q} = \mathbf{M}$.

B.2 BGG+ lattice homomorphism [BGG+14, GVW15, BTVW17]

We recall the core homomorphic gadget in the ABE construction of [BGG+14]. To simplify the notation, we adopt the formulation from [BTVW17]

Lemma B.1 ([BGG+14, BTVW17])**.** *Let n, m, q be lattice parameters, and $m = n\lceil \log q \rceil$ is consistent with the gadget matrix dimension. There exist efficient algorithms* EvalF, EvalFX *with the following syntax.*

- $\mathsf{EvalF}(\mathbf{A} \in \mathbb{Z}_q^{n\times \ell m}, f : \{0,1\}^\ell \to \{0,1\}^{\ell'}) \to \mathbf{H}_f \in \mathbb{Z}_q^{\ell m\times \ell' m}$.
- $\mathsf{EvalFX}(\mathbf{A} \in \mathbb{Z}_q^{n\times \ell m}, f : \{0,1\}^\ell \to \{0,1\}^{\ell'}, \mathbf{x} \in \{0,1\}^\ell) \to \mathbf{H}_{f,\mathbf{x}} \in \mathbb{Z}_q^{\ell m\times \ell' m}$.

The two algorithms provide the following correctness guarantee: For any $\mathbf{A} \in \mathbb{Z}_q^{n\times \ell m}$, any $\mathbf{x} \in \{0,1\}^\ell$, and any $f : \{0,1\}^\ell \to \{0,1\}^{\ell'}$ that is described as a depth d circuit, it holds that

$$(\mathbf{A} - \mathbf{x}^\intercal \otimes \mathbf{G}) \cdot \mathbf{H}_{f,\mathbf{x}} = \mathbf{A} \cdot \mathbf{H}_f - f(\mathbf{x})^\intercal \otimes \mathbf{G},$$

and that $\|\mathbf{H}_f\|_\infty, \|\mathbf{H}_{f,\mathbf{x}}\|_\infty \le (m+1)^d$.

Claim B.1. The correctness of EvalF and EvalFX in Lemma B.1 is provable in PV.

The correctness equation follows from an induction over the basic properties of matrix multiplications. Following [SC04, JKLM25], these statements can be formalized in PV. The norm bound of $\mathbf{H}$ follows from a direct induction using Equation (4), which is also provable in PV.

For the application for shiftable PRF, we need to rely on the weak attribute hiding property of the BGG+ homomorphic gadget from [GVW15], which allows EvalFX to not depend on part of the input when the dependency on the hidden input is linear. We formalize it as follow.

Lemma B.2 ([GVW15, BTVW17]). *Let n, m, q be lattice parameters, and $m = n\lceil \log q \rceil$ is consistent with the gadget matrix dimension. There exist efficient algorithms* PEvalF, PEvalFX *with the following syntax.*

- $\mathsf{PEvalF}(\mathbf{A} \in \mathbb{Z}_q^{n \times \ell m}, \mathbf{B} \in \mathbb{Z}_q^{n \times tm}, F : \{0,1\}^\ell \to \mathbb{Z}_q^{t \times \ell'}) \to \mathbf{H}_F \in \mathbb{Z}_q^{(\ell+t)m \times \ell' m}$.
- $\mathsf{PEvalFX}(\mathbf{A} \in \mathbb{Z}_q^{n \times \ell m}, \mathbf{B} \in \mathbb{Z}_q^{n \times tm}, F : \{0,1\}^\ell \to \mathbb{Z}_q^{t \times \ell'}, \mathbf{x} \in \{0,1\}^\ell) \to \mathbf{H}_{F,\mathbf{x}} \in \mathbb{Z}_q^{(\ell+t)m \times \ell' m}$.

The two algorithm provides the following correctness guarantee: For any $\mathbf{A} \in \mathbb{Z}_q^{n \times \ell m}$, any $\mathbf{B} \in \mathbb{Z}_q^{n \times tm}$, any $\mathbf{x} \in \{0,1\}^\ell$, any $\mathbf{r} \in \mathbb{Z}_q^t$, and any function $F : \{0,1\}^\ell \to \mathbb{Z}_q^{t \times \ell'}$ described as a depth d circuit, it holds that

$$(\mathbf{A} - \mathbf{x}^\intercal \otimes \mathbf{G} | \mathbf{B} - \mathbf{r}^\intercal \otimes \mathbf{G}) \cdot \mathbf{H}_{F,\mathbf{x}} = (\mathbf{A}|\mathbf{B}) \cdot \mathbf{H}_F - (\mathbf{r}^\intercal \cdot F(\mathbf{x})) \otimes \mathbf{G}$$

and that $\|\mathbf{H}_F\|_\infty, \|\mathbf{H}_{F,\mathbf{x}}\|_\infty \le \ell \ell' m^{\theta(d)}$, where $\theta(d)$ is a shorthand notation for some fixed linear function in d.

The correctness of the lemma follows a similar induction as in Lemma B.1, thus provable in PV.

Claim B.2. The correctness of PEvalF and PEvalFX in Lemma B.2 is provable in PV.

B.3 GSW leveled homomorphic encryption from LWE [GSW13]

We now recall the GSW leveled homomorphic encryption scheme from LWE [GSW13]. To better illustrate the PV-friendliness of the scheme, and to fit better with the later usage for correlation intractable hash functions, we describe the construction using the BGG formulation from Lemma B.1.

Lemma B.3 (GSW homomorphic encryption [GSW13]). *Let n, q be lattice parameters, $m = n\lceil \log q \rceil$ and $w = 2m$. Let χ be a truncated discrete Gaussian error distribution with bound B, and that $B \cdot (m+1)^{d+1} < q/4$ for some depth parameter d. Then, assuming polynomial hardness of $LWE_{n-1,w,q,\chi}$, the following scheme is a leveled homomorphic encryption scheme for circuits of depth at most d.*

- $\mathsf{KeyGen}(1^\lambda) \to (\mathsf{pk}, \mathsf{sk})$*: Sample* $\overline{\mathbf{A}} \leftarrow \mathbb{Z}_q^{(n-1) \times w}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^{(n-1)}$, $\mathbf{e} \leftarrow \chi^w$, *output*

$$\mathsf{pk} = \mathbf{A} = \begin{pmatrix} \overline{\mathbf{A}} \\ \mathbf{s}^\intercal \overline{\mathbf{A}} + \mathbf{e}^\intercal \end{pmatrix}, \quad \mathsf{sk} = (-\mathbf{s}^\intercal, 1)^\intercal.$$

- $\mathsf{Enc}(\mathsf{pk}, \mu \in \{0,1\}) \to \mathsf{ct}$*: Sample* $\mathbf{R} \leftarrow \{0,1\}^{w \times m}$. *Output* $\mathsf{ct} = \mathbf{A}\mathbf{R} + \mu \mathbf{G}$.
- $\mathsf{Eval}(f, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell) \to \mathsf{ct}'$*: Concatenate each ciphertext* $\mathsf{ct}_i \in \mathbb{Z}_q^{n \times m}$ *column-wise to form* $\mathbf{C} \in \mathbb{Z}_q^{n \times \ell m}$. *Compute* $\mathbf{H}_f = \mathsf{EvalF}(\mathbf{C}, f)$ *from Lemma B.1, and output* $\mathsf{ct}' = \mathbf{C} \cdot \mathbf{H}_f$.
- $\mathsf{Dec}(\mathsf{sk}, \mathsf{ct}) \to \mu \in \{0,1\}$*: Let* $\mathbf{w} = \mathbf{G}^{-1}((0, \ldots, \lfloor q/2 \rceil))$. *Compute* $z = \mathsf{sk}^\intercal \cdot \mathsf{ct} \cdot \mathbf{w}$. *Output* $\mu = 0$ *if* $|z| < q/4$, *and output* $\mu = 1$ *otherwise.*

It was shown in [Mat25] that the correctness of the GSW scheme from LWE is provable in PV.

Remark B.1. One crucial property of the GSW scheme is that the decryption circuit can be described with a circuit of depth $\mathsf{poly}(\log n, \log\log q)$. Therefore, via choosing appropriate parameters, the scheme can support homomorphically evaluating its own decryption circuit. The property is crucially used in the construction of SEH from LWE.

For the application for correlation intractable hash functions, we introduce the correctness of the randomness homomorphism in GSW evaluation.

Claim B.3. Let $\mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R})$ denote the natural extension to encrypting a vector $\mathbf{x}$ in a bit-by-bit manner by $\mathsf{ct} = \mathbf{AR} + \mathbf{x}^\intercal \otimes \mathbf{G}$, with randomness $\mathbf{R}$ that is not necessarily a bit matrix. There exists an efficient algorithm REval defined as follows:

- $\mathsf{REval}(f, \mathsf{ct}_\mathbf{x}, \mathbf{R}, \mathbf{x}) \to \mathbf{R}_f$: Compute $\mathbf{H}_{f,\mathbf{x}} = \mathsf{EvalFX}(\mathbf{C}, f, \mathbf{x})$ from Lemma B.1, where $\mathbf{C} = \mathsf{ct}_\mathbf{x} = \mathbf{AR} + \mathbf{x}^\intercal\mathbf{G}$. Output $\mathbf{R}_f = \mathbf{R} \cdot \mathbf{H}_{f,\mathbf{x}}$.

The following statement can be proven in PV: For every $\mathbf{A} \in \mathbb{Z}_q^{n\times w}$, $\mathbf{x} \in \{0,1\}^\ell$, $\mathbf{R} \in \mathbb{Z}^{w\times \ell m}$, and $f : \{0,1\}^\ell \to \{0,1\}^{\ell'}$, it holds that

$$\mathsf{Eval}(f, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R})) = \mathsf{Enc}(\mathbf{A}, f(\mathbf{x}); \mathsf{REval}(f, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R}), \mathbf{R}, \mathbf{x})).$$

and that $\|\mathbf{R}_f\|_\infty \le \|\mathbf{R}\|_\infty \cdot \ell m^{\theta(d)}$.

The statement follows directly from Lemma B.1, as

$$(\mathbf{C} - \mathbf{x}\otimes\mathbf{G})\mathbf{H}_{f,\mathbf{x}} = \mathbf{CH}_f - f(\mathbf{x})^\intercal \otimes \mathbf{G} \implies \mathsf{Eval}(f, \mathbf{C}) = \mathbf{ARH}_{f,\mathbf{x}} + f(\mathbf{x})^\intercal \otimes \mathbf{G}.$$

Therefore, Claim B.3 can be accordingly proven in PV.

For the application for shiftable PRF, we need the notion of "packed ciphertext", which turns GSW ciphertexts encrypting messages as $\mathbf{AR} + \mathsf{Bit}(\mathbf{M}) \otimes \mathbf{G}$ into Regev style ciphertexts $\mathbf{AR} + \mathbf{M}$.

Lemma B.4 (Packed GSW Evaluation)**.** *Define the algorithm* $\mathsf{PackEval}$ *as follow*

- $\mathsf{PackEval}(F, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$*: Given* $\mathbb{Z}_q$*-function* $F : \{0,1\}^\ell \to \mathbb{Z}_q^{n\times\ell'}$*, define function* f' *such that* $f'(\mathbf{x}) = \mathsf{Bit}(F(\mathbf{x}))$*. Compute* $\mathsf{ct}'_f = \mathsf{Eval}(f', \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$ *using the GSW evaluation from Lemma B.3. Pack the ciphertext as* $\mathsf{ct}_F = \mathsf{ct}'_f \cdot \mathbf{P}_{n,\ell',q}$*, where* $\mathbf{P}_{n,\ell',q}$ *is the packing matrix. Output* ct_F*.*

The following statement can be proven in PV*: For every public key* $\mathbf{A}$ *from* $\mathsf{GSW.KeyGen}$*, every input* $\mathbf{x} \in \{0,1\}^\ell$*, every function* $F : \{0,1\}^\ell \to \mathbb{Z}_q^{n\times\ell'}$ *of depth* d*, and every encryption randomness* $\mathbf{R} \in \mathbb{Z}^{m\times\ell m}$*, it holds that*

$$\|\mathsf{sk}^\intercal \cdot \mathsf{PackEval}(F, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R})) - \mathsf{sk}^\intercal F(\mathbf{x})\|_\infty \le B \cdot \|\mathbf{R}\|_\infty \cdot \ell\ell' m^{\theta(d)}.$$

The lemma follows immediately following the observation that

$$\begin{aligned}\mathsf{PackEval}(F, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R})) &= \mathsf{Enc}(\mathbf{A}, f'(\mathbf{x}); \mathbf{R}_{f'}) \cdot \mathbf{P}_{n,\ell',q} \\ &= \mathbf{AR}_{f'}\mathbf{P}_{n,\ell',q} + \mathsf{Bit}(F(\mathbf{x}))^\intercal \otimes \mathbf{G} \cdot \mathbf{P}_{n,\ell',q} = \mathbf{AR}_{f'}\mathbf{P}_{n,\ell',q} + F(\mathbf{x})\end{aligned}$$

where the norm $\|\mathbf{R}_{f'}\|_\infty \le \|\mathbf{R}\|_\infty \cdot \ell\ell' m^{\theta(d)}$ follows from Claim B.3. Therefore, the decryption outcome

$$\mathsf{PackDec}(\mathsf{sk}, \mathsf{PackEval}(F, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R}))) = (-\mathbf{s}^\intercal, 1) \cdot (\mathbf{AR}_{f'}\mathbf{P}_{n,\ell',q} + F(\mathbf{x})) = \mathbf{e}^\intercal\mathbf{R}_{f'}\mathbf{P}_{n,\ell',q} + (-\mathbf{s}^\intercal, 1)F(\mathbf{x}),$$

with the error norm bounded by $B \cdot \|\mathbf{R}\|_\infty \cdot \ell\ell' m^{\theta(d)}$. When the function $F(\mathbf{x})$ is set to be zero on the top $n-1$ rows, i.e, $F(\mathbf{x}) = \begin{pmatrix}\mathbf{0}_{(n-1)\times\ell'} \\ f(\mathbf{x})^\intercal\end{pmatrix}$, we immediately have $\mathsf{sk}^\intercal\mathsf{ct}_F \approx f(\mathbf{x})^\intercal$.

The randomness homomorphism REval also extends to the packed evaluation. In particular, by setting $\mathsf{REval}(F, \mathsf{ct}_\mathbf{x}, \mathbf{R}, \mathbf{x}) = \mathsf{REval}(f', \mathsf{ct}_\mathbf{x}, \mathbf{R}, \mathbf{x}) \cdot \mathbf{P}_{n,\ell',q}$, we have that

$$\mathsf{PackEval}(F, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R})) = \mathbf{A} \cdot \mathsf{REval}(F, \mathsf{Enc}(\mathbf{A}, \mathbf{x}; \mathbf{R}), \mathbf{R}, \mathbf{x}) + F(\mathbf{x}).$$

B.4 SEH with proof of prefix consistency from LWE [HW15, MDS25]

In this section, we recall the SEH scheme with proof of prefix consistency [MDS25]. The presentation is almost identical to [MDS25], with some simplification/specification on node indices to better elaborate the PV friendliness. The following construction describes an SEH scheme with extraction size 1. It is known from [HW15] that larger extraction size can be obtained via parallel repetition of the scheme.

Lemma B.5 (SEH with proof of prefix consistency [MDS25])**.** *Let* $\mathsf{FHE} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *be a leveled homomorphic encryption scheme supporting evaluation of circuits of depth at most* $d > d_{\mathsf{Dec}} + 1$*, where* d_{Dec} *is the depth of its decryption circuit. Then, the following scheme is a SEH scheme with proof of prefix consistency.*

- $\mathsf{Gen}(1^\lambda, N)$*: Run* $(\mathsf{hk}, \mathsf{td}) \leftarrow \mathsf{TGen}(1^\lambda, N, \emptyset)$ *as defined below, and output* hk*.*
- $\mathsf{TGen}(1^\lambda, N, E = \{i\})$*: Without loss of generality assume* $N = 2^n$*. For* $j = [0, n]$*, sample FHE keys* $(\mathsf{pk}_j, \mathsf{sk}_j) \leftarrow \mathsf{KeyGen}(1^\lambda)$*. Let* $b_1 \dots b_n$ *be the binary representation of* i*. For* $j = [n]$*, compute ciphertext* $c_j = \mathsf{Enc}(\mathsf{pk}_j, (\mathsf{sk}_{j-1}, b_j))$*. Output* $\mathsf{hk} = (\mathsf{pk}_0, \dots, \mathsf{pk}_n, c_1, \dots, c_n)$ *and* $\mathsf{td} = \mathsf{sk}_n$*.*
- $\mathsf{Hash}(\mathsf{hk}, \mathbf{x} \in \{0,1\}^N)$*: Let* $\mathbf{x} = x_0 \dots x_{N-1}$*. Let* T *be a full binary tree of height* n*, with the leaves being at level* 0 *and root being on level* n*. We associate each node on level* ℓ *with an index* $i \in \{0,1\}^\ell$*, such that its children are indexed* $(i||0)$ *and* $(i||1)$*. We deterministically associate each node with a ciphertext from leaves to root as follows:*
  - *For each leaf node at level* 0 *with index* $i \in \{0,1\}^n$*, associate it with ciphertext* $\mathsf{ct}_i = \mathsf{Enc}(\mathsf{pk}_n, x_i; \mathbf{0})$ *encrypting with fixed randomness* $\mathbf{0}$*. Here, we treat the bit string* i *as the integer index of the bit in* $\mathbf{x}$*.*
  - *For each internal node at level* ℓ *with index* $i \in \{0,1\}^{n-\ell}$ *for* $\ell = 1, \dots, n$*, associate it with ciphertext* $\mathsf{ct}_i = \mathsf{Eval}(F_{\mathsf{ct}_{i||0}, \mathsf{ct}_{i||1}}, c_j)$*, where the function* $F_{\mathsf{ct}_{i||0}, \mathsf{ct}_{i||1}}$ *is defined by:*

    $F_{\mathsf{ct}_{i||0}, \mathsf{ct}_{i||1}}(\mathsf{sk}, b)$: *Compute* $\mu_0 = \mathsf{Dec}(\mathsf{sk}, \mathsf{ct}_{i||0})$ *and* $\mu_1 = \mathsf{Dec}(\mathsf{sk}, \mathsf{ct}_{i||1})$*. Output* μ_b*.*

  Output the ciphertext associated to the root node ct_ϵ *as the hash value.*
- $\mathsf{Open}(\mathsf{hk}, \mathbf{x}, i)$*: Compute the ciphertexts as in* $\mathsf{Hash}(\mathsf{hk}, \mathbf{x})$*. Let* $b_n \dots b_1$ *be the binary representation of* i*. Output the sequence of ciphertexts associated to the co-path of* i*, i.e,* $\rho_i = (\mathsf{ct}_{\bar{b}_n}, \mathsf{ct}_{b_n \bar{b}_{n-1}}, \dots, \mathsf{ct}_{b_n \dots b_2 \bar{b}_1})$ *as the local opening, where* $\bar{b}$ *denotes the negation of bit* b*.*
- $\mathsf{Ver}(\mathsf{hk}, \tau, i, y, \rho_i)$*: Let* $b_n \dots b_1$ *be the binary representation of* i*. Parse* $\rho_i = (\mathsf{ct}_{\bar{b}_n}, \mathsf{ct}_{b_n \bar{b}_{n-1}}, \dots, \mathsf{ct}_{b_n \dots b_2 \bar{b}_1})$*. Compute* $\mathsf{ct}_{b_n \dots b_1} = \mathsf{Enc}(\mathsf{pk}_n, y; \mathbf{0})$*. For* $\ell = 1, \dots, n$*, compute*

  $$\mathsf{ct}_{b_n \dots b_{\ell+1}} = \mathsf{Eval}(F_{\mathsf{ct}_{b_n \dots b_\ell||0}, \mathsf{ct}_{b_n \dots b_\ell||1}}, c_\ell).$$

  Finally, output 1 if $\mathsf{ct}_\epsilon = \tau$*, and output 0 otherwise.*
- $\mathsf{Ext}(\mathsf{hk}, \tau, \mathsf{td})$*: Decrypt* $\mu = \mathsf{Dec}(\mathsf{td}, \tau)$ *and output* μ*.*
- $\mathsf{ProveCons}(\mathsf{hk}, \mathbf{x}_1, \mathbf{x}_2, i)$*: Same as* $\mathsf{Open}(\mathsf{hk}, \mathbf{x}_1, i)$ *and* $\mathsf{Open}(\mathsf{hk}, \mathbf{x}_2, i)$*, compute the openings* $\rho_i^1 = (\mathsf{ct}^1_{\bar{b}_n}, \mathsf{ct}^1_{b_n \bar{b}_{n-1}}, \dots, \mathsf{ct}^1_{b_n \dots b_2 \bar{b}_1})$ *and* $\rho_{i,2} = (\mathsf{ct}^2_{\bar{b}_n}, \mathsf{ct}^2_{b_n \bar{b}_{n-1}}, \dots, \mathsf{ct}^2_{b_n \dots b_2 \bar{b}_1})$*. Output* $\pi_i = (\rho_i^1, \rho_i^2, x_i)$*, where* x_i *is the* i*-th bit of* $\mathbf{x}_1$*.*

- $\mathsf{VerCons}(\mathsf{hk}, \tau_1, \tau_2, i, \pi_i)$: *Parse* $\pi_i = (\rho_i^1, \rho_i^2, y)$ *where* $\rho_i^\beta = \left(\mathsf{ct}^\beta_{\overline{b}_n}, \mathsf{ct}^\beta_{b_n\overline{b}_{n-1}}, \ldots, \mathsf{ct}^\beta_{b_n\ldots b_2\overline{b}_1}\right)$ *for* $\beta = 1, 2$. *Verify the following:*
  - $\mathsf{Ver}(\mathsf{hk}, \tau_\beta, i, y_1, \rho_i^\beta) = 1$ *for* $\beta = 1, 2$.
  - *Let* $b_n \ldots b_1$ *be the binary representation of* i. *For each index* k *where* $b_k = 1$, *check that* $\mathsf{ct}^1_{b_n\ldots b_{k+1}\overline{b}_k} = \mathsf{ct}^2_{b_n\ldots b_{k+1}\overline{b}_k}$.

  Output 1 if all checks pass, and output 0 otherwise.

We refer the reader to [MDS25] for the security of the scheme assuming any FHE, and focus only on the PV-friendliness of the somewhere extractability and somewhere statistical soundness when instantiated with the GSW scheme.

Claim B.4. When instantiated with the GSW scheme in Lemma B.3, the somewhere extractability and somewhere statistical soundness of the SEH scheme in Lemma B.5 can be formulated and proved in PV.

The two properties can be formulated as an inductive statement as in [JKLM25]. For somewhere extractability, the proof boils down to showing that, if ciphertext $\mathsf{ct}_{b_n\ldots b_\ell}$ decrypts to μ under secret key $\mathsf{sk}_{\ell-1}$ and that the extraction element i has prefix $b_n \ldots b_\ell$, then the parent ciphertext $\mathsf{ct}_{b_n\ldots b_{\ell+1}}$ decrypts to μ under secret key sk_ℓ. The statement follows directly from the correctness of the homomorphic evaluation of the decryption circuit, which can be proved in PV. The somewhere statistical soundness follows the same proof strategy, with the additional observation that, for every i with bitwise representation $b_n \ldots b_1$ and any $j < i$, there exists some index k such that $b_k = 1$ and that $b_n \ldots b_{k+1}\overline{b}_k$ is a prefix of j. Therefore, to argue that the extraction of index j is consistent with both hashes, one simply start from the base case where the ciphertexts $\mathsf{ct}^1_{b_n\ldots b_{k+1}\overline{b}_k} = \mathsf{ct}^1_{b_n\ldots b_{k+1}\overline{b}_k}$ are equal, and inductively propagate the consistency up to the root.

B.5 Shiftable PRF with approximate correctness from LWE [PS18]

In this section, we recall the shiftable PRF construction from [PS18]. The construction is based on the BGG homomorphic gadget from Lemma B.1. For the purpose of later application of perfectly correct shiftable PRF, we demonstrate the construction with *approximate correctness*, i.e, the master evaluation with shift $\mathsf{Eval}(k, x) + f(x)$ and the shifted evaluation $\mathsf{Eval}(k_f, x)$ differs by some bounded error e.

Lemma B.6 (Shiftable PRF with approximate correctness [PS18])**.** *Let* $\mathsf{GSW} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *be the GSW homomorphic encryption scheme from Lemma B.3 with secret key in* $\mathbb{Z}_q^t$ *and ciphertext description length* c, *and let* $\mathsf{PackEval}$ *be the packed evaluation algorithm from Lemma B.4. Let* n, m *be lattice parameters, where* $m = n\lceil \log q \rceil$. *Let* χ *be a truncated discrete Gaussian error distribution with bound* B. *The following construction is a shiftable PRF of input space* $\{0,1\}^\ell$, *output space* $\mathbb{Z}_q^m$, *supporting circuits* $C : \{0,1\}^\ell \to \mathbb{Z}_q^{\ell'}$ *of size at most* s *and depth at most* d, *and satisfying approximate correctness with error bound* $\overline{B} = B \cdot \mathsf{poly}(s, \ell, \ell') \cdot (m+1)^{\theta(d)}$, *where* $\theta(d)$ *and* $\mathsf{poly}(s, \ell, \ell')$ *omits some fixed linear function/polynomials.*

- $\mathsf{Setup}(1^\lambda)$: *Sample a GSW key pair* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{GSW.KeyGen}(1^\lambda)$, *matrices* $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times cs \cdot m}$, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n \times tm}$, *and* $\mathbf{r} \leftarrow \mathbb{Z}_q^{n-1}$. *Set* $\mathbf{s}^\intercal = (\mathbf{r}^\intercal, 1)$, *output* $(\mathsf{pp} = (\mathbf{A}, \mathbf{B}, \mathsf{pk}), k = (\mathbf{s}, \mathsf{sk}))$.

- $\mathsf{Eval}(\mathsf{pp}, k, \mathbf{x} \in \{0,1\}^\ell)$: *Define the universal circuit* $\mathcal{U}_{\mathbf{x}}$*, which takes as input a circuit description* $f : \{0,1\}^\ell \to \mathbb{Z}_q^{\ell'}$ *of size at most* s*, and output the matrix* $\begin{pmatrix} \mathbf{0}_{(t-1)\times\ell'} \\ -f(\mathbf{x})^\intercal \end{pmatrix}$*. Define circuit* $\mathcal{V}_{\mathbf{x}}$ *to be the circuit* $\mathsf{GSW.PackEval}(\mathcal{U}_{\mathbf{x}}, \cdot)$ *with input space* $\{0,1\}^{cs}$ *and output space* $\mathbb{Z}_q^{t\times\ell'}$*. Compute* $\mathbf{H}_{\mathcal{V}_{\mathbf{x}}} = \mathsf{PEvalF}(\mathbf{A}, \mathbf{B}, \mathcal{V}_{\mathbf{x}})$ *from Lemma B.2. Set* $\mathbf{u}^\intercal = (0, \ldots, 0, 1) \in \mathbb{Z}_q^n$ *be the unit column vector with 1 on the last coordinate. Compute*

$$\mathbf{y}^\intercal = \mathbf{s}^\intercal \cdot \mathbf{A} \cdot \mathbf{H}_{\mathcal{V}_{\mathbf{x}}} \cdot (\mathbf{I}_{\ell'} \otimes \mathbf{G}^{-1}(\mathbf{u})) \in \mathbb{Z}_q^{1\times\ell'}.$$

and output $\mathbf{y}$*.*

- $\mathsf{Shift}(\mathsf{pp}, k, f)$: *Parse* f *as a bit vector of length* s*. Compute the GSW ciphertext* $\mathsf{ct}_f = \mathsf{Enc}(\mathsf{pk}, f)$*, where* $\mathsf{Bit}(\mathsf{ct}_f) \in \{0,1\}^{cs}$*. Compute the LWE sample*

$$\mathbf{v}^\intercal = \mathbf{s}^\intercal \left(\mathbf{A} - \mathsf{Bit}(\mathsf{ct}_f)^\intercal \otimes \mathbf{G} \middle| \mathbf{B} - \mathsf{sk}^\intercal \otimes \mathbf{G}\right) + \mathbf{e}^\intercal$$

where $\mathbf{e} \leftarrow \chi^{csm+tm}$ *is a* B*-bounded error. Output* $k_f = (\mathbf{v}, \mathsf{ct}_f)$*.*

- $\mathsf{EvalShifted}(\mathsf{pp}, k_f, \mathbf{x} \in \{0,1\}^\ell)$: *Parse* $k_f = (\mathbf{v}, \mathsf{ct}_f)$*. Define the universal circuit* $\mathcal{U}_{\mathbf{x}}$ *and* $\mathcal{V}_{\mathbf{x}}$ *as in* Eval*. Compute* $\mathbf{H}_{\mathcal{V}_{\mathbf{x}},\mathsf{ct}_f} = \mathsf{PEvalF}(\mathbf{A}, \mathbf{B}, \mathcal{V}_{\mathbf{x}}, \mathsf{ct}_f)$ *from Lemma B.2. Compute*

$$\mathbf{y}^\intercal = \mathbf{v}^\intercal \cdot \mathbf{H}_{\mathcal{V}_{\mathbf{x}},\mathsf{ct}_f} \cdot (\mathbf{I}_{\ell'} \otimes \mathbf{G}^{-1}(\mathbf{u})) \in \mathbb{Z}_q^{1\times\ell'}$$

and output $\mathbf{y}$*.*

Again, we refer the reader to [PS18] for the security of the scheme, and focus only on the PV-friendliness of the approximate correctness.

Claim B.5. The approximate correctness of the construction in Lemma B.6 can be proved in PV.

Proof. The proof is straightforward by expanding the definitions. Following the correctness of PEvalF and PEvalFX from Lemma B.2, we have that

$$\begin{aligned}
\mathbf{v}^\intercal \cdot \mathbf{H}_{\mathcal{V}_{\mathbf{x}},\mathsf{ct}_f} \cdot (\mathbf{I}_{\ell'} \otimes \mathbf{G}^{-1}(\mathbf{u})) &= \mathbf{s}^\intercal \left(\mathbf{A} - \mathsf{Bit}(\mathsf{ct}_f)^\intercal \otimes \mathbf{G} \middle| \mathbf{B} - \mathsf{sk}^\intercal \otimes \mathbf{G}\right) \mathbf{H}_{\mathcal{V}_{\mathbf{x}},\mathsf{ct}_f} \cdot (\mathbf{I}_{\ell'} \otimes \mathbf{G}^{-1}(\mathbf{u})) + \mathbf{e}'^\intercal \\
&= \mathbf{s}^\intercal (\mathbf{A}|\mathbf{B}) \cdot \mathbf{H}_{\mathcal{V}_{\mathbf{x}}} \cdot (\mathbf{I}_{\ell'} \otimes \mathbf{G}^{-1}(\mathbf{u})) - \mathsf{sk}^\intercal \cdot \mathcal{V}_{\mathbf{x}}(\mathsf{Bit}(\mathsf{ct}_f)) + \mathbf{e}'^\intercal \\
&= \mathsf{Eval}(\mathsf{pp}, k, \mathbf{x})^\intercal - \mathsf{sk}^\intercal \cdot \mathsf{PackEval}(\mathcal{U}_{\mathbf{x}}, \mathsf{ct}_f) + \mathbf{e}'^\intercal,
\end{aligned}$$

where $\mathbf{e}'^\intercal = \mathbf{e}^\intercal \mathbf{H}_{\mathcal{V}_{\mathbf{x}}} \cdot (\mathbf{I}_{\ell'} \otimes \mathbf{G}^{-1}(\mathbf{u}))$ has norm upper bounded by $B \cdot (csm + tm) \cdot s\ell' m^{\theta(d)} \cdot \ell' m = B \cdot \mathsf{poly}(s, \ell, \ell') \cdot m^{\theta(d)}$ following Equation (4). Finally, following the correctness of packed GSW evaluation from Lemma B.4, we have that

$$-\mathsf{sk}^\intercal \cdot \mathsf{PackEval}(\mathcal{U}_{\mathbf{x}}, \mathsf{ct}_f) = -\mathbf{e}_{\mathsf{GSW}} - \mathsf{sk} \cdot \mathcal{U}_{\mathbf{x}}(f) = -\mathbf{e}_{\mathsf{GSW}} + f(\mathbf{x})^\intercal,$$

with the error norm bounded by $B \cdot \ell\ell' m^{\theta(d)}$. Overall, we have that

$$\mathsf{EvalShifted}(\mathsf{pp}, k_f, \mathbf{x}) = \mathsf{Eval}(\mathsf{pp}, k, \mathbf{x}) + f(\mathbf{x}) + \mathbf{e},$$

where the error norm $\|\mathbf{e}\|_\infty \le \overline{B}$. □

Remark B.2 (Approximate correctness to negligible correctness)**.** In the shiftable PRF construction in Lemma B.6, the modulus q can be composite, and can be chosen large enough such that $q/\overline{B}$ is $2^{\mathsf{poly}(\lambda)}$ by correctly picking all the lattice parameters and assuming the corresponding hardness of LWE.

With the observation, it was shown in [PS18] that one can obtain negligible correctness error by "rounding" the output of the shiftable PRF. More specifically, pick modulus $q = p\Delta$ for output domain p and rounding parameter $\Delta \gg \overline{B}$. To get a shiftable PRF over $\mathbb{Z}_p$, one can use the same construction as in Lemma B.6, and define the output of both Eval and $\mathsf{EvalShifted}$ to be $\lfloor \mathbf{y}/\Delta \rceil \in \mathbb{Z}_p$, and define shift $\mathbb{Z}_p$ functions by shifting its Δ multiple. The approximate correctness now translates to

$$\begin{aligned}\left\lfloor \frac{\mathsf{EvalShifted}(\mathsf{pp}, k_f, \mathbf{x})}{\Delta} \right\rceil &= \left\lfloor \frac{\mathsf{Eval}(\mathsf{pp}, k, \mathbf{x}) + \Delta \cdot f(\mathbf{x}) + \mathbf{e}}{\Delta} \right\rceil \\ &= \left\lfloor \frac{\mathsf{Eval}(\mathsf{pp}, k, \mathbf{x}) + \mathbf{e}}{\Delta} \right\rceil + f(\mathbf{x}) \\ \text{(with high probability)} \ &= \left\lfloor \frac{\mathsf{Eval}(\mathsf{pp}, k, \mathbf{x})}{\Delta} \right\rceil + f(\mathbf{x}).\end{aligned}$$

Our construction of *perfectly correct* shiftable PRF in Construction 6.1 also uses the observation in Remark B.2. In particular, we choose modulus $q = p\Delta$, and $\Delta > 2^{m'(\lambda)}\overline{B}$, where there exists some correlation intractable hash function for efficiently searchable relations with output length $m'(\lambda)$.

B.6 Correlation intractable hash functions from GSW [PS19]

In this section, we recall the (base) construction of correlation intractable hash functions for circuits from [PS19].

Lemma B.7. *Let* $\mathsf{GSW} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *be the GSW homomorphic encryption scheme from Lemma B.3 with secret key in* $\mathbb{Z}_q^n$ *and bounded noise* B*, let* $\mathsf{PackEval}$ *be the packed evaluation algorithm from Lemma B.4. The following construction is a correlation intractable hash function for* $\mathcal{C}_{m,s,d} = \{C_\lambda\}$ *where* $m = n\lceil \log q \rceil$ *and depth bound* d *satisfies* $q/4B > s^2 m^{\theta(d)}$*.*

- $\mathsf{Gen}(1^\lambda) \to \mathsf{hk}$*: Run* $\mathsf{SGen}(1^\lambda, C)$ *for some arbitrary fixed circuit* $C \in \mathcal{C}_\lambda$*.*
- $\mathsf{SGen}(1^\lambda, C)$*: Sample GSW key pair* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{GSW.KeyGen}(1^\lambda)$*. Compute ciphertext* $\mathsf{ct}_C = \mathsf{Enc}(\mathsf{pk}, C)$*. Output* $\mathsf{hk} = \mathsf{ct}_C$*.*
- $\mathsf{Hash}(\mathsf{hk}, \mathbf{x})$*: Define the universal circuit* $\mathcal{U}_{\mathbf{x}}$ *with input space* $\{0,1\}^s$ *and output space* $\{0,1\}^m$ *which operates as follows:*
  - *Parse its input as a circuit description* C*. If the circuit* C *does not take* $|\mathbf{x}|$*-bit input, output* 0^m*.*
  - *Compute* $C(\mathbf{x}) \in \{0,1\}^m$*, output* $\mathbf{G} \cdot C(\mathbf{x}) + \lfloor q/2 \rceil \mathbf{u}$*, where* $\mathbf{u}^\intercal = (0, \ldots, 0, 1) \in \mathbb{Z}_q^n$ *is the unit column vector with 1 on the last coordinate.*

  Parse hk *as a GSW ciphertext* ct_C*. Compute* $\mathsf{ct} = \mathsf{PackEval}(\mathcal{U}_{\mathbf{x}}, \mathsf{ct}_C)$ *from Lemma B.4, where* $\mathsf{ct} \in \mathbb{Z}_q^n$*. Output* $\mathbf{G}^{-1}(\mathsf{ct}) \in \{0,1\}^m$ *as the hash value.*

Again, we refer the reader to [PS19] for the security of the scheme, and focus only on the PV-friendliness of its statistical soundness.

Claim B.6. The statistical soundness of the construction in Lemma B.7 can be proved in PV.

Proof. The correlation intractability follows largely from the correctness of the randomness evaluation REval from Claim B.3, which can be proved in PV. In particular, if we denote the GSW key pair as $\mathsf{pk} = \mathbf{A} = \begin{pmatrix} \overline{\mathbf{A}} \\ \mathbf{s}^\intercal \overline{\mathbf{A}} + \mathbf{e}^\intercal \end{pmatrix} \in \mathbb{Z}_q^{n \times w}$, $\mathsf{sk} = (-\mathbf{s}^\intercal, 1)^\intercal \in \mathbb{Z}_q^n$, and that $\mathsf{ct}_C = \mathsf{Enc}(\mathsf{pk}, C; R) = \mathbf{AR} + \mathsf{Bit}(C)^\intercal \otimes \mathbf{G}$ with $\mathbf{R} \in \{0,1\}^{w \times ms}$, then for any $\mathbf{x} \in \{0,1\}^\ell$ and every C with ℓ-bit input, it holds that

$$\mathsf{PackEval}(\mathcal{U}_\mathbf{x}, \mathsf{ct}_C) = \mathbf{AR}_{\mathcal{U}_\mathbf{x}} + \mathcal{U}_\mathbf{x}(C) = \mathbf{AR}_{\mathcal{U}_\mathbf{x}} + \mathbf{G} \cdot C(\mathbf{x}) + \lfloor q/2 \rceil \mathbf{u},$$

where $\mathbf{R}_{\mathcal{U}_\mathbf{x}} = \mathsf{REval}(\mathcal{U}_\mathbf{x}, \mathsf{ct}_C, \mathbf{R}, \mathsf{Bit}(C))$ is the homomorphically computed randomness with norm upper bounded by $s^2 \cdot m^{\theta(d)}$ following Claim B.3. Therefore, the hash output satisfies that

$$\mathsf{sk}^\intercal \cdot \mathbf{G} \cdot \mathsf{Hash}(\mathsf{hk}, \mathbf{x}) = (-\mathbf{s}^\intercal, 1)^\intercal \cdot (\mathbf{AR}_{\mathcal{U}_\mathbf{x}} + \mathbf{G} \cdot C(\mathbf{x}) + \lfloor q/2 \rceil \mathbf{u}) = \mathbf{e}^\intercal \mathbf{R}_{\mathcal{U}_\mathbf{x}} + \mathsf{sk}^\intercal \cdot \mathbf{G} \cdot C(\mathbf{x}) + \lfloor q/2 \rceil .$$

Since $\|\mathbf{e}^\intercal \mathbf{R}_{\mathcal{U}_\mathbf{x}}\|_\infty \le B \cdot s^2 \cdot m^{\theta(d)} < q/4$, it holds that $\mathbf{e}^\intercal \mathbf{R}_{\mathcal{U}_\mathbf{x}} \neq -\lfloor q/2 \rceil$. Therefore for every $\mathbf{x} \in \{0,1\}^\ell$,

$$\mathsf{sk}^\intercal \cdot \mathbf{G} \cdot \mathsf{Hash}(\mathsf{hk}, \mathbf{x}) \neq \mathsf{sk}^\intercal \cdot \mathbf{G} \cdot C(\mathbf{x}) \implies \mathsf{Hash}(\mathsf{hk}, \mathbf{x}) \neq C(\mathbf{x}),$$

which concludes the proof. □

Remark B.3 (On the Dependency between Hash Size m and Intractability Depth d)**.** From Lemma B.7, we obtain a correlation intractable hash function for circuits with depth bound d, but require the output size to be at least $m = n\lceil \log q \rceil$. (Note that the CI guarantee trivially extends when one enlarges the output space). This leads to a $d^{1+1/\delta}$ dependency on the output size m, as $\log q$ grows linearly in d, and $q \le 2^{n^\delta}$ for some $\delta < 1$ is required for reducing LWE from worst case lattice assumptions.

In [PS19], the authors show how to bootstrap the construction to obtain a CIH without the depth dependency on the minimal output size. For our application in Construction 6.1, we will be considering CIH with sufficiently large output space. Therefore, we adopt this base construction to simplify the presentation on PV provability.

B.7 Injective PRG with trapdoor [BPR12, MP12]

In this section, we recall the PRG construction from learning with rounding from [BPR12], where $\mathsf{PRG}(\mathbf{s}) = \lfloor \mathbf{s}^\intercal \mathbf{A} \rceil$. We show that by sampling the matrix $\mathbf{A}$ with a trapdoor, the PRG becomes injective and invertible, and that both properties can be proven in PV.

We start with the lemma for sampling Ajtai's lattice trapdoor.

Lemma B.8 ([Ajt99, MP12])**.** *Let n, m, q be lattice parameters, where $m \ge 3n \log q$. There exists an efficient probabilistic algorithm $\mathsf{TrapGen}(1^n, m, q)$ that outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ together with a trapdoor $\mathbf{T} \in \mathbb{Z}^{m \times m}$ such that*

- *The distribution of $\mathbf{A}$ is $\mathsf{negl}(n)$-close to uniform over $\mathbb{Z}_q^{n \times m}$.*
- *$\mathbf{T}$ is a full rank matrix in $\mathbb{Z}$ satisfying $\mathbf{AT} = \mathbf{0}^{n \times m} \mod q$ and $\|\mathbf{T}\|_\infty \le B$ for some polynomial $B(n, \log q) = \mathsf{poly}(n, \log q)$.*

We now plug the trapdoor sampling procedure into the PRG construction from [BPR12], obtaining a PRG scheme with an additional inversion algorithm, allowing proof of non-membership.

Construction B.1 (Injective PRG with trapdoor (Adapted from [BPR12]))**.** Let n, m, q be lattice parameters, where $m \geq 3n \log q$. Let $q/p \geq 2^{\omega(\log \lambda)}$ and $p \geq 4mB$, where B is the bound from Lemma B.8. One can construct an injective PRG with input space $\mathbb{Z}_q^n$ and output space $\mathbb{Z}_p^m$ as follows:

- $\mathsf{Setup}(1^\lambda)$: Sample $(\mathbf{A}, \mathbf{T}) \leftarrow \mathsf{TrapGen}(1^n, m, q)$ following Lemma B.8. Let $\mathbf{A} = (\mathbf{A}_0 | \mathbf{A}_1)$ where $\mathbf{A}_0 \in \mathbb{Z}_q^{n \times n}$. If $\mathbf{A}_0$ is not invertible mod q, use some fixed $(\mathbf{A}, \mathbf{T})$ where $\mathbf{A}_0$ is invertible. Output $\mathsf{pp} = \mathbf{A}$ and $\mathsf{td} = (\mathbf{A}_0^{-1} \in \mathbb{Z}_q^{n \times n}, \mathbf{T} \in \mathbb{Z}^{m \times m}, \mathbf{T}^{-1} \in \mathbb{Z}^{m \times m})$.
- $\mathsf{PRG}_{\mathsf{pp}}(\mathbf{s})$: Compute $\mathbf{y}^\intercal = \mathbf{s}^\intercal \mathbf{A} \in \mathbb{Z}_q^m$, parse it as an integer vector in $[-q/2, q/2)$, and output $\mathbf{z} = \left\lfloor \frac{p}{q} \cdot \mathbf{y} \right\rceil \in \mathbb{Z}_p^m$.
- $\mathsf{Inv}_{\mathsf{td}}(\mathbf{z})$: Compute $\mathbf{y}^* = \left\lfloor \frac{q}{p} \cdot \mathbf{z} \right\rceil$, compute

$$(\mathbf{e}^*)^\intercal = ((\mathbf{y}^*)^\intercal \cdot \mathbf{T} \bmod q) \cdot \mathbf{T}^{-1} \in \mathbb{Z}^{1 \times m}, \quad (\mathbf{s}^*)^\intercal = (\mathbf{y}^* - \mathbf{e}^* \bmod q)^\intercal \cdot \begin{pmatrix} \mathbf{A}_0^{-1} \\ \mathbf{0}_{(m-n) \times n} \end{pmatrix}.$$

  Output $\mathbf{s}^*$.

The security of the construction follows immediately from the fact that $\mathbf{A}$ is statistically close to uniform, and that the PRG is pseudorandom when $\mathbf{A}$ is uniform assuming the hardness of LWE ([BPR12]). We now show that the injectivity and invertibility of the PRG can be proved in PV.

Claim B.7. The injectivity of the PRG in Construction B.1 can be proved in PV. Furthermore, for every $\mathbf{z}$ not in the image of $\mathsf{PRG}_{\mathsf{pp}}$, there exists a PV proof for the statement $\mathbf{z} \notin \mathsf{Im}(\mathsf{PRG}_{\mathsf{pp}})$.

Proof. Note that both statements can be proved assuming the perfect invertibility of Inv. Injectivity would then follow by the fact that Inv is deterministic, and the non-membership proof would be the efficient evaluation $\mathsf{PRG}_{\mathsf{pp}}(\mathsf{Inv}_{\mathsf{td}}(\mathbf{z})) \neq \mathbf{z}$. We therefore focus on proving the correctness of Inv, which is essentially the following statement.

$$\begin{aligned} &\forall \mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{A}_0^{-1} \in \mathbb{Z}_q^{n \times n}, \mathbf{T} \in \mathbb{Z}^{m \times m}, \mathbf{T}^{-1} \in \mathbb{Z}^{m \times m}, \\ &\mathbf{A} \cdot \begin{pmatrix} \mathbf{A}_0^{-1} \\ \mathbf{0}_{(m-n) \times n} \end{pmatrix} = \mathbf{I}_n \pmod q \ \wedge \ \mathbf{A}\mathbf{T} = \mathbf{0}^{n \times m} \pmod q \ \wedge \ \mathbf{T}\mathbf{T}^{-1} = \mathbf{I}_m \ \wedge \ \|\mathbf{T}\|_\infty \leq B \\ &\qquad \implies \forall \mathbf{s} \in \mathbb{Z}_q^n, \mathsf{Inv}_{\mathsf{td}=(\mathbf{A}_0^{-1}, \mathbf{T}, \mathbf{T}^{-1})}\left(\mathsf{PRG}_{\mathsf{pp}=\mathbf{A}}(\mathbf{s})\right) = \mathbf{s}. \end{aligned}$$

The proof follows from basic arithmetic. In particular, for every $\mathbf{s} \in \mathbb{Z}_q^n$, by expanding the definition of Inv and PRG, we have that

$$\|\mathbf{y} - \mathbf{y}^*\|_\infty = \left\| \mathbf{y} - \left\lfloor \frac{q}{p} \cdot \left\lfloor \frac{p}{q} \cdot \mathbf{y} \right\rceil \right\rceil \right\|_\infty \leq \frac{q}{2p} + \frac{1}{2} \leq \frac{q}{4mB},$$

Therefore, by Equation (4), we can prove in PV that $\|(\mathbf{y} - \mathbf{y}^*)^\intercal \cdot \mathbf{T}\|_\infty \leq \frac{q}{4}$, implying that

$$\left((\mathbf{y}^* - \mathbf{y})^\intercal \cdot \mathbf{T} \bmod q\right) = (\mathbf{y}^* - \mathbf{y})^\intercal \cdot \mathbf{T}.$$

On the other hand, we have that $\mathbf{y}^\intercal \mathbf{T} = \mathbf{s}^\intercal \mathbf{A}\mathbf{T} = \mathbf{0}^{1 \times m} \pmod q$. Therefore,

$$\left((\mathbf{y}^* - \mathbf{y})^\intercal \cdot \mathbf{T} \bmod q\right) = \left((\mathbf{y}^*)^\intercal \cdot \mathbf{T} \bmod q\right).$$

Hence

$$(\mathbf{e}^*)^\intercal = ((\mathbf{y}^*)^\intercal \cdot \mathbf{T} \bmod q) \cdot \mathbf{T}^{-1} = (\mathbf{y}^* - \mathbf{y})^\intercal \cdot \mathbf{T} \cdot \mathbf{T}^{-1} = (\mathbf{y}^* - \mathbf{y})^\intercal.$$

By plugging in the final equation of Inv, we have that

$$\mathbf{s}^* = (\mathbf{y}^* - \mathbf{e}^* \bmod q)^\intercal \cdot \begin{pmatrix} \mathbf{A}_0^{-1} \\ \mathbf{0}_{(m-n)\times n} \end{pmatrix} = (\mathbf{y})^\intercal \cdot \begin{pmatrix} \mathbf{A}_0^{-1} \\ \mathbf{0}_{(m-n)\times n} \end{pmatrix} = \mathbf{s}^\intercal \mathbf{A} \cdot \begin{pmatrix} \mathbf{A}_0^{-1} \\ \mathbf{0}_{(m-n)\times n} \end{pmatrix} = \mathbf{s}^\intercal,$$

which concludes the proof. □